

Caching

02 January 2026 17:17

What is Caching?

Definition 1

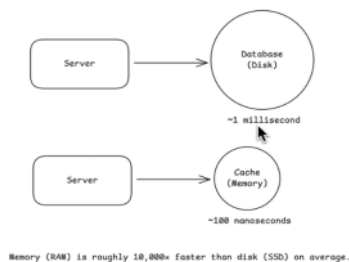
The cache is a high-speed data storage mechanism used to temporarily store frequently accessed or recently used data and instructions. It is situated between the main memory (RAM) and the central processing unit (CPU) in a computer system

Definition 2

A cache is a hardware or software component that stores data so that future requests for that data can be served faster; the data stored in a cache might be the result of an earlier computation or a copy of data stored elsewhere.

Definition 3

The cache is a high-speed data storage mechanism used to temporarily store frequently accessed or recently used data and instructions. **It is situated between the main memory (RAM) and the central processing unit (CPU) in a computer system.** By storing frequently accessed data closer to the CPU, cache enables quicker access times, reducing the latency associated with fetching data from slower main memory.



Why Caching?

Reading a user profile from **Postgres** may take **50 milliseconds**, but reading from an in-memory cache like **Redis** takes just **1 millisecond**. That's a 50x improvement in latency. **Databases store data on disk (mainly SSDs)**, and every query pays the cost of disk access. **Memory sits** much closer to the **CPU** and avoids that entirely.

When use cache?

- High Latency or Slow Data Access
- Frequent Access to the Same Data
- Expensive Computations or Queries
- Read-Heavy Workloads
- Scalability Under Load
- Network Bandwidth Constraints
- Temporary Data Availability
- Fault Tolerance or Offline Support

Cache technologies

In-Memory Caching Technologies

Redis: Fast in-memory key-value store, often used for real-time data processing and caching. Supports persistence.

Memcached: Simple and lightweight key-value store designed for high-performance caching.

Hazelcast: Distributed in-memory data grid, often used for application-level caching.

Apache Ignite: In-memory data fabric for caching and data processing.

Database Caching Technologies

MySQL Query Cache: Caches query results to reduce database load.

PostgreSQL Cache: Third-party extensions like `pg_bouncer` or `pgpool-II` are often used for caching.

Oracle Database Cache: Offers in-memory database caching for accelerating read queries.

Content Delivery Network (CDN) Caching

Cloudflare: CDN with caching capabilities for faster content delivery.

AWS CloudFront: Caches content across global edge locations.

Akamai: Provides caching for large-scale content delivery.

Fastly: Edge cloud platform for real-time caching and acceleration.

Application-Level Caching

Spring Cache: Abstraction in Spring Boot for caching method

results.

ASP.NET Cache: Built-in caching mechanism for .NET applications.

Guava Cache: In-memory caching for Java applications.

Caffeine Cache: High-performance Java caching library with time-based and size-based eviction policies.

Distributed Caching Technologies

Amazon ElastiCache: Managed Redis or Memcached service.

Azure Cache for Redis: Managed Redis cache for Azure workloads.

Google Cloud Memorystore: Fully managed Redis and Memcached.

NCache: .NET-based distributed caching platform.

Browser and Client-Side Caching

Browser Cache: Caches web pages, images, and other assets locally.

Service Workers: Client-side caching for offline functionality.

IndexedDB: Database storage in browsers for caching data-intensive apps.

Hybrid and Specialized Caching

Apache Cassandra: NoSQL database with built-in caching mechanisms.

Aerospike: High-performance, real-time database with built-in caching.

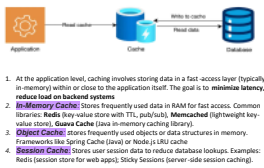
Varnish: HTTP accelerator primarily used for caching web content.

Squid: Proxy cache for web traffic optimization.

Types of Cache

- 1. Application level Cache
- 2. Database Caching
- 3. Distributed Caching
- 4. Web Caching
- 5. Hardware level Caching
- 6. DNS Caching

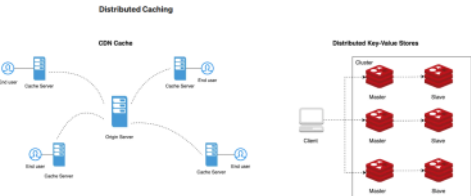
Application-Level Caching



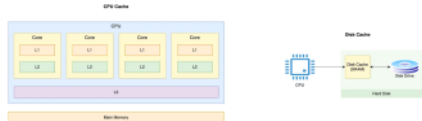
Database

- 1. Database Caching (e.g., an in-memory cache for database queries)
- 2. Query Cache
- 3. Row-Level Cache
- 4. Full-Page Cache
- 5. Write-Through Caching
- 6. Write-Behind Caching

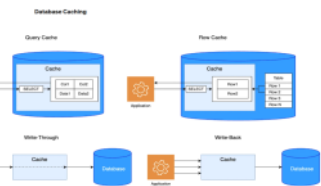
Distributed Caching



Hardware-Level Caching



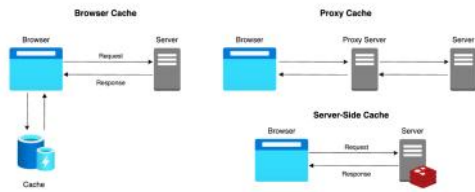
Caching



Caching involves keeping copies of database query results, rows, or objects in a cache memory store like **Redis** or **Memcached**, or even the **database's own built-in cache**.

- QEC**: Stores the results of expensive database queries. Example: MySQL Query Cache (in MySQL 5.6).
- RC**: Caches individual database rows in memory. Example: Hibernate Second-Level Cache.
- Full-Page Cache**: Stores entire database-generated pages for faster retrieval. Example: Full-Page Cache.
- Write-Through Cache**: Writes data to the cache and database at the same time. Ensures that data is always up-to-date.
- Write-Back Cache**: Writes data to cache first, then asynchronously updates the database. Improves write performance but risks data loss.

Web Caching



1. Web caching involves temporarily saving web data at various points between the user and the origin server—such as in the browser, a proxy server, or a content delivery network (CDN).
2. **Browser Cache**: Stores static assets in a user's browser. Example: **Cache-Control** headers in HTTP.
3. **Proxy Cache**: Stores HTTP responses between client and server. Example: **Varnish Cache**, **Squid Proxy**.
4. **Server-Side Cache**: Stores generated HTML to avoid repeated computation. Example: **Nginx FastCGI Cache**.

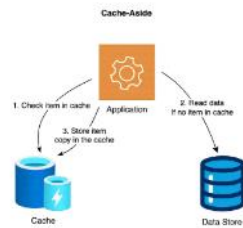
DNS Caching



1. DNS cache, also known as a resolver cache, is a mechanism used to temporarily store recently resolved domain name-to-IP address mappings locally on client devices or DNS servers.
2. **Client-Side DNS Cache**: Client devices, such as computers, smartphones, and routers, maintain their own local DNS caches. These caches store resolved DNS records for recently visited domain names, speeding up subsequent DNS lookups and improving browsing performance.
3. **DNS Server Cache**: DNS servers, including recursive DNS servers operated by Internet service providers (ISPs) and public DNS resolvers like Google Public DNS and OpenDNS, maintain their own DNS caches. These caches store resolved DNS records on behalf of multiple clients, reducing DNS resolution latency and improving overall network performance.

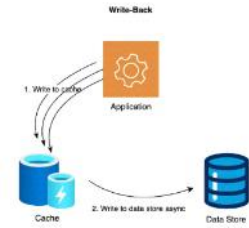
Types of Cache Strategies

1. Cache Aside
2. Write Through
3. Write Back
4. Read Through
5. Pre Fetching



Cache-Aside (Lazy Loading): The application checks the cache first. On a miss, it fetches data from the source (e.g., database) and manually stores it in the cache.

- **Pros:** Simple, app controls what's cached.
- **Cons:** Risk of stale data; cache misses slow down first requests.
- **Use Case:** App-level caching with Redis (e.g., "check Redis, if empty, query DB and set").



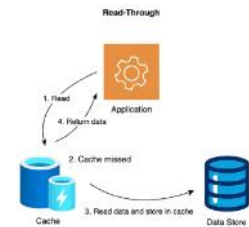
Write-Back (Write-Behind): Data is written to the cache first, then asynchronously synced to the backing store later.

- **Pros:** Faster writes; reduces load on the source.
- **Cons:** Risk of data loss if cache fails before sync.
- **Use Case:** Disk controllers, distributed caches with eventual consistency.



Write-Through: Data is written to both the cache and the backing store (e.g., database) simultaneously.

- **Pros:** Cache stays consistent with the source; no stale data.
- **Cons:** Slower writes due to dual updates.
- **Use Case:** Hardware caches (e.g., CPU L1) or systems needing strong consistency.



Read-Through: The cache itself fetches data from the source on a miss, transparently to the app.

- **Pros:** Simplifies app logic; cache handles loading.
- **Cons:** Tighter coupling between cache and source.
- **Use Case:** Some ORMs or caching libraries (e.g., Hibernate second-level cache).

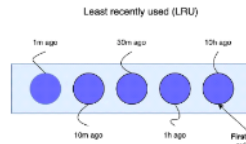


Pre-Fetching (Proactive Loading): Predictively load data into the cache before it's requested, based on patterns or locality.

- **Pros:** Reduces misses; speeds up access.
- **Cons:** Wastes space if predictions are wrong.
- **Use Case:** CPU caches (spatial locality), CDNs pre-loading assets.

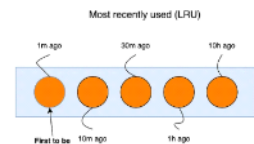
Cache Eviction Strategies

1. Least Recently Used (LRU)
2. Most Recently
3. Least Frequently Used (LFU)
4. First-In, First-Out (FIFO)
5. Time-to-Live (TTL)
6. Random Eviction



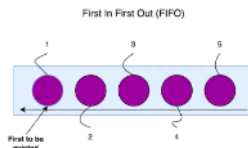
Least Recently Used (LRU): Evict the least recently accessed item.

- **Pros:** Good for workloads with temporal locality.
- **Cons:** Overhead to track usage order.
- **Use Case:** Redis, CPU caches.
- **When to Use:** When you have workloads with temporal locality, meaning recently accessed items are likely to be used again in the near future



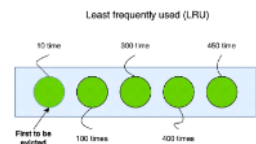
Most Recently Used (MRU): Evict the most recently accessed item.

- **Pros:** Useful when recent data is less likely to be reused.
- **Cons:** Rare in practice.
- **Use Case:** Niche streaming apps.
- **When to Use:** When recent data is less likely to be reused soon after access, making eviction of recent data more beneficial.



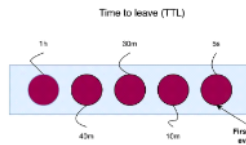
First-In, First-Out (FIFO): Evict the oldest item, regardless of usage.

- **Pros:** Simple to implement.
- **Cons:** Ignores access patterns.
- **Use Case:** Basic queues or hardware buffers.
- **When to Use:** When you don't care about the usage patterns of the items and just need to evict the oldest data in a predictable way.



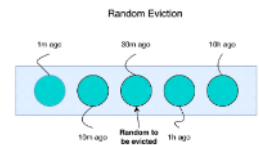
Least Frequently Used (LFU): Evict the least frequently accessed item.

- **Pros:** Prioritizes heavily used data.
- **Cons:** Needs frequency tracking; slow to adapt to changing patterns.
- **Use Case:** Web caches with stable access patterns.
- **When to Use:** When you want to prioritize data that is frequently accessed over time, and the access pattern is relatively stable.



Time-to-Live (TTL): Data expires after a set time, regardless of usage.

- **Pros:** Predictable expiration; prevents staleness.
- **Cons:** May evict useful data too soon.
- **Use Case:** Web caching (HTTP Cache-Control), Redis with expiration.
- **When to Use:** When you need data to expire after a fixed period, regardless of usage, to avoid serving stale or outdated data.

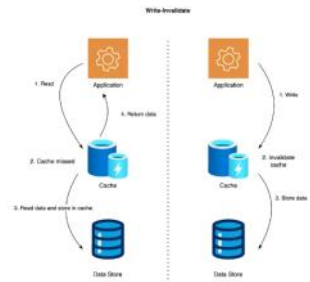


Random Eviction: Evict a random item.

- **Pros:** Very simple; low overhead.
- **Cons:** Unpredictable performance.
- **Use Case:** Lightweight systems or as a fallback.
- **When to Use:** When simplicity and low overhead are more important than eviction precision, or when eviction can be done with minimal performance concerns.

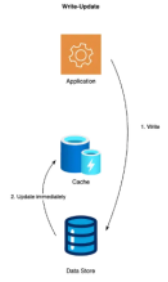
Cache Consistency Strategies

- 1. Write-Invalidate
- 2. Write-Update
- 3. Eventual Consistency
- 4. Strong Consistency



Write-Invalidate: When data is updated in the source, invalidate (remove) the corresponding cache entry.

- Pros: Ensures fresh data on next read.
- Cons: Next read causes a miss.
- Use Case: Distributed caches (e.g., invalidate Redis key on DB update).



Write-Update: Update the cache immediately when the source changes.

- Pros: Cache stays fresh; no misses after updates.
- Cons: Higher overhead to propagate updates.
- Use Case: Multi-core CPU cache coherence (e.g., MESI protocol).



Eventual Consistency: Caching refers to a strategy where data in the cache and the underlying data source (like a database) may not be immediately synchronized, but they will become consistent over time.

- Pros: Scales well; tolerates delays.
- Cons: Temporary staleness.
- Use Case: Distributed systems like CDNs.



Strong Consistency: Cache and source are always in sync (e.g., via write-through or locking).

- Pros: No stale data.
- Cons: Slower; less scalable.
- Use Case: Critical systems (e.g., financial apps).

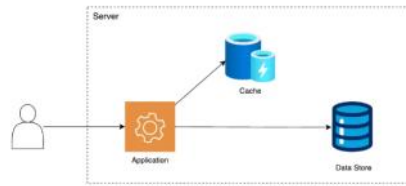
Cache Placement Strategies

03 January 2026 18:10

Cache Placement Strategies

- Local Caching
- Distributed Caching
- Edge Caching

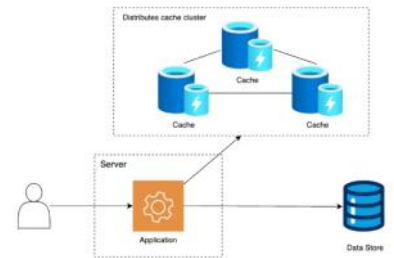
Local Caching



Local Caching: Cache is on the same machine or process as the app.

- **Pros:** Fastest access; no network.
- **Cons:** Limited size; not shared.
- **Use Case:** CPU caches, app-level in-memory stores.

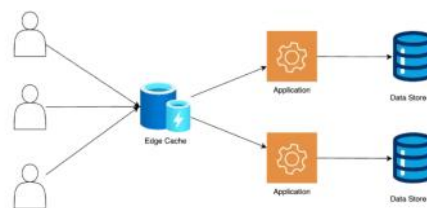
Distributed Caching



Distributed Caching: Cache spans multiple nodes in a cluster.

- **Pros:** Scalable; shared across apps.
- **Cons:** Network latency; complex.
- **Use Case:** Redis Cluster, Memcached in microservices.

Edge Caching



Edge Caching: Cache is near the user (e.g., CDN edge nodes).

- **Pros:** Reduces latency for end users.
- **Cons:** Harder to invalidate globally.
- **Use Case:** Web caching with Cloudflare.

Specialized Strategies : These are tailored to specific domains or needs.

Opportunistic Caching:

It's a "take advantage of the opportunity" approach — whenever data is fetched, the system stores it in the cache in case it's needed again soon.

Use Case: Mobile apps caching data during syncs.

Adaptive Caching:

It uses algorithms and monitoring to optimize cache size, eviction policies, or data retention dynamically.

Use Case: Advanced systems like databases (PostgreSQL buffer manager).

Negative Caching:

Caching strategy where the system caches failed or negative responses (e.g., "not found" or "error").

Use Case: DNS caching, web proxies.

Query Result Caching:

When you want to cache the results of expensive or frequent database queries

Use Case: SQL databases, web applications with repetitive queries.

Geospatial Caching:

When you need to cache geospatial data for faster retrieval of location-based queries, especially in systems dealing with proximity searches, maps, or location-based services.

Use Case: Location-based services, ride-sharing apps, geospatial search engines.

Cache Trade Offs

03 January 2026 18:48

Cache Trade-offs

Speed vs. Accuracy :

Caching improves data retrieval speed but the data could be outdated if not updated frequently

Memory Usage vs. Cache Size:

More data in the cache means faster access times for a broader range of queries but too much data may lead to **Higher Memory Overhead**

Cache Hit Ratio vs. Eviction Policies:

Cache hit ratio directly impacts the effectiveness of the cache. More cache hit means more eviction In order to maintain accurate data leading to **Eviction Overhead**

Data Freshness vs. Caching Duration:

Longer Caching Duration, more chances of Stale Data

Write Performance vs. Read Performance:

While cache can improve read performance,
Every time data is written to the cache, it is also written to the underlying storage. This can slow down write operations (in case of write-through)

Cache Invalidation Complexity:

Effective Invalidation Ensures that data is always up-to-date, but invalidation logic can be complicated to implement and could introduce additional overhead.

Simplicity vs. Sophistication of Caching Strategies:

Simple caching strategies (like **FIFO** or **Random Eviction**) are easy to implement but may not be as effective as more sophisticated ones (like **LRU** or **LFU**) in specific use cases

Latency vs. Cache Update Latency:

Caching strategies often aim to reduce latency, but the process of updating the cache itself may introduce delays

Distributed Caching vs. Local Caching:

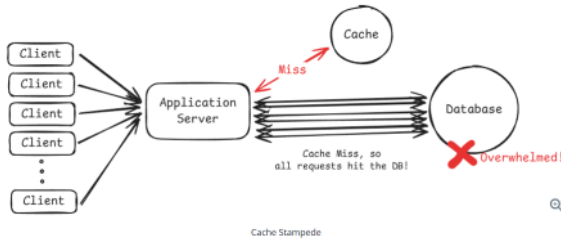
Distributed caches provide shared cache access across multiple servers, improving scalability and consistency across nodes, but they introduce network overhead and potential latency. **Local Caching**: Faster due to no network overhead, but the cache is not shared across servers, limiting scalability.

Common Caching Problems

03 January 2026 19:02

Cache Stampede (Thundering Herd)

A cache stampede happens when a popular cache entry expires and many requests try to rebuild it at the same time. There is a brief window, even if only a second, where every request misses the cache and goes straight to the database. Instead of one query, you suddenly have hundreds or thousands, which can overload the database.



For example, imagine your system caches the homepage feed with a TTL of 60 seconds. When the cache expires at exactly 12:01:00, every request at that moment misses the cache and queries the database. If traffic is high, this spike can overwhelm the database and cause cascading failures.

How to handle it:

- **Request coalescing (single flight):** Allow only one request to rebuild the cache while others wait for the result. This is the most effective solution.
- **Cache warming:** Refresh popular keys proactively before they expire. This only helps when using TTL-based expiration. If you invalidate cache on writes instead, warming does not prevent stampedes.

Cache Penetration

Cache penetration is a caching problem where **requests repeatedly query data that does not exist**, causing those requests to **bypass the cache and hit the database every time**, which can overwhelm the backend.

Normally:

Request → Cache → Database (only on cache miss)

With cache penetration:

Request (non-existent key)
→ Cache miss
→ Database query (returns null)
→ Result not cached
→ Next request repeats the same process

Attackers or bugs can exploit this by sending **many requests for invalid IDs**, forcing constant database access.

How to handle it:

- **Cache empty (null) results**: Store a placeholder for non-existent data.
- Use a Bloom Filter (best practice)
- Stores **all valid keys**
- Quickly checks if a key *might exist*
- Blocks invalid requests **before hitting cache or DB**
Request → Bloom Filter → Cache → Database
- Rate limiting

Cache Consistency

Cache consistency problems are some of the most commonly discussed in system design interviews. They happen when the cache and database return different values for the same data. This is common because most systems read from the cache but write to the database first. That creates a window where the cache still holds stale data.

For example, if a user updates their profile picture, the new value is written to the database but the old value might still be in the cache. Other users may see the outdated profile picture until the cache eventually refreshes.

There is no perfect solution. You choose a strategy based on how fresh the data must be.

How to handle it:

- **Cache invalidation on writes:** Delete the cache entry after updating the database so it gets repopulated with fresh data.
- **Short TTLs for stale tolerance:** Let slightly stale data live temporarily if eventual consistency is acceptable.
- **Accept eventual consistency:** For feeds, metrics, and analytics, a short delay is usually fine.

Hot Keys

A hot key is a cache entry that receives a huge amount of traffic compared to everything else. Even if the cache hit rate is high, a single hot key can overload one cache node or one Redis shard and become a bottleneck.

For example, if you are building Twitter and everyone is viewing Taylor Swift's profile, the cache key for her user data (user:taylorswift) may receive millions of requests per second. That one key can overload a single Redis node even though everything is working "correctly."

How to handle it:

- **Replicate hot keys:** Store the same value on multiple cache nodes and load balance reads across them.
- **Add a local fallback cache:** Keep extremely hot values in-process to avoid pounding Redis.
- **Apply rate limiting:** Slow down abusive traffic patterns on specific keys.

Big Key

A **Big Key** (or **large key**) is a term mainly used in **Redis and other key-value stores** to describe a key whose **value is excessively large**, either in **memory size** or in **number of elements**.

What is a Big Key?

A key is considered a **Big Key** if:

- The **value size is very large** (e.g., MBs of data)
- OR the **number of elements is huge**
 - List with millions of items
 - Set / ZSet with millions of members
 - Hash with too many fields

There is no strict standard, but commonly:

- Value size > **1 MB**
- Element count > **10,000**

Why are Big Keys dangerous?

Memory issues

- Consumes large amounts of Redis memory
- Causes uneven memory usage
- Increases eviction pressure

Performance problems

- Operations become **slow or blocking**

Network latency

- Transferring large values increases bandwidth usage
- Slower responses

Replication & persistence impact

- Slow RDB/AOF persistence
- Replication lag

Common Big Key scenarios

- Caching **entire objects or pages**
- Using one key to store **all user data**
- Time-series data in a single list
- Logs stored under one key

How to handle it:

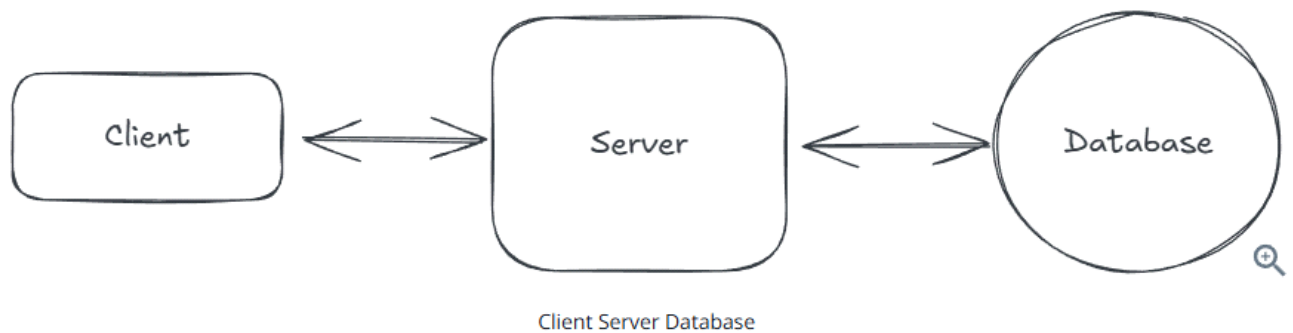
- **Split the key (sharding)**
 - ✗ user:all
 - ☑ user:1001, user:1002
- **Use expiration (TTL)**
- **Use UNLINK instead of DEL**: Deletes asynchronously (non-blocking)
- **Compress data**
 - Store compressed JSON / binary

Consistent Hashing

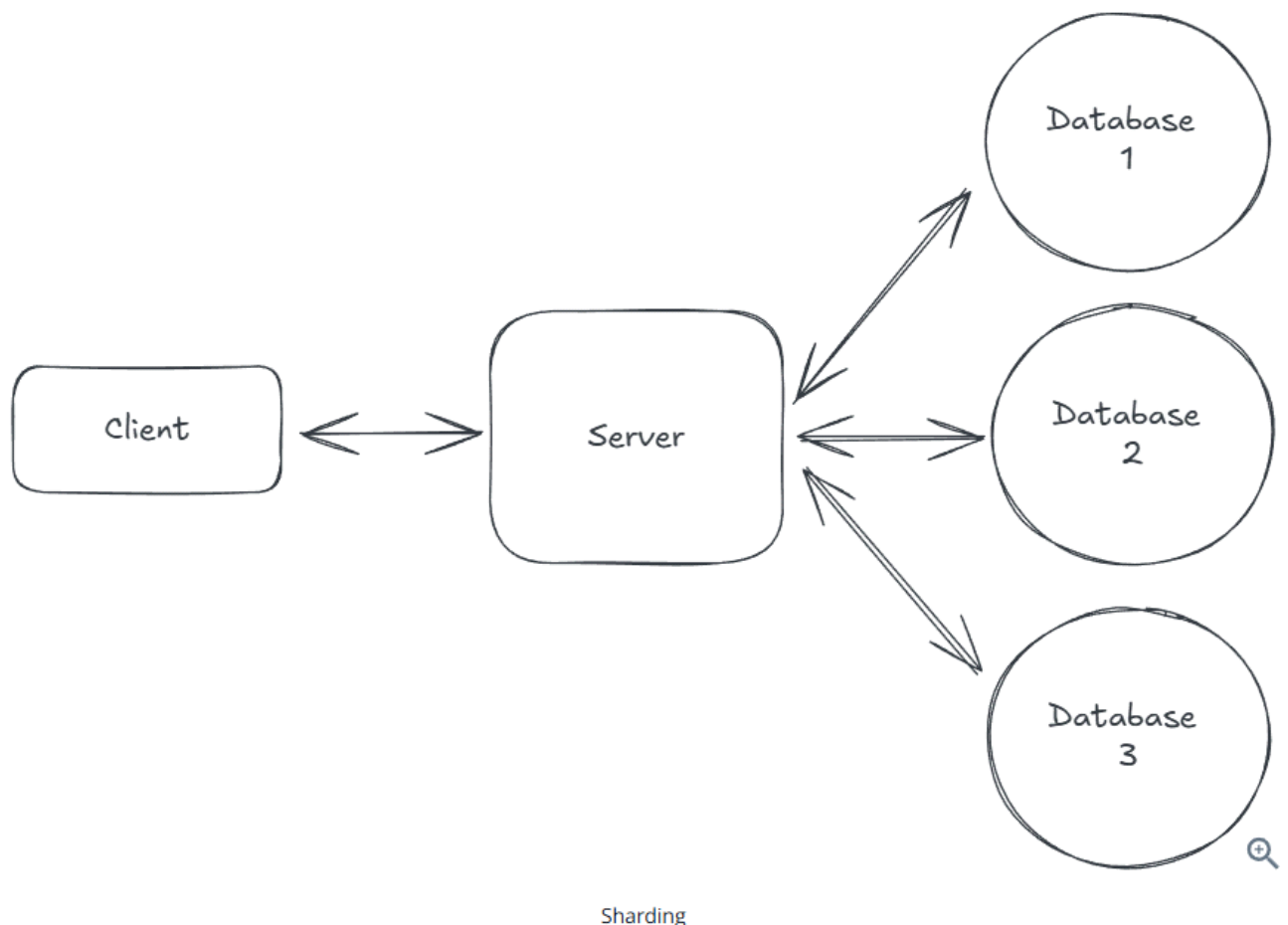
10 January 2026 21:13

Imagine you're designing a ticketing system like TicketMaster. Initially, your system is simple:

- One database storing all event data
- Clients making requests to fetch event information
- Everything works smoothly at first



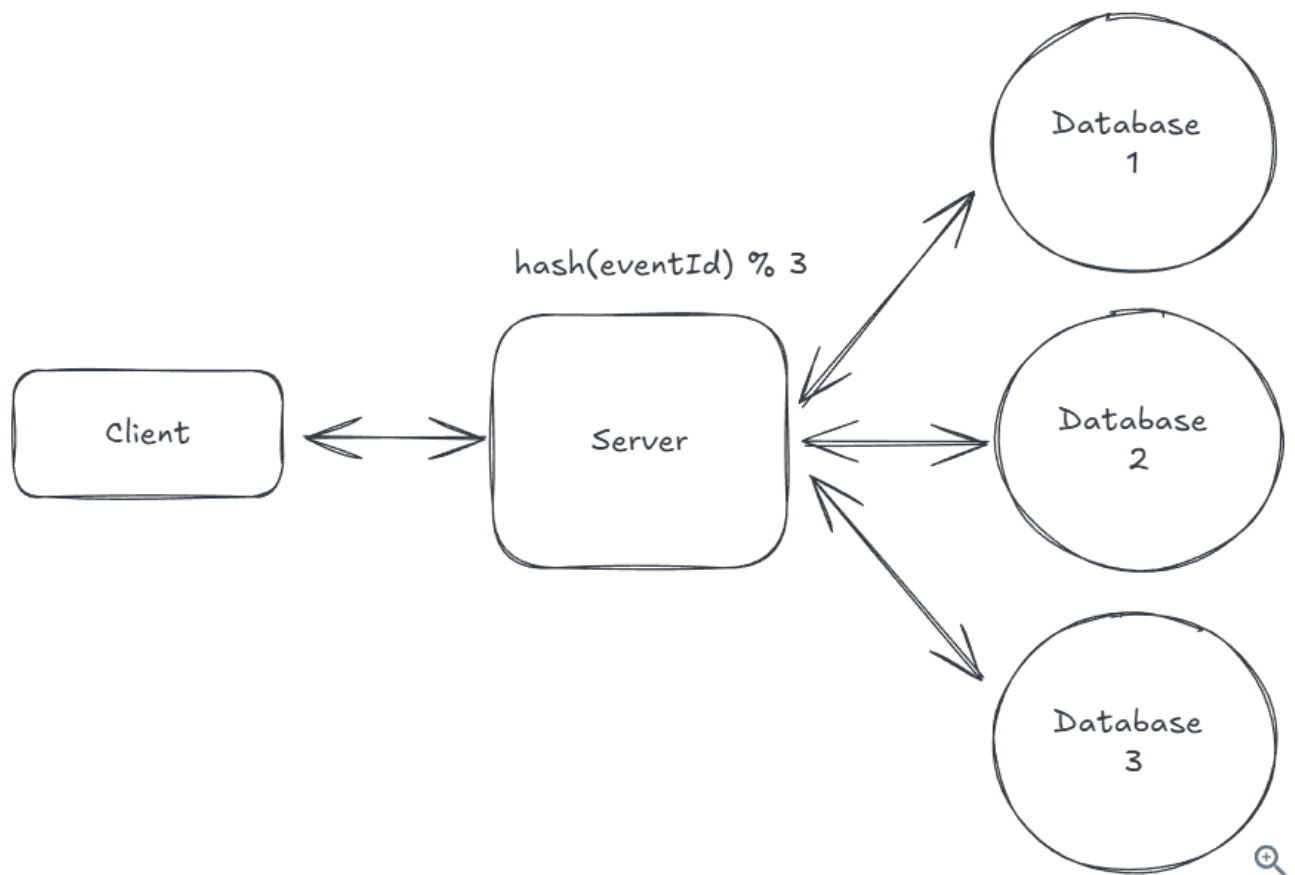
As your platform grows popular and hosts more events, a single database can no longer handle the load. You need to distribute your data across multiple databases – a process called [sharding](#).



How do we know which events to store on which database instance?

First Attempt: Simple Modulo Hashing

1. First, we take the event ID and run it through a hash function, which converts it into a number
2. Then, we take that number and perform the modulo operation (%) with the number of databases
3. The result tells us which database should store that event



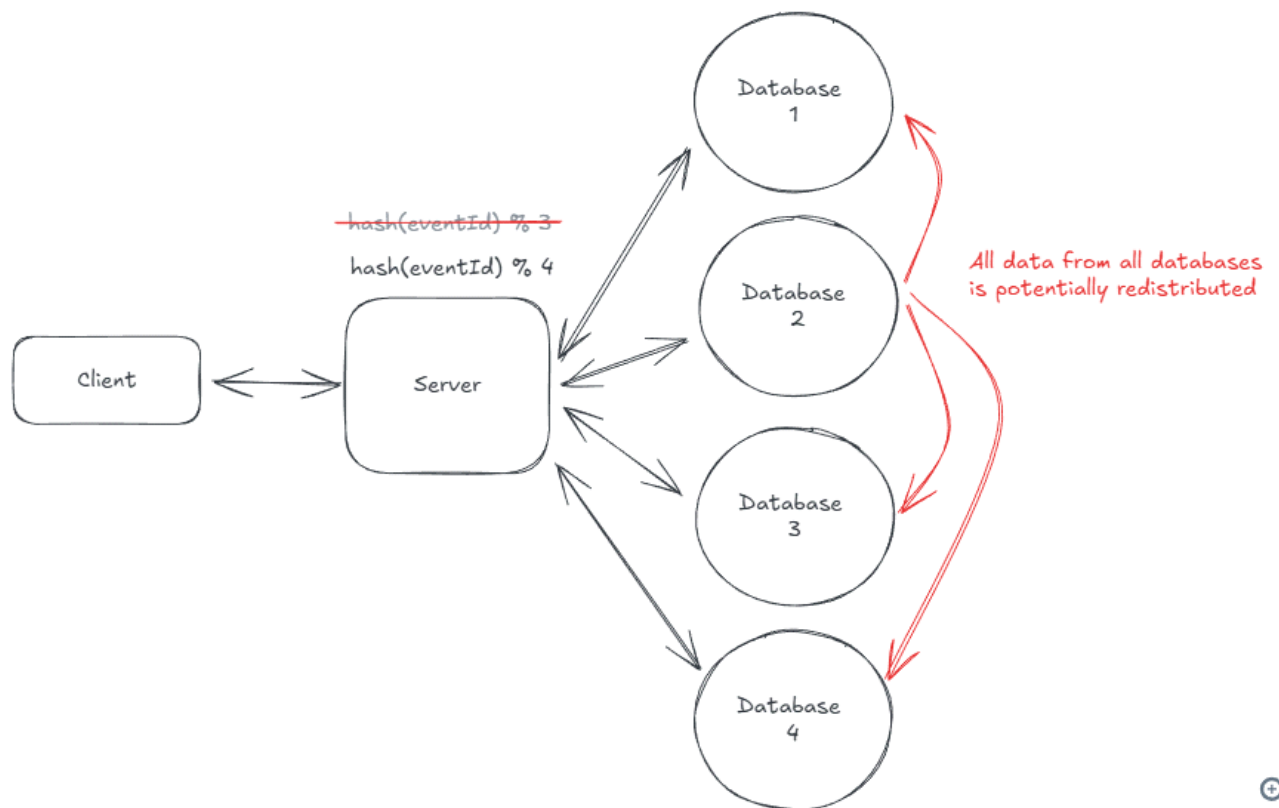
Modulo Hashing

```
database_id = hash(event_id) % number_of_databases
```

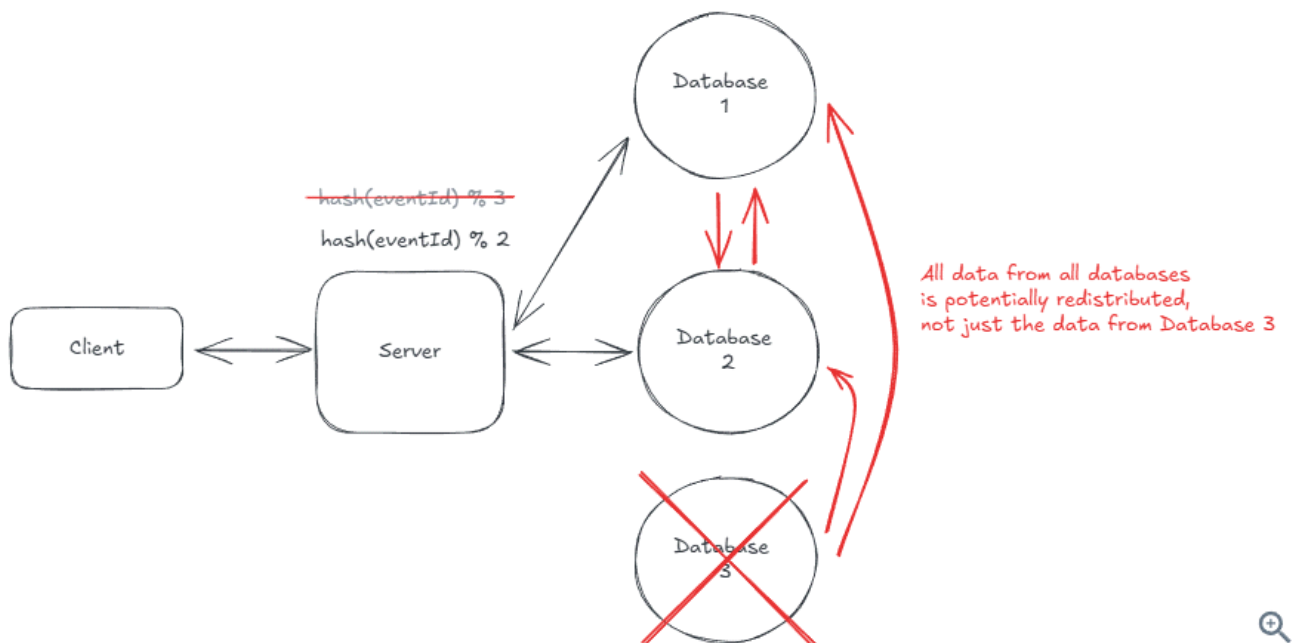
For a concrete example with 3 databases:

```
Event #1234 → hash(1234) % 3 = 1 → Database 1  
Event #5678 → hash(5678) % 3 = 0 → Database 0  
Event #9012 → hash(9012) % 3 = 2 → Database 2
```

Now If you add a fourth server then the number of databases would become 4 from 3..The hashing function $\%4$ would change the outcome, which will affect the rest of the data and also create problem in accessing the old data.



Issue adding a Node



Issue removing a Node

Consistent Hashing

Consistent hashing is a technique that solves the problem of data redistribution when adding or removing an instance in a distributed system. The key insight is to arrange both our data and our databases in a circular space, often called a "hash ring."

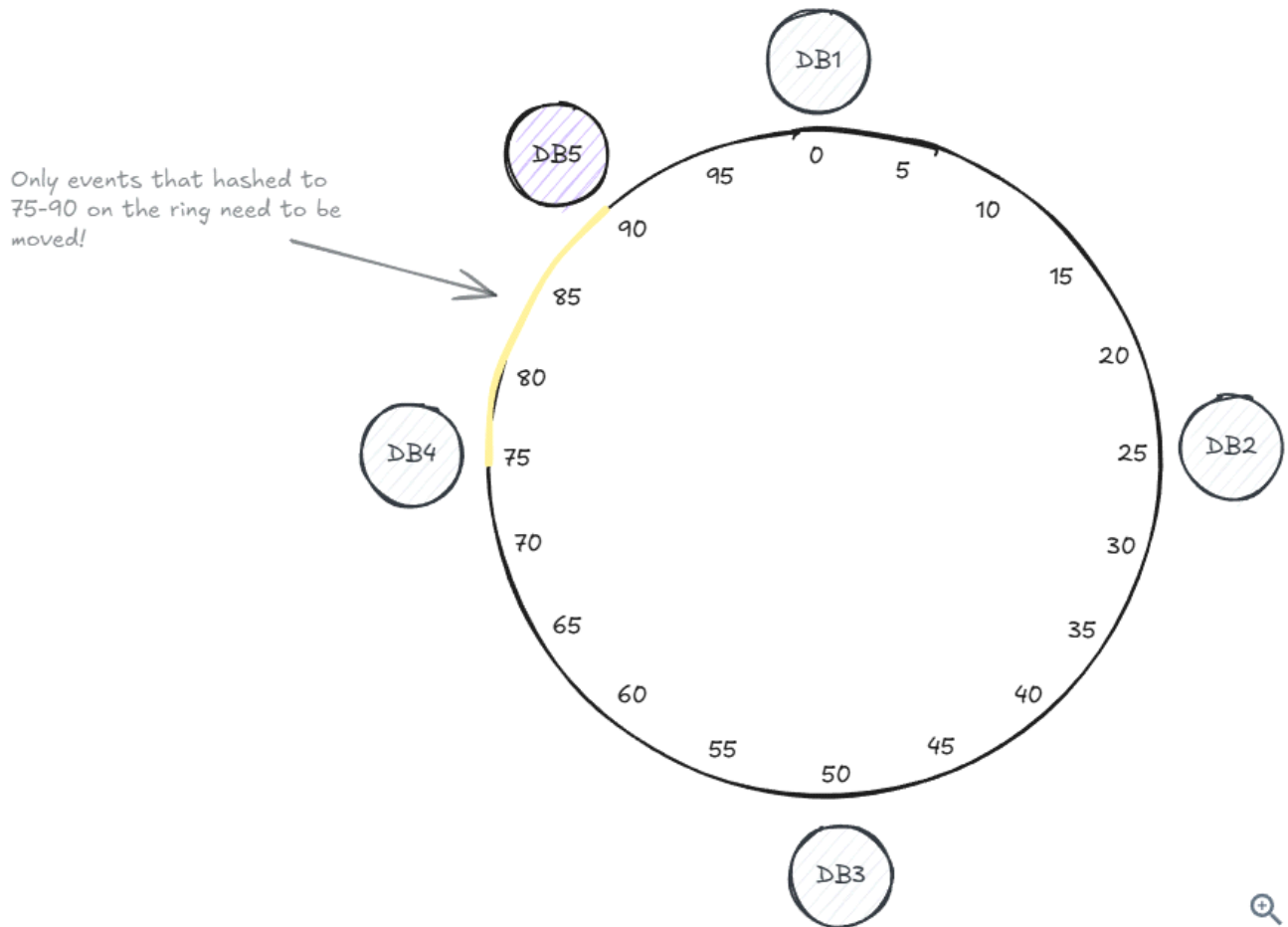
Here's how it works:

1. We first create a hash ring with a fixed number of points. To keep it simple, let's say

- 100.
2. We then evenly distribute our data across the hash ring. In the case where we have 4 databases, then we could put them at points 0, 25, 50, and 75.
3. In order to know which database an event should be stored on, we first hash the event ID like we did before, but instead of using modulo, we just find the hash value on the ring and then move clockwise until we find a database instance.

Adding a Database (Database 5) Let's say we add a fifth database at position 90 on our ring. Now:

- Only events that hash to positions between 75 and 90 need to move
- These events were previously going to DB1 (at position 0)
- All other events stay exactly where they are!

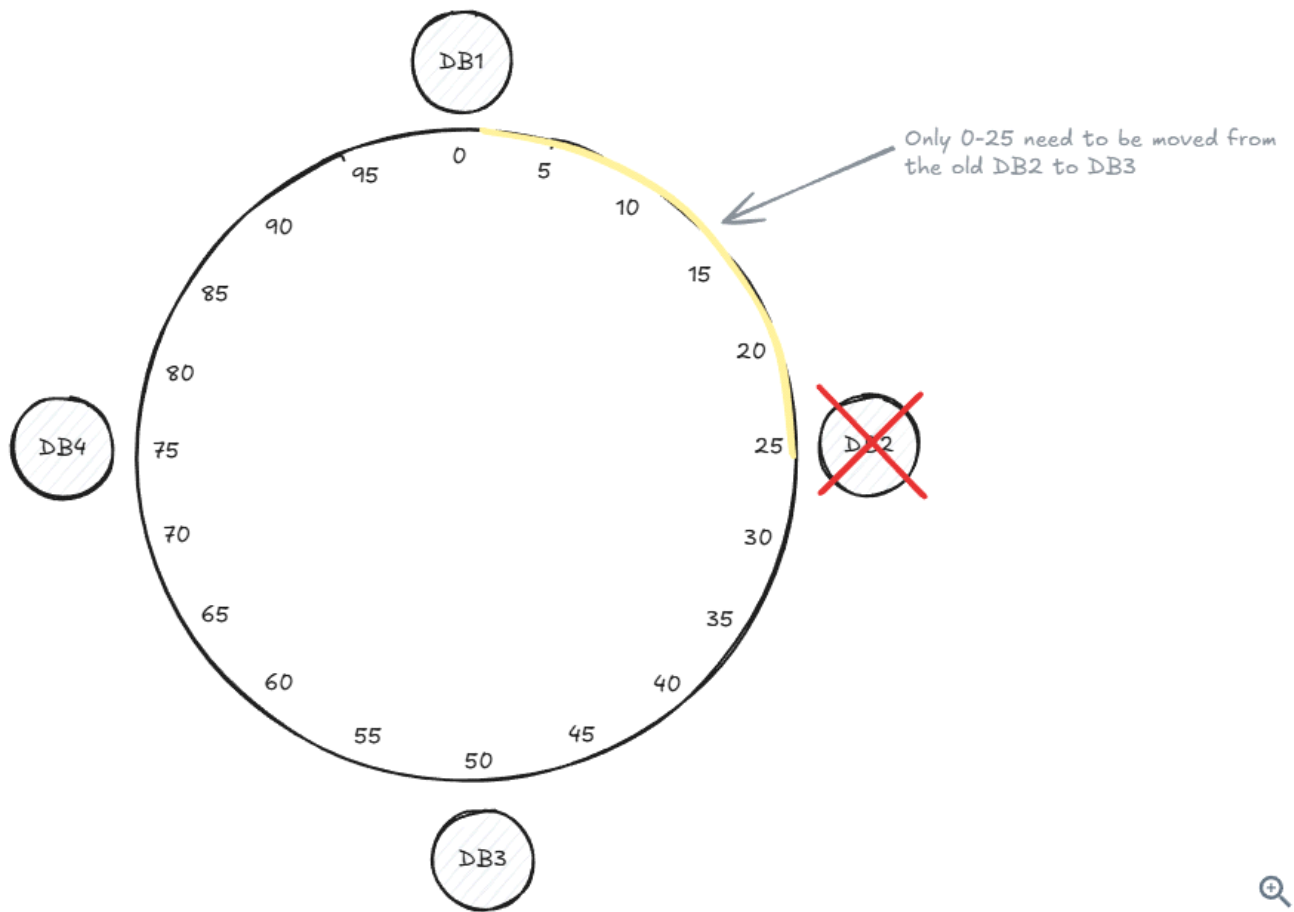


Hash Ring with DB5 added

Whereas before almost all data needed to be redistributed, with consistent hashing, we're only moving about 30% of the events that were on DB1, or roughly 15% of all our events.

Removing a Database Similarly, if Database 2 (at position 25) fails:

- Only events that were mapped to Database 2 need to move
- These events will now map to Database 3 (at position 50)
- Everything else stays put

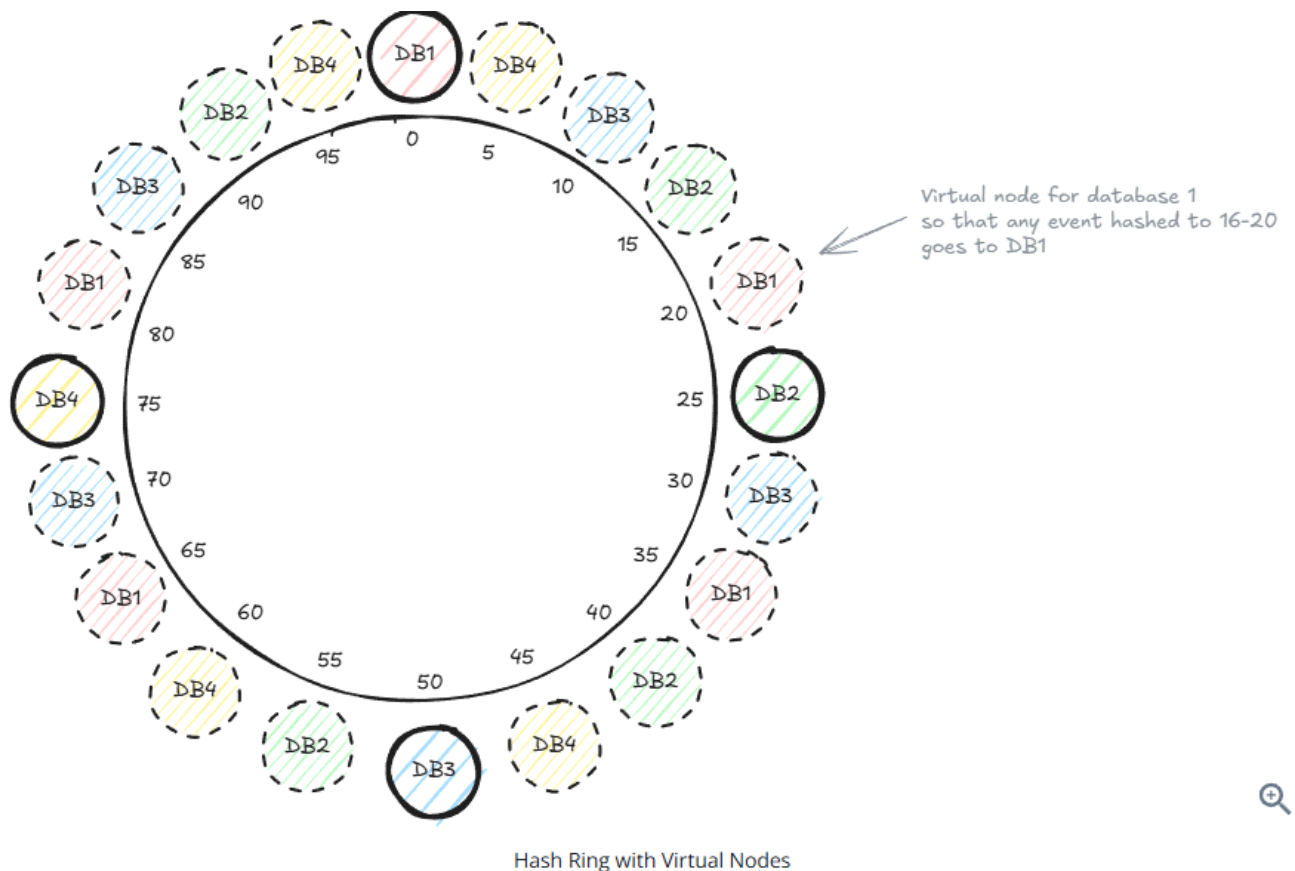


Hash Ring with DB2 removed

Virtual Nodes

We've made good progress, but there is still just one problem. In our example above where we removed database 2, we had to move all events that were stored on database 2 to database 3. Now database 3 has 2x the load of database 1 and database 4. We'd much prefer if we could spread the load more evenly so database 3 wasn't overloaded.

The solution is to use what are called "virtual nodes". Instead of putting each database at just one point on the ring, we put it at multiple points by hashing different variations of the database name.



For example, instead of just hashing "DB1" to get position 0, we hash "DB1-vn1", "DB1-vn2", "DB1-vn3", etc., which might give us positions 15, 25, 40 and so on. We do this for each database, which results in the virtual nodes being naturally intermixed around the ring.

Now when Database 2 fails:

- The events that were mapped to "DB2-vn1" will be redistributed to Database 1
- The events that were mapped to "DB2-vn2" will go to Database 3
- The events that were mapped to "DB2-vn3" will go to Database 4
- And so on...

This means the load from the failed database gets distributed much more evenly across all remaining databases instead of overwhelming just one neighbor. The more virtual nodes you use per database, the more evenly distributed the load becomes.

Consistent Hashing in the Real World

While our example focused on scaling a database, note that consistent hashing applies to any scenarios where you need to distribute data across a cluster of servers. This cluster could be databases, sure, but they could also be caches, message brokers, or even just a set of application servers.

We see consistent hashing used in many heavily relied on, scaled, systems. For example:

1. [Apache Cassandra](#): Uses consistent hashing to distribute data across the ring
2. [Amazon's DynamoDB](#): Uses consistent hashing under the hood
3. [Content Delivery Networks \(CDNs\)](#): Use consistent hashing to determine which edge server should cache specific content

Popular Hashing Used

1. MurmurHash (v3)

- **Very popular** for consistent hashing
- Fast, non-cryptographic

- Excellent distribution
 - Available in many languages
- Use when:** you want speed + good uniformity
- Example:** Cassandra, Kafka

2. xxHash

- Extremely fast
 - Non-cryptographic
 - Very good avalanche behavior
- Use when:** high-throughput systems

3. CityHash / FarmHash

- Developed by Google
 - Optimized for modern CPUs
 - Good distribution
- Use when:** performance matters and keys are strings

5. MD5

- Frequently seen in older consistent hashing implementations
 - Uniform output
 - Cryptographically broken (but OK for hashing keys)
- Use when:** compatibility with legacy systems

6. SHA-1 / SHA-256

- Very uniform distribution
 - Much slower than non-crypto hashes
- Use when:** security or untrusted inputs matter

Best Practices for Consistent Hashing

- Prefer **non-cryptographic hashes** (MurmurHash3, xxHash)
- Use **64-bit or 128-bit output**
- Combine with **virtual nodes** for better balance
- Keep hash function **stable across deployments**

Load Balancer

Load balancers distribute incoming client requests to computing resources such as application servers and databases. In each case, the load balancer returns the response from the computing resource to the appropriate client

Summary of Load Balancing in Distributed Systems

This video provides a comprehensive introduction to **load balancing**, a fundamental concept in distributed systems crucial for handling large volumes of simultaneous requests efficiently. Load balancers act as traffic controllers, distributing requests across multiple servers to prevent any one server from becoming a bottleneck or failing under heavy load.

Core Concepts and Importance

- **Load balancing** ensures system availability and performance by routing requests evenly across multiple servers.
- It is essential for **horizontal scaling**, where additional servers are added to handle increasing traffic.
- Load balancers can operate at multiple layers of an application architecture:
 - Between clients and front-end servers
 - Between front-end and backend servers
 - Between backend servers and databases
- **Health checks** are performed regularly by load balancers to monitor server availability and performance, rerouting traffic away from unhealthy or overloaded servers.

Load Balancing Algorithms			
The video divides load balancing strategies into two main categories:			
Algorithm Type	Characteristics	Pros	Cons
Static Algorithms	Use predefined rules (CPU capacity, connection limits)	Simple and fast	Not adaptive; no real-time adjustment
Dynamic Algorithms	Adapt based on real-time metrics (response time, active connections)	More accurate and adaptive	Requires constant monitoring and overhead

Static Algorithm Examples

- **Random Strategy**
 - Routes each request to a randomly chosen server
 - Works well if servers are identical and traffic is evenly distributed
 - Can lead to uneven load if server capacities differ or traffic spikes occur
- **Round Robin**
 - Requests are distributed sequentially through the server list in a fixed order
 - Effective for equal-capacity servers with predictable load
 - Does not consider current server load or performance

- **Weighted Round Robin**
 - Assigns requests proportional to server capacity (e.g., CPU power)
 - Optimizes resource use by allowing more powerful servers to handle more requests
- **Sticky (Session) Round Robin**
 - Routes a client's requests to the same server during a session
 - Supports session consistency for applications like shopping carts or dashboards

Dynamic Algorithm Examples

- **Least Connections**
 - Directs requests to the server with the fewest active connections
 - Suitable for variable traffic and processing times, preventing server overload
- **Lowest Response Time**
 - Chooses the server with the fastest average response time over a recent period
 - Often combined with least connections to optimize both load and speed

Hash-Based Load Balancing

- **IP Hashing**
 - Applies a hash function on the client's IP address to route all requests from the same client to the same server
 - Ensures session consistency and even traffic distribution
- **URL Hashing**
 - Hashes URL paths to direct types of requests (e.g., payments, orders) to specific servers
 - Useful for domain-specific routing and data locality

Practical Considerations and Hybrid Approaches

- No single load balancing algorithm fits all scenarios; the choice depends on:
 - System architecture
 - Application requirements
 - Traffic characteristics
- Complex systems often use **hybrid strategies**, combining multiple algorithms:
 - Example: Using weighted least connections normally, switching to IP hashing during peak loads for session consistency

Key Insights

- **Load balancing is critical for scalability, reliability, and performance in distributed systems.**
- **Health checks ensure that traffic is routed only to healthy servers, maintaining system stability.**
- **Static algorithms are easy to implement but lack adaptability; dynamic algorithms provide better real-time responsiveness but with additional overhead.**
- **Session persistence is essential for user experience in many applications, achievable via sticky sessions or hashing techniques.**
- **Load balancer placement varies by application layer, reflecting the complexity of modern distributed architectures.**

This foundational overview prepares viewers to understand and select appropriate load balancing strategies, setting the stage for deeper exploration of load balancer types and deployment scenarios in future discussions.

[Source: Scalable system design patterns](#)

Load balancers distribute incoming client requests to computing resources such as application servers and databases. In each case, the load balancer returns the response from the computing resource to the appropriate client. Load balancers are effective at:

- Preventing requests from going to unhealthy servers
- Preventing overloading resources
- Helping eliminate single points of failure

Load balancers can be implemented with hardware (expensive) or with software such as HAProxy.

Additional benefits include:

- SSL termination - Decrypt incoming requests and encrypt server responses so backend servers do not have to perform these potentially expensive operations
 - Removes the need to install [X.509 certificates](#) on each server
- Session persistence - Issue cookies and route a specific client's requests to same instance if the web apps do not keep track of sessions

To protect against failures, it's common to set up multiple load balancers, either in [active-passive](#) or [active-active](#) mode.

Load balancers can route traffic based on various metrics, including:

- Random
- Least loaded
- Session/cookies
- [Round robin or weighted round robin](#)
- [Layer 4](#)
- [Layer 7](#)

Layer 4 load balancing

Layer 4 load balancers look at info at the [transport layer](#) to decide how to distribute requests. Generally, this involves the source, destination IP addresses, and ports in the header, but not the contents of the packet. Layer 4 load balancers forward network packets to and from the upstream server, performing [Network Address Translation \(NAT\)](#).

layer 7 load balancing

Layer 7 load balancers look at the [application layer](#) to decide how to distribute requests. This can involve contents of the header, message, and cookies. Layer 7 load balancers terminates network traffic, reads the message, makes a load-balancing decision, then opens a connection to the selected server. For example, a layer 7 load balancer can direct video traffic to servers that host videos while directing more sensitive user billing traffic to security-hardened servers.

At the cost of flexibility, layer 4 load balancing requires less time and computing resources than Layer 7, although the performance impact can be minimal on modern

commodity hardware.

Horizontal scaling

Load balancers can also help with horizontal scaling, improving performance and availability. Scaling out using commodity machines is more cost efficient and results in higher availability than scaling up a single server on more expensive hardware, called Vertical Scaling. It is also easier to hire for talent working on commodity hardware than it is for specialized enterprise systems.

Disadvantage(s): horizontal scaling

- Scaling horizontally introduces complexity and involves cloning servers
 - Servers should be stateless: they should not contain any user-related data like sessions or profile pictures
 - Sessions can be stored in a centralized data store such as a [database](#) (SQL, NoSQL) or a persistent [cache](#) (Redis, Memcached)
- Downstream servers such as caches and databases need to handle more simultaneous connections as upstream servers scale out

Disadvantage(s): load balancer

- The load balancer can become a performance bottleneck if it does not have enough resources or if it is not configured properly.
- Introducing a load balancer to help eliminate single points of failure results in increased complexity.
- A single load balancer is a single point of failure, configuring multiple load balancers further increases complexity.

Source(s) and further reading

- [NGINX architecture](#)
- [HAProxy architecture guide](#)
- [Scalability](#)
- [Wikipedia](#)
- [Layer 4 load balancing](#)
- [Layer 7 load balancing](#)
- [ELB listener config](#)

What is Kafka ?

[Apache Kafka](#) is an open-source distributed event streaming platform that can be used either as a [message queue](#) or as a [stream processing system](#).

Kafka Core Concepts

- **Kafka** is a distributed, append-only **event log**
- **Kafka Cluster** is made up of multiple brokers
- **Topic** → Logical grouping of partitions. Topics are always **multi-producer**.
- **Partition** → Ordered, immutable sequence of messages that is continually appended to like a **log file**
- **Offset** → Consumer's position in a partition. Its an '**incremental ID**'
- **Broker** → Kafka server that hosts partitions. Can be **physical** or **virtual**
- **Producer** → Process that writes records to Kafka topic
- **Replicas** → Is a "backup" of a partition. Also known as **In Sync Replicas (ISR)**
- **Replication Factor** → Number of copies of data over multiple brokers. Should be greater than 1.
- **Leader/Follower** → A single partition will have **one leader** and can have **multiple follower**.
- **Consumer** → Process that reads records from Kafka topic
- **Consumer Group** → group consumers together
- **Message** → Data that kafka handles. Has three parts
 - **Header** - contains metadata
 - **Key** - Message Key
 - **Value** - Actual payload
- **Ordering guarantee** → only **within a partition**

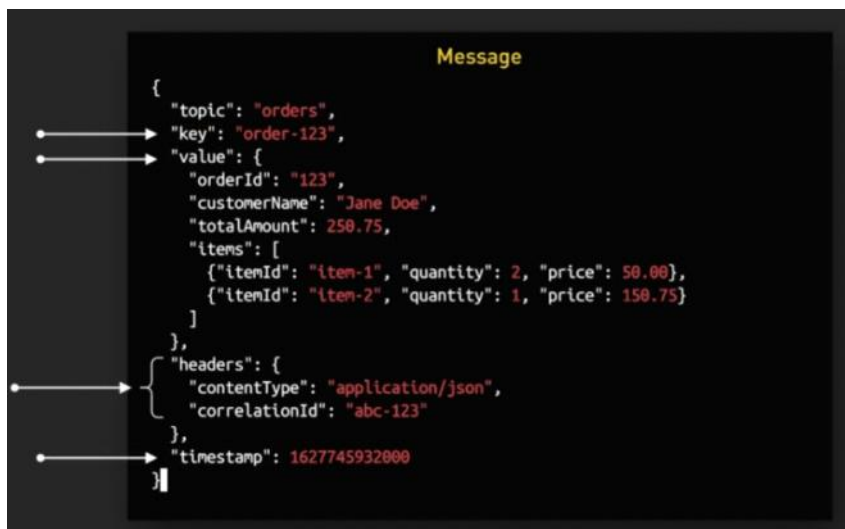
Why Kafka is needed?

- **Real-time processing**
- **Fault tolerant**
- **Scalable**
- **Event Driven Architecture**

How Kafka Integrates Different Data Processing Models

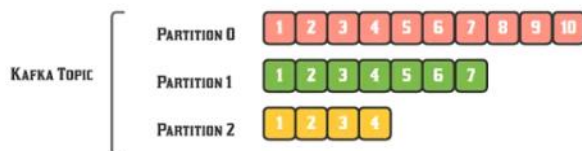
- **Event Streaming (Publish-Subscribe Model)** - Producers publishes message to topic and Consumer subscribes to that topic and receives the messages.
Example : A stock trading platform.
- **Message Queue (Point to Point Processing)** - Acts like a message queue using consumer group. Kafka distributes messages among them.
Example : A ride hailing app like Uber.
- **Batch processing** - Messages can be stored in Kafka topics and processed later. Tools like Apache Spark or Hadoop can read data from Kafka in batches and perform analytics.
Example : An e-commerce company can collect website visitor data in Kafka and analyze it later to improve product recommendations.
- **Hybrid Model (Real Time + Batch Processing)** - It can send data immediately for real-time analytics while also storing it for batch processing later. This is often done using Kafka Streams, Spark Streaming, or Flink.
Example : A fraud detection system can process transactions in real time to flag suspicious activity while also running deeper batch analysis at the end of the day.

Typical Kafka Message

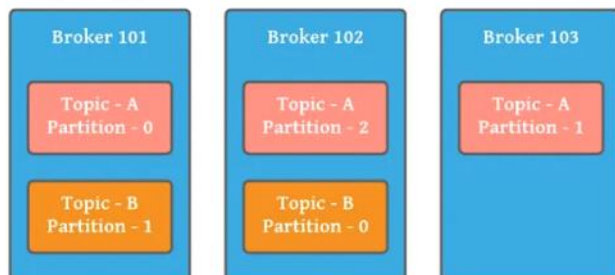


General Scoops

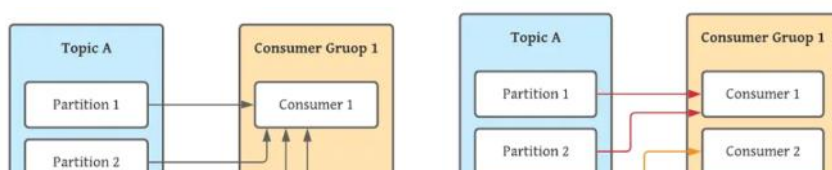
- By default messages are stored for only **7 days** in a partition
- Message order is only guaranteed within a partition

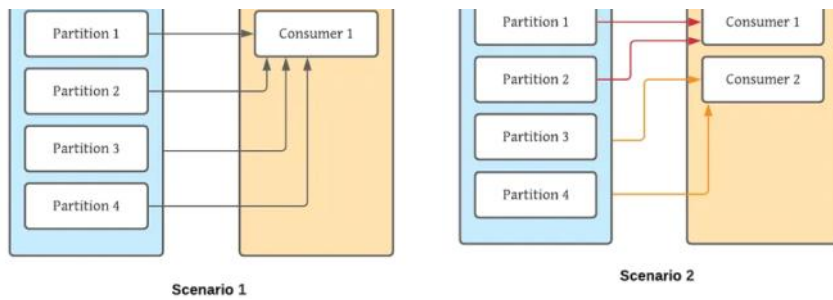


- Ideally** there should be **3 brokers** for a system to function well
- Each broker has a **unique ID** number.
- Topic Partition and Broker relationship

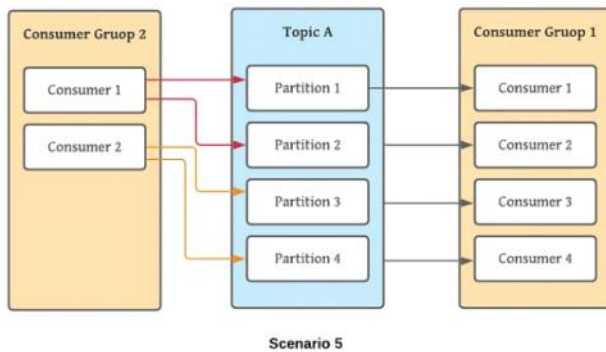


- Ideal **replication factor** should be between **2 and 3**.
- Each **partition** has a **single leader** across **multiple brokers**.
- Leader** handles **reads and writes** while **followers sync** them.
- If a **leader fails**, a **follower** will **automatically** be **chosen** and become the new leader.
- When the broker recovers, it will be leader again.
- A producer can receive **3 types of acks** from ISR
 - acks=0** (Producer **don't wait** for the acknowledgment — possible data loss)
 - acks=1** (Producer **wait for leader** acknowledgment — limited data loss)
 - acks=all** (Producer **wait for leader + replicas** acknowledgment — no data loss)
- Ordering of messages is achieved through keys.
- If a key is provided, the partitioner will hash the key with (**murmur2**) algorithm and **divide it by number of partition**. The result is that the same key is always assigned to the same partition.
- Kafka can not guarantee that key A goes to partition 1. But all message for that key will always go to the same partition (whichever partition is selected internally).
- In case of Broker failures, Consumers will automatically recover. The consumer read in order within each partition.
- When **multiple consumers** read from a topic and belong to the **same consumer group**, each consumer will receive messages from a **different subset of the topic's partitions**.

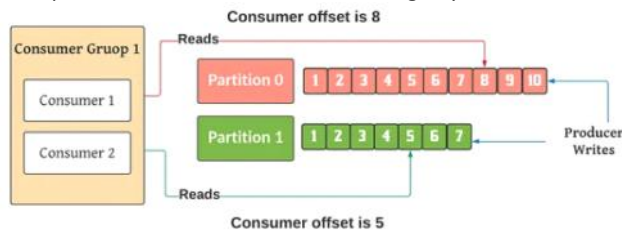




- If there are more consumers in a CG1 then the Topic Partitions, then **excess consumers** will be **idle**
- If you have **multiple consumer group** consuming from same topic, then the same message will flow twice, **1 message each per consumer group**



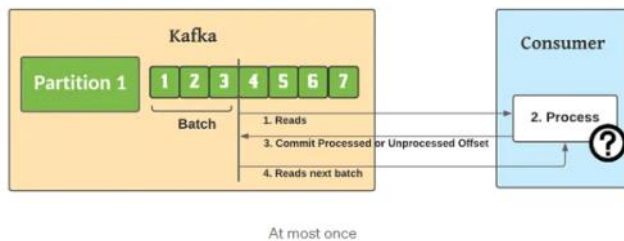
- Each partition maintains last read message by an offset, also known as a position.



3 Different Delivery Semantics

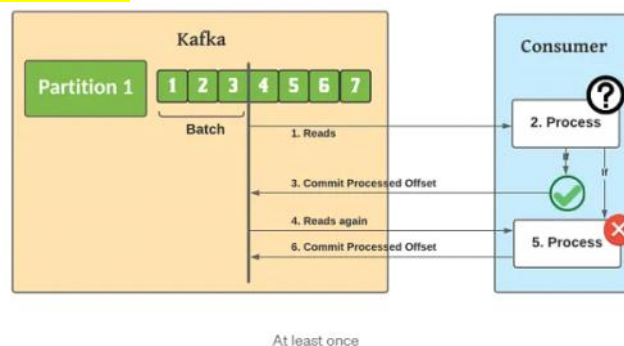
► At Most Once:

A message should be **delivered once**. It's acceptable to lose a message rather than delivering a message twice in this semantic. Offsets are committed as soon as the message is received, and if the processing goes wrong, the message will be lost.



► At Least Once:

It is acceptable to **deliver a message more than once**, but no message should be lost. The consumer ensures that all messages are read and processed, and if the processing goes wrong, the message will read again. This may result in processing duplicate messages. Therefore application should be idempotent. **This is the most preferred semantics.**



► Exactly Once:

A message must be **delivered only once, and no message should be lost**. This can be achieved by Kafka to Kafka workflows using Kafka streaming API.

- **Zookeeper** is a centralized open-source server for maintaining and managing configuration information, naming conventions, and synchronization for distributed cluster environment. **Previous versions** of Kafka used zookeeper to manage its configurations. It has been replaced by **KRaft** in **newer versions**.

Advanced Scoops

- **Immutability**: By restricting operations to appending data at the end of the log, Kafka **minimizes disk seek times**.
- **Efficiency**: Once message is written in a partition, it cannot be altered or deleted.
- **Scalability**: The simplicity of the append-only log mechanism facilitates horizontal scaling
- Message offset is used by consumers to track their progress in reading messages from the topic. As consumers read messages, they maintain their current offset and periodically commit this offset back to Kafka. This way, they can resume reading from where they left off in case of failure or restart.
- **Controller's Role in Replication**: The controller within the Kafka cluster also manages the replication process. It monitors the health of all brokers and manages the leadership and replication dynamics. When a broker fails, the controller reassigns the leader role to one of the in-sync follower replicas to ensure continued availability of the partition
- Kafka is a **Pull-based model**
- **Disadvantages of Push Based Model** :
 - ▶ **Difficult for consumers to cope up with consumption rate** if there is a message rate spike. **Consumer might crash**.
 - ▶ A push-based system must choose to either send a request immediately or accumulate more data and then send it later without knowledge of whether the downstream consumer will be able to immediately process it.
- **Advantages of Pull Based Model** :
 - ▶ Since the consumer is pulling the messages, if there is a message rate spike, it simply catches up at one point.
 - ▶ So, the consumer control the consumption rate
 - ▶ Prevents overwhelming or slowness of consumers
 - ▶ The consumer can simply accumulate the amount of messages and process them. Thus simplifying batching.
- **Disadvantages of Pull Based Model**:
 - ▶ Native Pull based model, may end up polling in a tight loop if messages are not available.
- **Fix for Pull Based Model**:
 - ▶ Kafka indulges in "Long Polling" waiting until data arrives or optionally until a given number of bytes has is available
- A consumer can rewind the offset value and reconsume the messages after that offset. Also known as **Message Replay**.
- Message byte size can be configured via `message.max.bytes` . Recommended to keep size **under 1MB**
- **Kafka is not a database**. You should not store large files here.
- Typically a single broker can handle around **1 TB of data** and as many as **1M messages per second**.
- **Horizontal Scaling** : Adding more brokers in order to distribute load evenly and add more parallelism.
- Hashing logic used to locate correct partition : `partition = hash(key) % num_partitions`
- Popular managed Kafka Services are **Confluent Cloud and AWS MSK**
- It is always advisable to keep consumer less heavy and small.
- To detect stalled consumers, Kafka revokes a partition when a consumer does not poll within the max poll interval - `max.poll.interval.ms` . The **default max poll** interval is **5 minutes**. After a partition is revoked, it will be assigned to another consumer.
- With the default configuration, the consumer **automatically stores offsets to Kafka**. You can use the `auto.commit.interval.ms` config to tweak the frequency of commits if `enable.auto.commit` is set to true.

How to handle hot partition?

- **Random partitioning with no key**: If you don't provide a key, Kafka will randomly assign a partition to the message. But you loose message ordering.
- **Random Salting**: We can add a random number or timestamp to the ad ID when generating the partition key. But may complicate aggregation logic later on consumer side.
- **Compound Key**: Use a combination of ad ID and another attribute, such as geographical region or user ID segments, to form a compound key. This is most suitable among all.
- **Back Pressure**: You can slow down the producer. The producer check the lag on the partition and slow down if it's too high. Managed Kafka Service might have this mechanism built-in.

What happens when a consumer goes down?

- **Offset Management**: When the consumer comes back on, it starts to read from the last offset it committed.
- **Rebalancing** : When part of a consumer group, if one consumer goes down, Kafka will redistribute the partitions among the remaining consumers so that all partitions are still being processed.

Handling Retries and Errors

- **Producer Retries:** Kafka producers supports automatic retries.

```
const producer = kafka.producer({
  retry: {
    retries: 5, // Retry up to 5 times
    initialRetryTime: 100, // Wait 100ms between retries
  },
  idempotent: true,
});
```

- **Consumer Retries:** Kafka does not support retries for consumers out of the box. We can setup DLQ or a separate topic if we get an exception and process the failed messages later separately by another consumer process.

Performance Optimizations

- **Batching:** We can batch messages in the producer before sending them to Kafka.

```
const producer = kafka.producer({
  batch: {
    maxSize: 16384, // Maximum batch size in bytes
    maxTime: 100, // Maximum time to wait before sending a batch
  }
});
```

- **Compress Messages:** We can compress messages by setting the `compression` option in the producer configuration. It supports **GZIP, Snappy and LZ4**

```
const producer = kafka.producer({
  compression: CompressionTypes.GZIP,
});
```

- **Good Message Key :** Choosing an intelligent key will introduce even distribution of messages in partitions thus avoiding hot partition.

Retention Policies

- The retention policy of messages in Kafka can be configured via `retention.ms` and `retention.bytes` settings. **Default retention** is **7 days**.

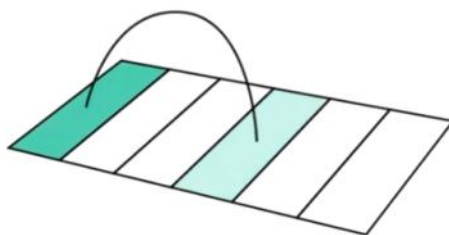
Why Kafka is fast?

1) **Kafka relies on Sequential I/O.**

There are two types of disk access patterns - **Random** and **Sequential**.

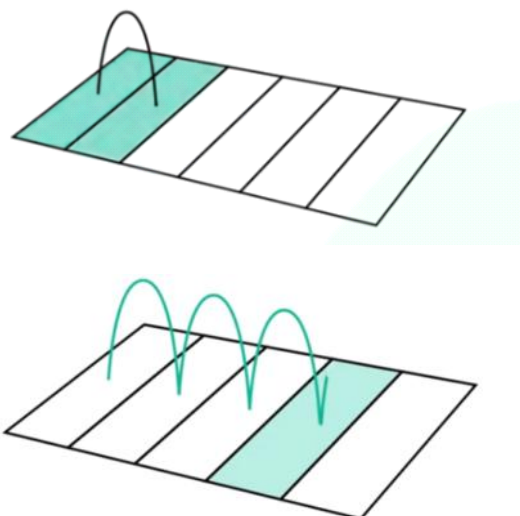
For hard drives it takes time to physically move the arm to different locations on the magnetic disks. This is what makes random access slow.

RANDOM



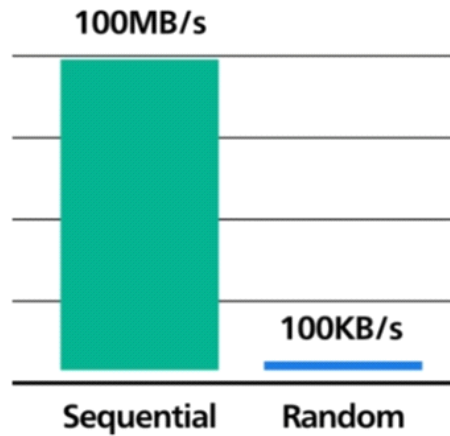
For Sequential access, since your arm does not need to jump around, it is much faster to read and write blocks of data one after the other.

SEQUENTIAL

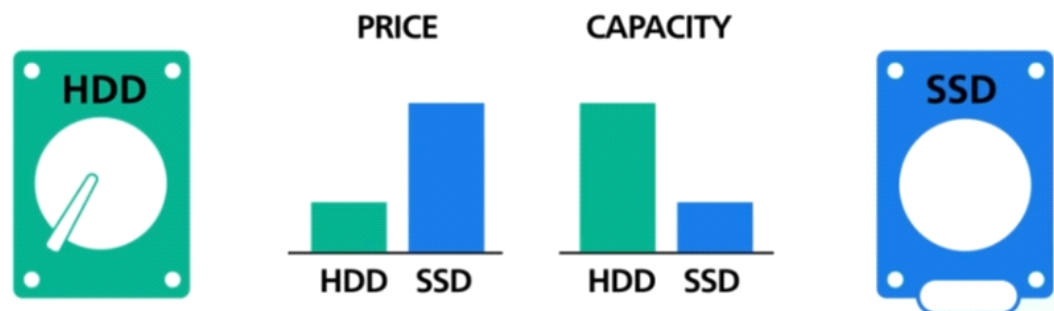


Kafka takes advantage of this by using an **append-only log** as its primary data structure. An append-only log adds new data to the end of the file. **This access pattern is sequential.**

On modern hardware, Sequential writes reach **100mbps**, while random writes are in **100 kbps**. Sequential writes is much faster as we can see.



One more advantage is that hard disk comes at a very lower cost and provides more capacity than SSDs

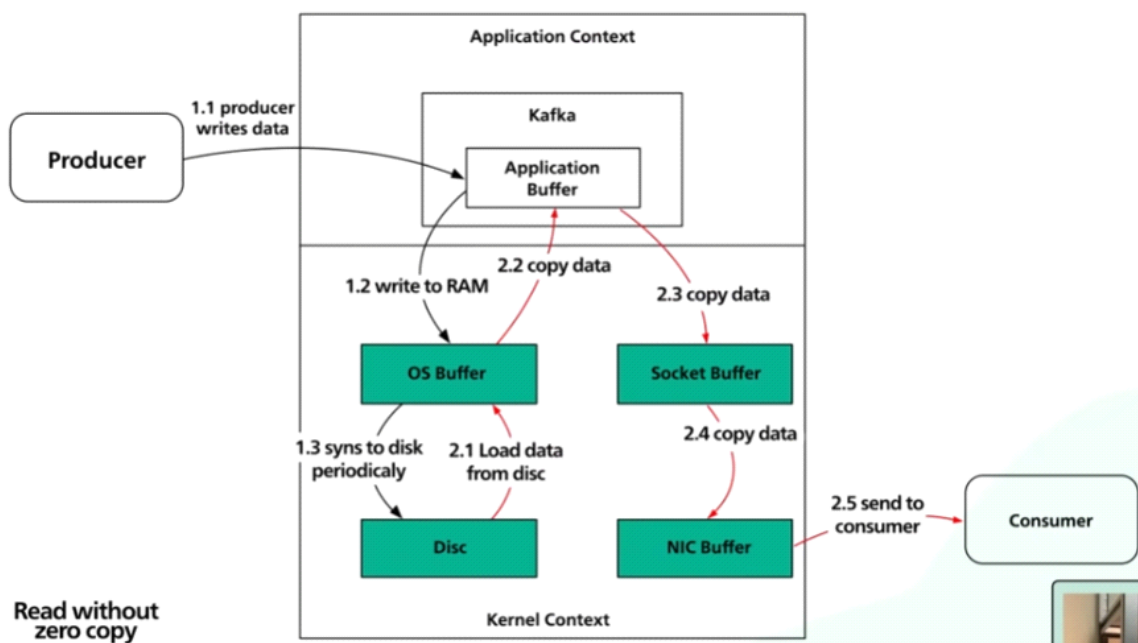


Thus using HDDs in kafka , means it can have a large capacity to store messages cost-effectively for a long period of time.

2) Zero Copy Principle

Modern unix operating systems are highly optimized to transfer data from disk to network without copying data excessively. First lets see how kafka sends pages of data to consumer when **zero copy is not used** at all.

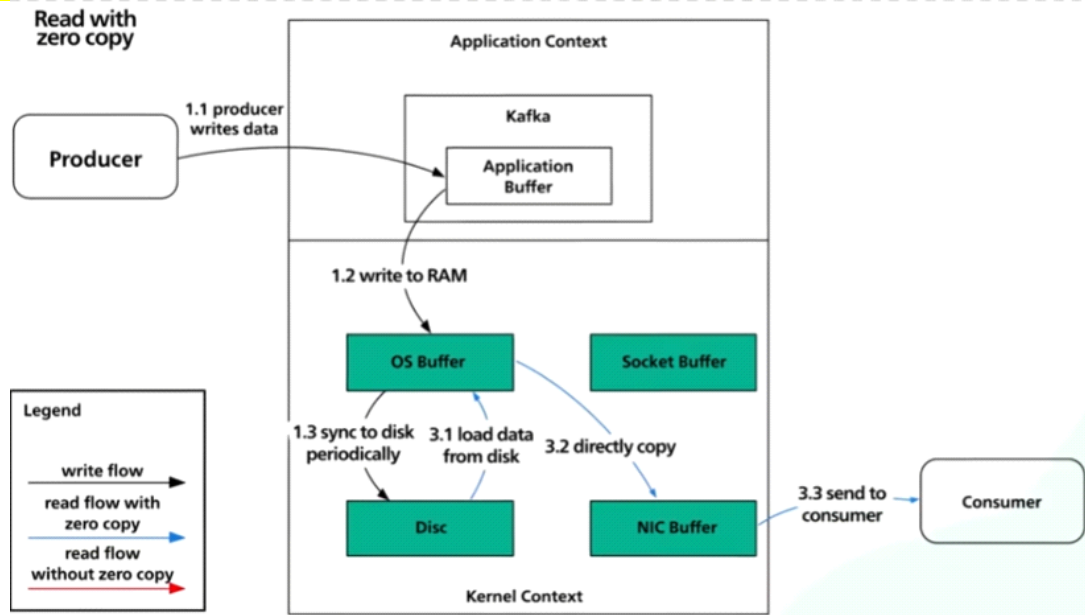
- First the data is loaded from the disk to **OS cache**
- Second, the data is copied from **OS cache** into the **Kafka** application
- Third the data is copied from **Kafka** to the **Socket Buffer**
- Fourth, the data is copied from **Socket Buffer** into the **NIC Buffer** (Network Interface Card)
- Finally, the data is sent through the Network to the Consumer



This is clearly inefficient, there are **4 copies** and **2 system calls**.

Read with zero copy.

- First, the data page is loaded from the disk to the OS Cache.
- With zero copy, the Kafka application uses a system call **sendFile** to tell the operating system to directly copy the data from OS Cache to NIC Buffer. So, here the only copy is from OS Cache into the NIC Buffer. With modern NIC, this copy is done with **DMA(Direct Memory Access)**. When DMA is used, the **CPU is not involved**, making it even more efficient.



3) Files are indexed for faster lookup

What is the basic difference between message-queues and stream?

The only meaningful difference is with how consumers interact with the data. In a message queue, consumers read messages from the queue and then acknowledge that they have processed the message. In a stream, consumers read messages from the stream and then process them, but they don't acknowledge that they have processed the message. This allows for more complex processing of the data.

What happens when a broker goes down? Then how kafka hashes the key with murmur2 and divide by no of partitions?

When a broker fails, the count of partitions are still the same. There is only one leader and multiple followers, when a broker fails, kafka immediately elects a leader from one of the followers and serves traffic. Since the count of partitions is not changing, so rehashing is not required.

`partition = murmur2(key) % number_of_partitions`

Only the partition to broker mapping gets changed, so that producer can get the correct partition to send messages

What if we add a new partition to a kafka topic will it affect the key distribution in the existing partitions?

Generally it's not recommended to repartition in between. When partitions are added, Kafka recalculates key-to-partition mapping using the new partition count, so new records for existing keys may go to different partitions, breaking per-key ordering.

Workaround :

- 1) You can pin particular message key to specific partition but that still cannot avoid reshash issue completely.
- 2) You can also use consistent hashing but that adds complexity and it's a non-standard kafka practise.
- 3) The recommended procedure is therefore to create a new topic with the desired partition count, and copy the original topic's messages into it. To perform repartitioning automatically, you can use tools like [ksqlDB](#) and [Kafka Streams](#).

Is Kafka blocking or non-blocking. How many threads gets blocked internally?

Kafka is **internally non-blocking** and asynchronous, but **exposes some blocking APIs** for convenience. No Kafka broker thread is *permanently blocked* waiting on a client. Only application threads block.

Producer Internally is non-blocking

- Producer uses **async I/O**
- Uses a **background network thread**
- Requests are batched and sent asynchronously

```
producer.send(record); // NON-BLOCKING
producer.send(record).get(); // BLOCKING
```

Consumer Internally is non-blocking

- Uses async network I/O
- Heartbeats handled in background

```
consumer.poll(timeout); // BLOCKING
```

- Blocks **until data arrives or timeout expires**
- This is **intentional & controlled**

Only **polling thread blocks**, not Kafka internals. Kafka trades CPU for throughput using batching, async I/O, and backpressure instead of thread blocking.

What does `num.network.threads` and `num.io.threads` do?

`num.network.threads` controls non-blocking socket I/O (**default 3. Increase if many producers/consumers or large number of partitions**), while `num.io.threads` controls request processing and disk access (**default 8. Increase if CPU is underutilized or request queues grows**); tuning is CPU- and workload-dependent, and improper values can cause latency or resource contention. These settings are **per broker**. They do **not scale automatically**

Rule of thumb .

`num.network.threads` $\approx$ number of CPU cores / 2
`num.io.threads` $\approx$ number of CPU cores

Small cluster (8 cores) : `num.network.threads=4` & `num.io.threads=8`

Medium cluster (16 cores) : `num.network.threads=6` & `num.io.threads=16`

Large cluster (32+ cores) : `num.network.threads=8` & `num.io.threads=32`

Network threads move data; IO threads do the work.

How you can control Kafka Consumer API thread blocking?

In Kafka consumers, only the application thread blocks inside `poll()` while waiting for data, metadata, or rebalances; Kafka's internal network and heartbeat threads are fully non-blocking.

```
ConsumerRecords<K, V> records = consumer.poll(Duration.ofMillis(1000));
```

This call can block in **three situations**:

Waiting for data - No records available yet / Consumer waits **up to timeout** .

Waiting for metadata - Partition leader just changed / Broker failed

Waiting for rebalancing to complete - New consumer joins / Consumer leaves / Reassign Partition

- `poll(timeout)` → waits for data

- `max.poll.interval.ms` → max time between polls

If you exceed `max.poll.interval.ms`:

- Consumer is considered dead
- Rebalance happens

When using the thread per consumer model, you can deal with this problem by tuning the following config values:

- Set `max.poll.records` to a smaller value
- Set `max.poll.interval.ms` to a higher value
- Perform a combination of both

Can you explain how a multi-threaded Kafka consumer would work?

There are many ways to design multi-threaded models for a Kafka consumer. A naive approach might be to process each message in a separate thread taken from a thread pool, while using automatic offset commits (default config). Unfortunately, this may cause some undesirable effects:

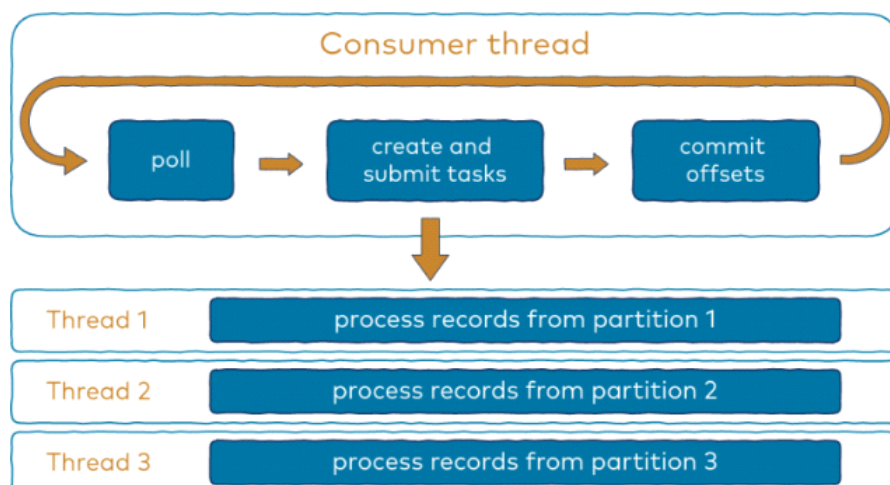
- Offset might be committed before a record is processed
- Message processing order can't be guaranteed since messages from the same partition could be processed in parallel

While the goal is to achieve record processing parallelization, you also want your multi-threaded solution to maintain the features that a common, single-threaded approach has: processing order guarantees per partition and at-least-once delivery semantics.

Note: This implementation might not be optimal for all use cases. Its goal is to demonstrate key aspects that need to be considered when implementing a multi-threaded consumer model.

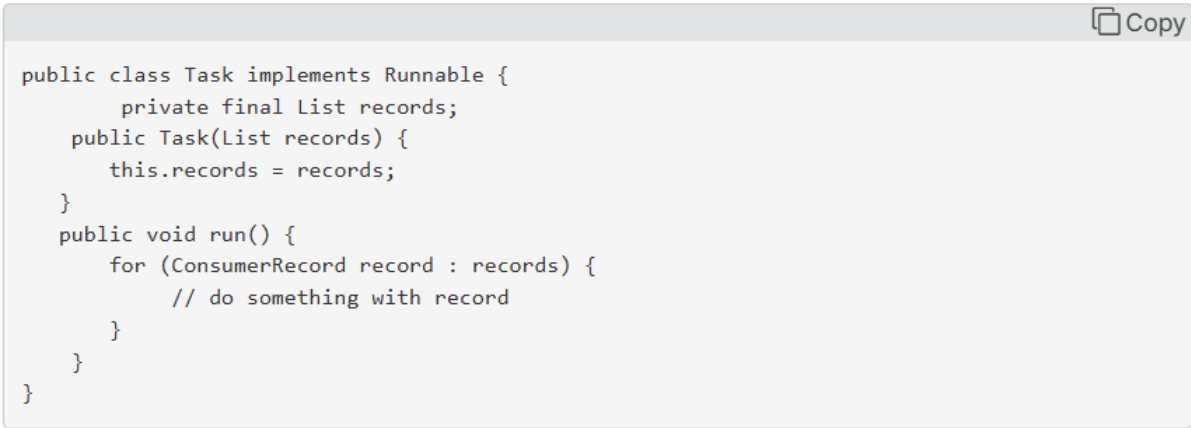
Decoupling consumption and processing

In a multi-threaded implementation, the main consumer thread delegates records processing to other threads.



Records are retrieved using the **poll method**, the same as with a single-threaded implementation. Then, the **records are grouped by partition**, which results in **multiple collections** where each holds records **from only a single partition**. In order to process those collections, **runnable tasks are created for each** and submitted to an instance of Java's built-in thread pool implementation.

The task implementation is quite simple, and the following represents a basic version that will be improved upon and discussed later in this blog post:



```
public class Task implements Runnable {
    private final List records;
    public Task(List records) {
        this.records = records;
    }
    public void run() {
        for (ConsumerRecord record : records) {
            // do something with record
        }
    }
}
```

Note that the number of collections resulting from grouping by partition varies depending on the number of partitions assigned to a consumer, throughput, and some consumer configuration settings such as **max.poll.records** and **max.partition.fetch.bytes**.

Since records from each partition are processed sequentially, a low number of partitions can lead to underutilized CPU. If the number of partitions is very high, you don't want to process all of them in parallel, since this would require a high number of threads.

In order to utilize the CPU efficiently, a thread pool with a fixed number of threads is used. That number is based on the number of available CPU cores as well as the nature of processing—is it CPU intensive or does it wait for I/O, etc.? For simplicity, assume eight core CPUs are used; therefore, a fixed thread pool with eight threads is configured:



```
private ExecutorService executor = Executors.newFixedThreadPool(8);
```

Tasks are submitted using the **ExecutorService.submit()** method, which returns immediately even if all the threads of the thread pool are currently in use. It simply adds a task to the thread pool's internal queue where it waits to be processed.

After all of the tasks are submitted to the `ExecutorService`, the main consumer thread continues to poll for new records. The poll method is called frequently since there is not much to do between calls.

This means that there are no longer problems with excessive delays between poll method calls, and as a result, group rebalancing is very fast. Sounds great, right? But in order to maintain processing order and at-least-once delivery guarantees, some additional steps have to be taken.

Ensuring processing order guarantees

Since the main thread doesn't wait for record processing threads to finish and continues with fetching, the probability of **ending up with more than one task processing records from the same partition** in parallel is high. This would obviously break processing order guarantees. **To prevent it**, pause the specific

partition using the **KafkaConsumer.pause()** method after submitting the task holding records from it.

Note: When the partition is paused, subsequent calls to the **poll method will not return any records** from that partition—no matter how many times it is called—until that partition is resumed using the **KafkaConsumer.resume()** method.

It's clear when to pause a partition, but **when should it be resumed?** The answer is: **not until the task currently handling records from that partition is finished**. The approach taken in this blog post is to call the consumer's methods solely from the main thread, avoiding multi-threaded access synchronization. In **each iteration of the poll loop**, the **main thread checks which tasks are finished and resumes corresponding partitions**.

Committing offsets

Using automatic offset commit, which happens during poll method calls, is no longer an option. **Offsets** have to be **committed manually** at appropriate times. The first step is to **disable auto commit** in the consumer configuration:

```
config.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
```

Offsets are committed manually from the main thread loop, but only Task instances know which record from the corresponding partition was last processed and, consequently, which offsets can be committed. In order to provide that information to the main consumer thread, the **getCurrentOffset()** method is **added** to the **Task class**. It returns an offset of the last processed record incremented by one (when committing the offset, **it should always be the offset of the next message that should be processed, not the last processed record offset**).

What does concurrency in Spring Kafka signify?

```
java
@KafkaListener(
    topics = "orders",
    groupId = "CG1",
    concurrency = "2"
)
```

Spring Kafka's concurrency setting creates multiple independent KafkaConsumer instances, each running in its own thread within the same consumer group; actual parallelism is bounded by the number of partitions.

What concurrency actually means is Kafka will Create 2 independent KafkaConsumer instances, each running in **its own thread**, all belonging to the **same consumer group**. This is **not** a thread pool over one consumer.

2 Internal object model (important)

Spring Kafka creates:

```
nginx

ConcurrentMessageListenerContainer
|
|-- KafkaMessageListenerContainer (consumer #1)
|-- KafkaMessageListenerContainer (consumer #2)
```

Each `KafkaMessageListenerContainer`:

- Owns **one `KafkaConsumer`**
- Runs in **one dedicated thread**
- Calls `poll()` independently

How listener execution affects `poll()`?

Since listener runs on consumer thread

- Poll is blocked until listener returns
- Heartbeats handled in background
- If processing is slow → rebalance risk

What Spring Kafka does during rebalance?

During a rebalance, Spring Kafka pauses polling, revokes partitions, commits offsets according to the ack mode, assigns new partitions, and then resumes consumption.

Steps

1. Kafka coordinator triggers rebalance
2. Spring container:
 - Pauses polling
 - Calls rebalance callbacks
3. Partitions are revoked
4. New assignment happens
5. Poll resumes

Spring exposes rebalance lifecycle hooks:

ConsumerRebalanceListener

```
public void onPartitionsRevoked(...) { }
public void onPartitionsAssigned(...) { }
```

Offset commit behavior during rebalance

Auto-commit enabled

- Offsets committed periodically
- On revoke → last committed offset used
- Potential duplicates

Manual commit (recommended)

Spring commits:

- On partition revoke
- After listener execution

This minimizes:

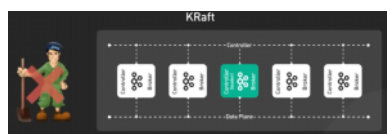
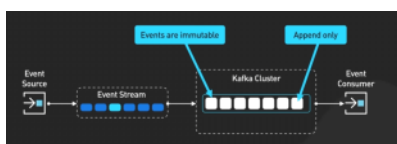
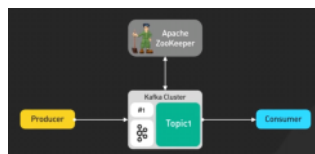
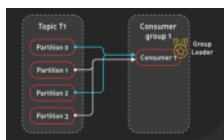
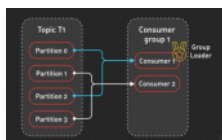
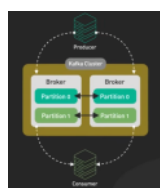
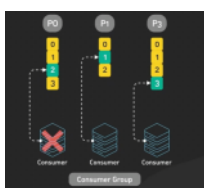
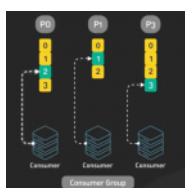
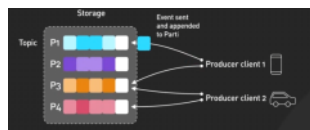
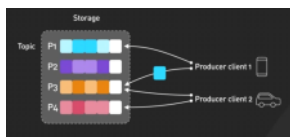
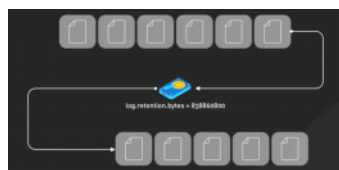
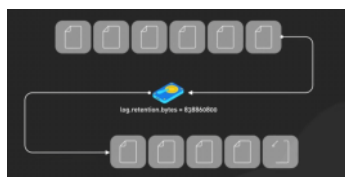
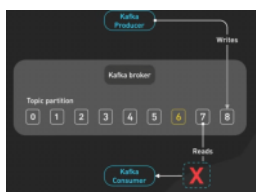
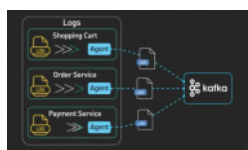
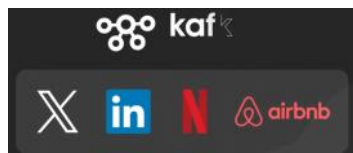
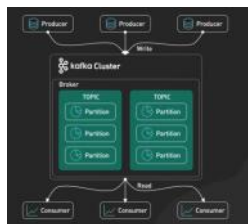
- Message loss
- Duplicate processing

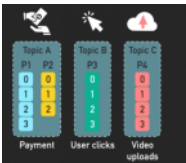
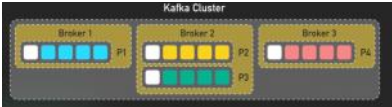
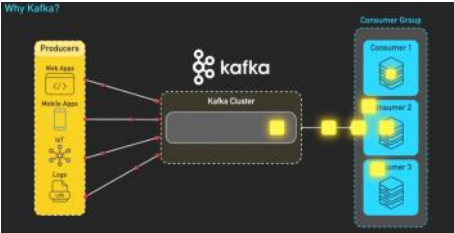
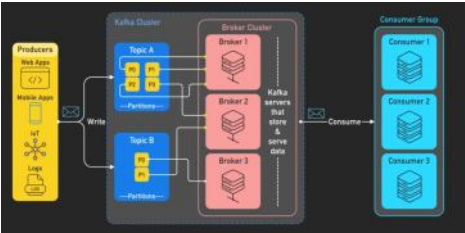
What happens to in-flight messages?

Spring Kafka **does NOT stop** your listener mid-execution

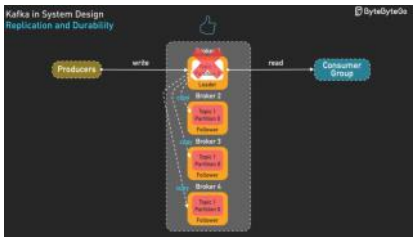
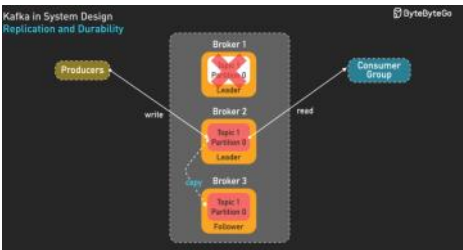
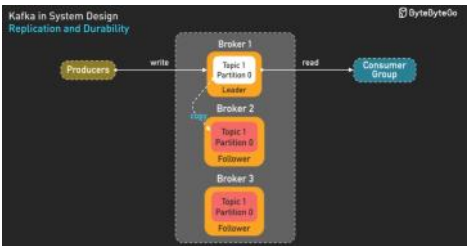
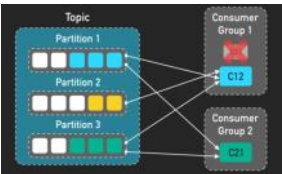
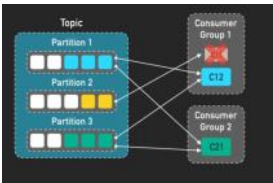
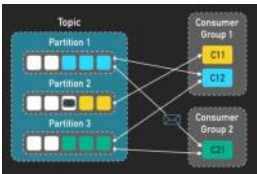
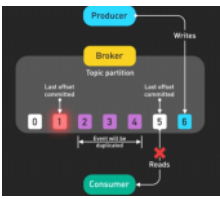
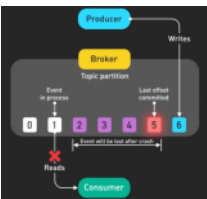
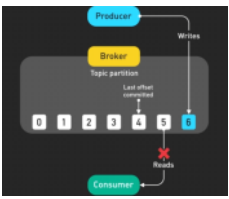
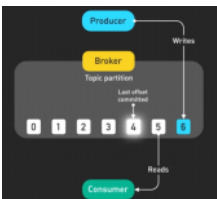
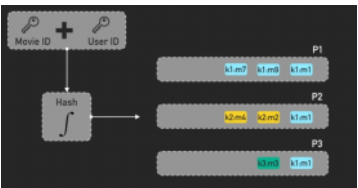
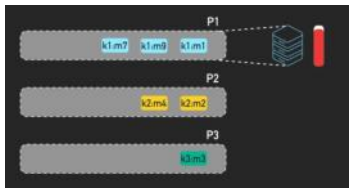
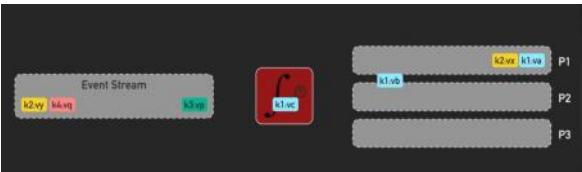
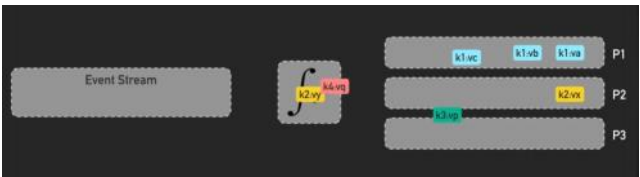
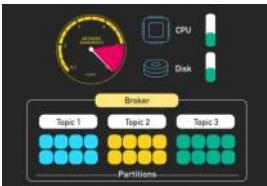
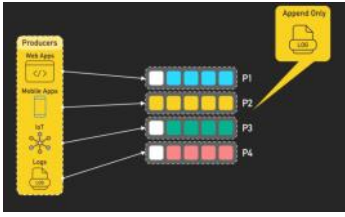
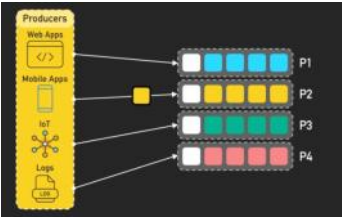
If a rebalance happens:

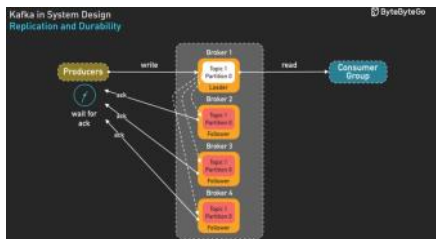
- Current record continues processing
- Partition may already be revoked
- Offset commit may fail or be skipped

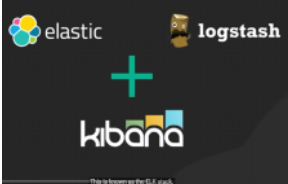
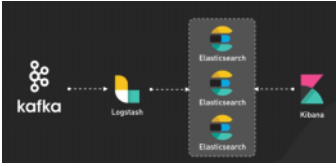
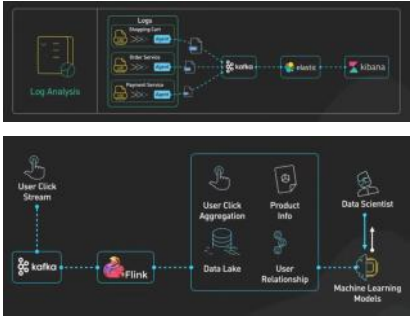




```
Message
{
  "topic": "orders",
  "key": "order-123",
  "value": {
    "orderId": "123",
    "customerName": "John Doe",
    "totalAmount": 150.75,
    "items": [
      { "itemId": "item-1", "quantity": 1, "price": 10.00 },
      { "itemId": "item-2", "quantity": 1, "price": 150.75 }
    ]
  },
  "headers": {
    "contentType": "application/json",
    "correlationId": "abc-123"
  },
  "timestamp": 162749310000
}
```









Zookeeper vs KRaft

19 January 2026 17:13

The original Kafka architecture relied on ZooKeeper to manage the metadata of the cluster. However, **ZooKeeper introduced some complexity and limitations for Kafka**. Therefore, a series of Kafka Improvement Proposals (KIPs) were proposed to replace ZooKeeper with a self-managed metadata quorum. The first version of this self-managed mode was released as an early access feature in Kafka 2.8.

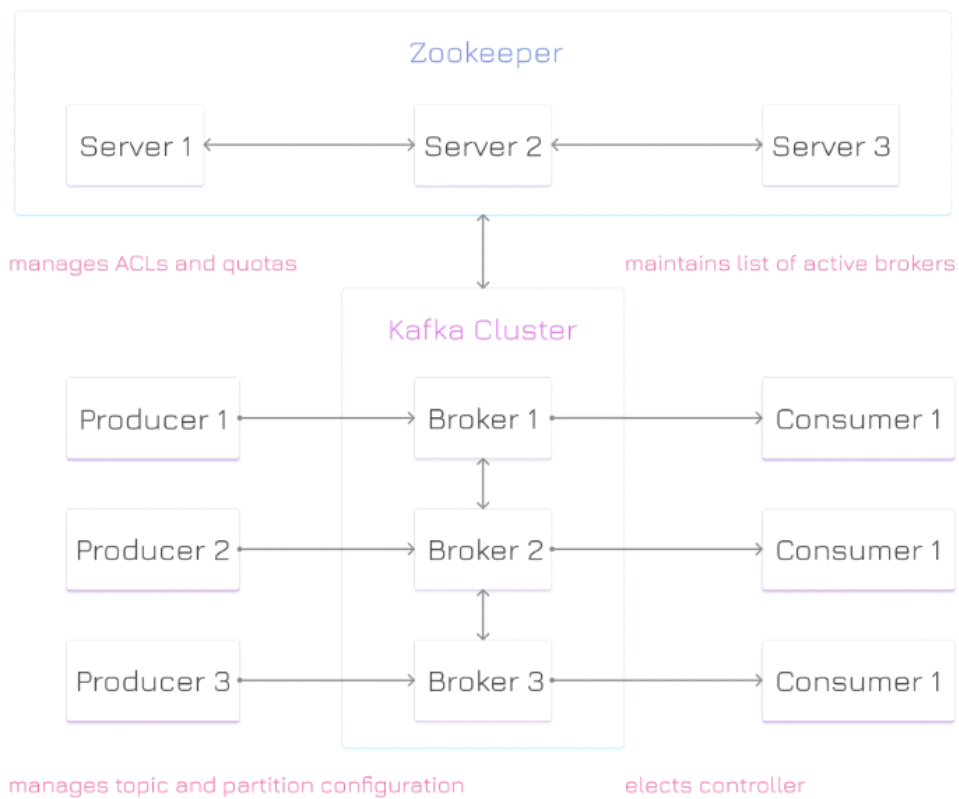
The self-managed mode simplifies the metadata management by delegating it to a new quorum controller service in Kafka. The quorum controller adopts an event-sourced storage model and uses **Kafka Raft (Kraft) as the consensus protocol** to ensure the consistency and durability of the metadata across the quorum.

Kraft is an event-based implementation of the Raft consensus protocol, which is similar to the ZAB protocol used by ZooKeeper, but with an event-driven design. The quorum controller maintains an event log to store the state transitions, which are periodically compacted into snapshots to avoid unbounded growth.

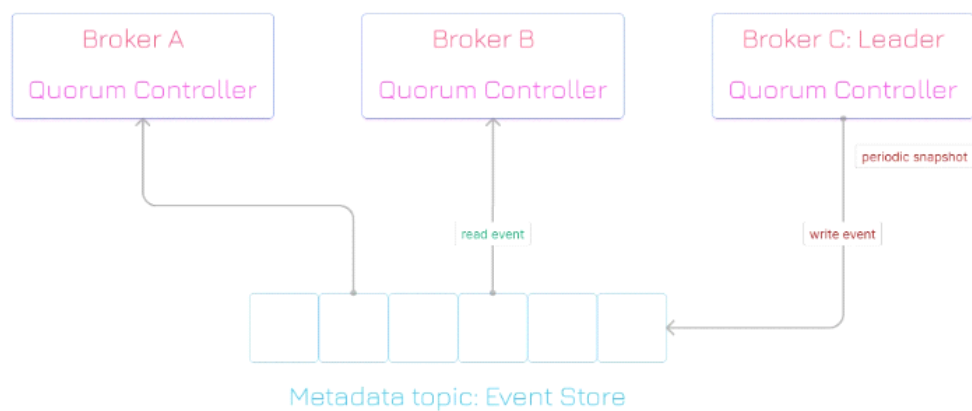
The quorum controller operates in a leader-follower mode, where the leader controller generates events in the metadata topic within Kafka, and the follower controllers consume and apply these events. If a broker fails or disconnects, it can resume from the last committed event when it rejoins. This minimizes the downtime.

The quorum controller also has some advantages over the ZooKeeper-based controller, such as not needing to load the state from an external service, and having all the committed metadata records in memory.

Transitioning from ZooKeeper to KRaft is a straightforward process that involves upgrading to Kafka 2.8 or later and enabling KRaft mode. Since August 2019, Kafka has undergone a major architectural overhaul with the Collin McCabe's KIP-500, which removes the dependency on ZooKeeper for managing metadata and leader election. This resulted in the development of KRaft mode, which has since been marked as production-ready. **As of Kafka 3.5, the ZooKeeper mode is being deprecated, and it will be completely removed from Kafka with the release of Kafka 4.0.**



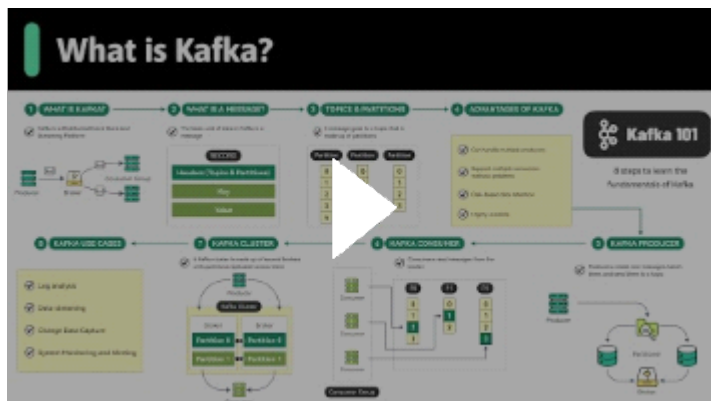
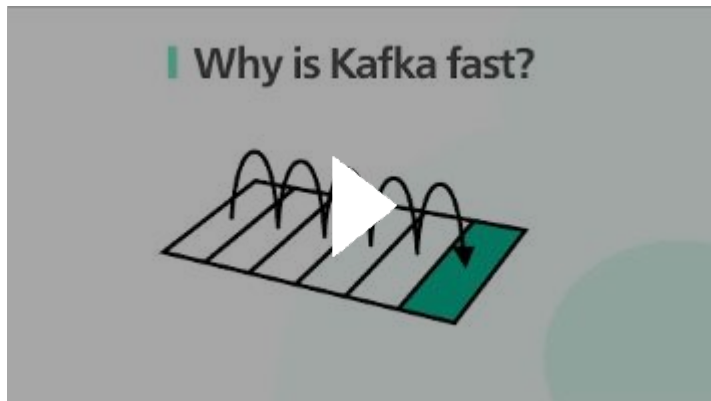
High overview KRaft architecture



Resources

19 January 2026 14:18

<https://medium.com/geekculture/apache-kafka-a352140977ac>
<https://www.hellointerview.com/learn/system-design/deep-dives/kafka>
<https://www.educba.com/kafka-consumer-group/>
https://www.youtube.com/results?search_query=kafka+architecture+deep+dive
[Apache Kafka Fundamentals You Should Know](#)
[System Design: Why is Kafka fast?](#)



Producer Configuration Tuning

24 March 2026 10:54

Common Kafka Issues in Production

1. Consumer Lag

The most frequent issue. Consumers can't keep up with producer throughput.

Causes: slow processing per message, insufficient consumer instances, GC pauses, downstream bottlenecks (DB, API calls).

Signals: `kafka.consumer:type=consumer-fetch-manager-metrics,attribute=records-lag-max` climbing continuously.

Fix: scale consumers up to partition count, parallelize processing, or increase partitions (carefully — irreversible).

2. Partition Imbalance

Some partitions get disproportionate traffic, causing hot partitions while others sit idle.

Causes: poor key design (e.g., keying on a high-cardinality but skewed field like `customerId` when one customer sends 80% of traffic), or using null keys which round-robins but loses ordering.

Fix: composite keys, custom partitioners, or sticky partitioning for null-key producers.

3. Rebalance Storms

Consumer group rebalances too frequently, causing processing pauses across the entire group.

Causes: consumers exceeding `max.poll.interval.ms` (slow processing between polls), aggressive `session.timeout.ms`, or rolling deploys triggering rapid join/leave cycles.

Fix: tune `max.poll.interval.ms` to match realistic processing time, reduce `max.poll.records`, use **incremental cooperative rebalancing** (`CooperativeStickyAssignor`) instead of eager rebalancing.

4. Duplicate Messages / At-Least-Once Delivery

Consumer processes a message, commits fails, reprocesses on restart.

Fix: idempotent consumers — use the message key or a business ID as an idempotency key, check-and-skip before processing. For producers, enable `enable.idempotence=true` + `acks=all`.

5. Message Loss

Producer side: default `acks=1` means leader acknowledges before replication — leader crash = lost message.

Fix: `acks=all` + `min.insync.replicas=2` + `replication.factor ≥ 3`. Never use `acks=0`

in production.

6. Broker Disk Full

Kafka doesn't back-pressure producers when disk is full — brokers start failing writes.

Causes: retention misconfiguration, log compaction lag, sudden traffic spike.

Fix: set `log.retention.bytes` per topic, monitor disk with alerts at 70–80%, use tiered storage (e.g., Confluent or MSK) for long-retention topics.

7. Under-Replicated Partitions (URPs)

Replicas fall behind the leader — visible via

`kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions`.

Causes: broker overload, network saturation, GC pauses on follower brokers.

Impact: if a leader goes down and all replicas are behind, you're in data loss territory.

Fix: alert at `URP > 0`, investigate broker CPU/network, tune `replica.lag.time.max.ms`.

8. Schema Evolution / Deserialization Errors

Producer publishes new schema version, consumer fails to deserialize.

Fix: use a Schema Registry (Avro/Protobuf), enforce backward/forward compatibility rules, never change field types in place.

9. Long GC Pauses on Brokers

ZooKeeper/Kafka brokers are JVM-based — GC pauses cause leader elections, URP spikes, and client timeouts.

Fix: tune heap (typically 6–8 GB for brokers, no more), use G1GC or ZGC, keep broker heap lean and let the OS page cache do the heavy lifting for reads.

10. Compacted Topic Issues

Log compaction can lag, leaving consumers reading stale or tombstoned records longer than expected.

Causes: `min.cleanable.dirty.ratio` too high, slow cleaner threads.

Fix: tune `log.cleaner.min.cleanable.ratio`, monitor `log-cleaner-manager` metrics, ensure tombstone records aren't consumed as valid data.

Quick Reference — Key Configs to Always Tune

Config	Safe Production Value
<code>acks</code>	all
<code>min.insync.replicas</code>	2
<code>replication.factor</code>	3
<code>enable.idempotence</code>	true
<code>max.poll.interval.ms</code>	Match your slowest processing time

These come up heavily in EM interviews at companies like Vonage — especially consumer lag diagnosis, rebalance root-causing, and the idempotency/exactly-once tradeoffs.

Kubernetes Overview

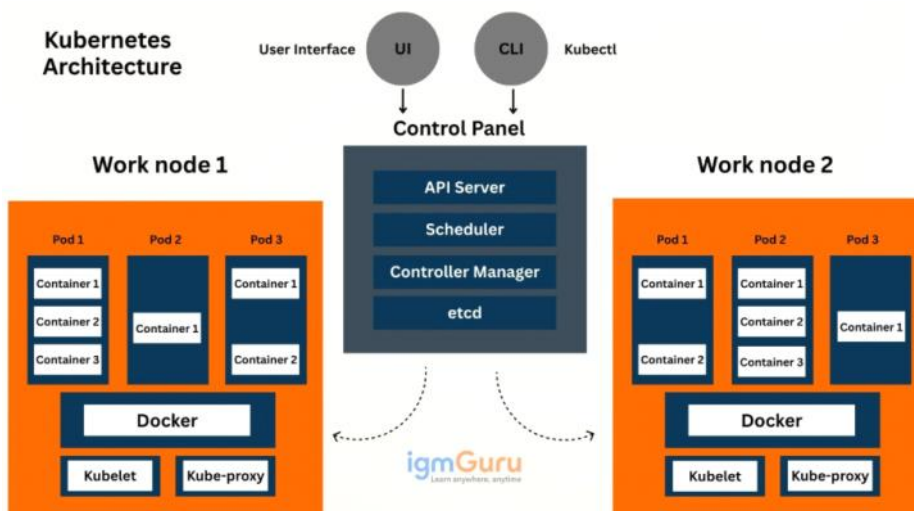
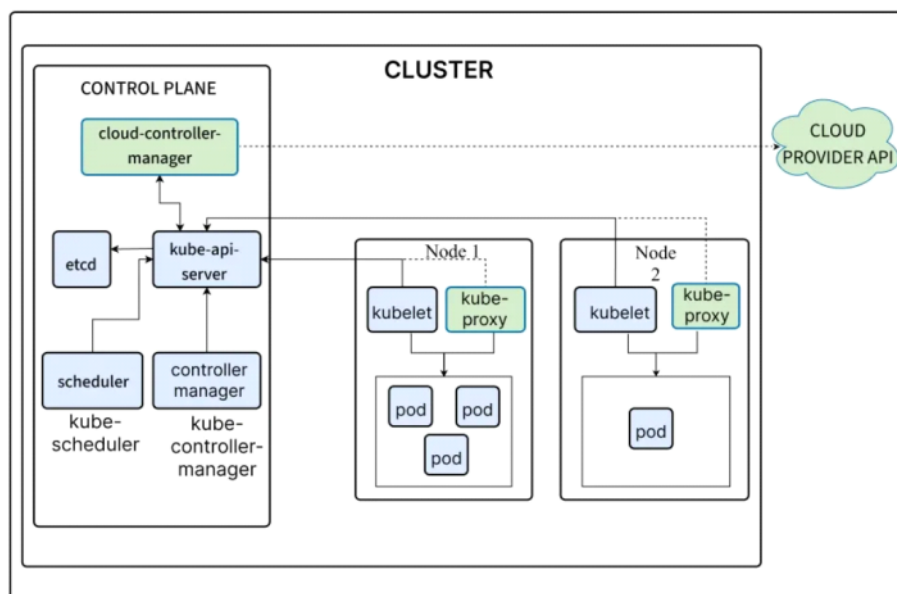
17 February 2026 13:36

What is Kubernetes?

[Kubernetes](#) often shortened to K8s (K, 8 letters, s) is an open-source Container Management tool that automates container deployment, container scaling, descaling, and container load balancing (also called a **container orchestration tool**).

- **Origin:** Developed by **Google** and written in **Golang**
- **Launch:** Officially released in **2014**.
- **CNCF Donation:** Donated to the Cloud Native Computing Foundation (CNCF) in **2015**, which now maintains it.
- **Adoption:** Widely used across major cloud providers today.

Architecture



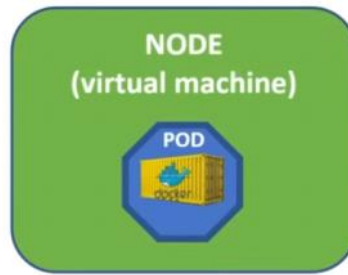
Definitions Table

Term	Definition
Pod	The smallest deployable unit in Kubernetes; a wrapper around one or more containers.
Node	A worker machine (virtual or physical) in Kubernetes that runs pods.
Cluster	A set of nodes managed by Kubernetes as a single system.
Deployment	A Kubernetes object managing ReplicaSets and Pods, enabling scaling and rolling updates.
ReplicaSet	Ensures a specified number of pod replicas are running at any given time.
ConfigMap	Stores non-sensitive configuration data for pods.
Secret	Stores sensitive data such as passwords or tokens; not encrypted by default.
Persistent Volume	A piece of storage in the cluster provisioned by an administrator or dynamically provisioned.
Persistent Volume Claim	A request for storage by a user that binds to a Persistent Volume.
Service	Provides stable network access to a set of pods, abstracting their ephemeral IPs.

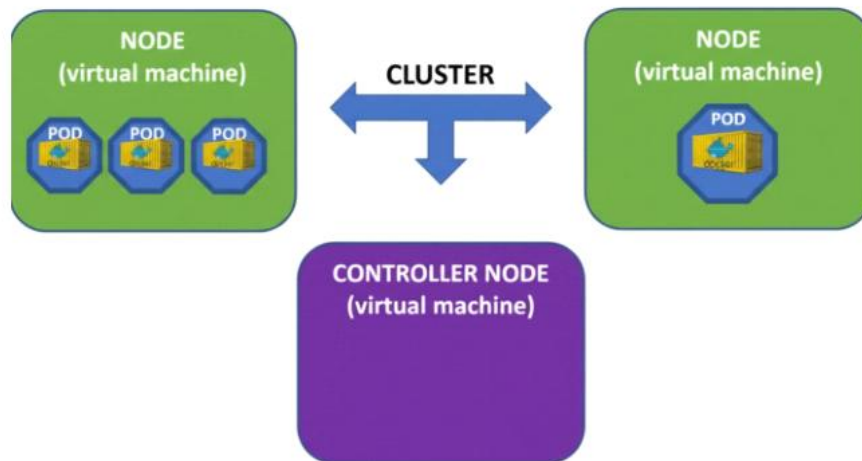
Network Policy	Controls traffic flow to and from pods based on labels and namespace selectors.
kubelet	Agent running on each node that manages pod lifecycle and container runtime integration.
API Server	Kubernetes control plane component that exposes the Kubernetes API.
etcd	Distributed key-value store containing cluster state.
Scheduler	Assigns pods to nodes based on resource availability and constraints.
Controller Manager	Manages various controllers that regulate the state of the cluster.
kube-proxy	Network proxy running on each node that facilitates service networking and load balancing.

Scoops

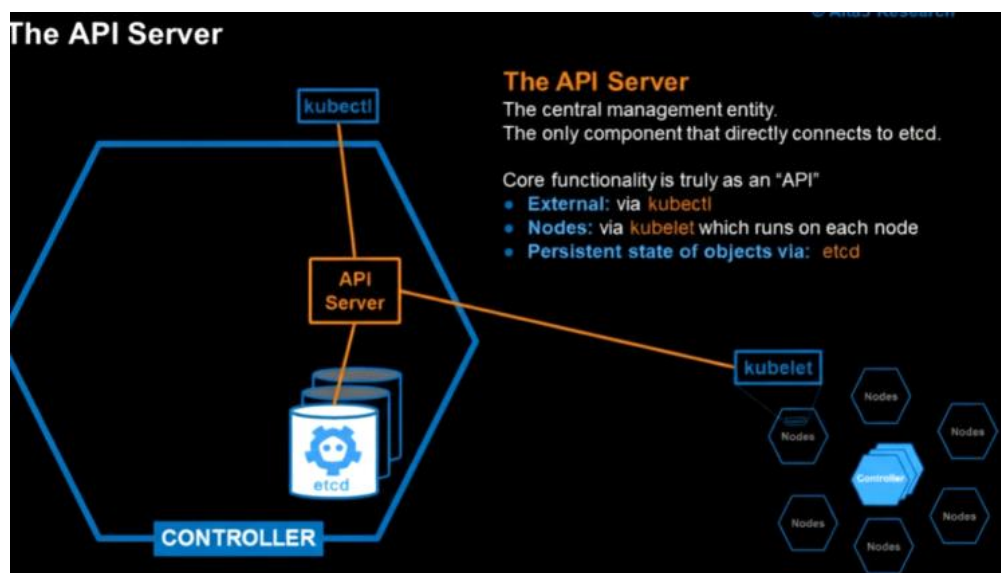
- **Containers** : We put all of our code and its dependencies inside of an isolated environment called a container. Its easy to spin or replace containers. You can create multiple copies of it.
- **Problems** with containers
 - ▶ You have multiple containers in a VM. It may happen that you **run out of room in your VM**. Thus your host machinr runs out of resources
 - ▶ Containers **die when they get ignored**.
 - ▶ If the **VM fails all containers die** even if they are healthy.
- Containers in Kubernetes are placed inside of Kubernetes objects called **pods**
- **Pod** : A pod is the **smallest unit** you can deploy in **Kubernetes**. **It wraps one or more containers** that need to run together, sharing the same network and storage. Containers inside a Pod can easily communicate and work as a single unit. **Pods** are really just **wrappers** that are being placed **around containers**.
- **Kubernetes manages pods, not containers**
- The container needs to draw its resources from somewhere. Traditionally, that can be on a virtual machine. In a Kubernetes environment, when you add virtual machines to it, those are called **nodes**
- **Node** : A node is a **machine (physical or virtual)** in a Kubernetes cluster that runs your applications. Each **Node contains** the tools needed to run Pods, including the **container runtime (like Docker)**, the **Kubelet** (agent), and the **Kube proxy** (networking).



- When you combine **all of those nodes together**, you get what are **called a Cluster**. A Kubernetes cluster is a group of computers (called nodes) that work together to run your containerized applications. These **nodes can be real machines or virtual ones**.
- There are two types of Nodes
 - ▶ **Worker Nodes** : There can be multiple worker nodes. Each worker node can have multiple pods. Each pod can have multiple containers.
 - ▶ **Controller Node** : The controller node is the hub of all of the tools that are required in order to keep your cluster running. Think of it as **the brain of the cluster**.



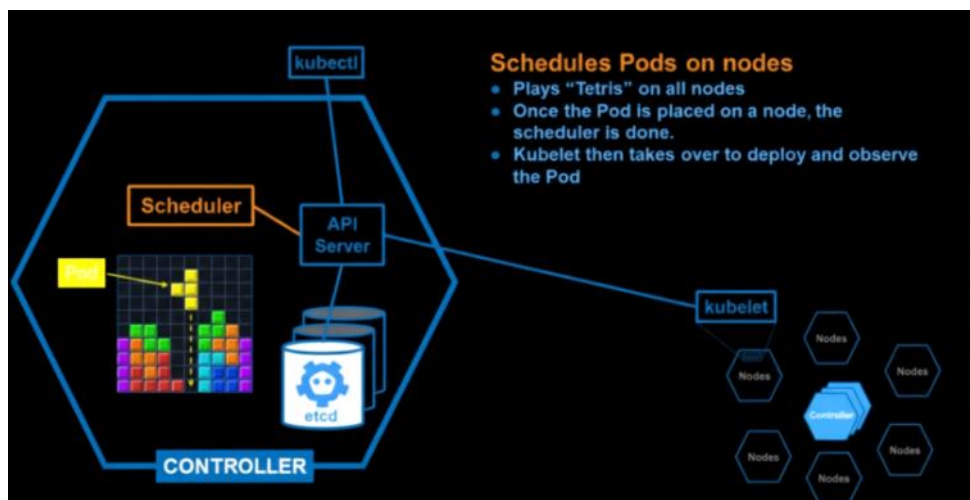
- **Kubectl** : **Kube control is the CLI tool** installed inside of your personal workstation that enables you to access the cluster. **Kubectl** can do so by connecting to the **controller node** and all **Kubectl requests are received by API Server inside Controller Node**.
- **API Server** : The **API Server** is the entry point for all Kubernetes operations. **All commands (kubectl) communicate through the API Server**. The **API server is also the only thing that talks to ETCD**



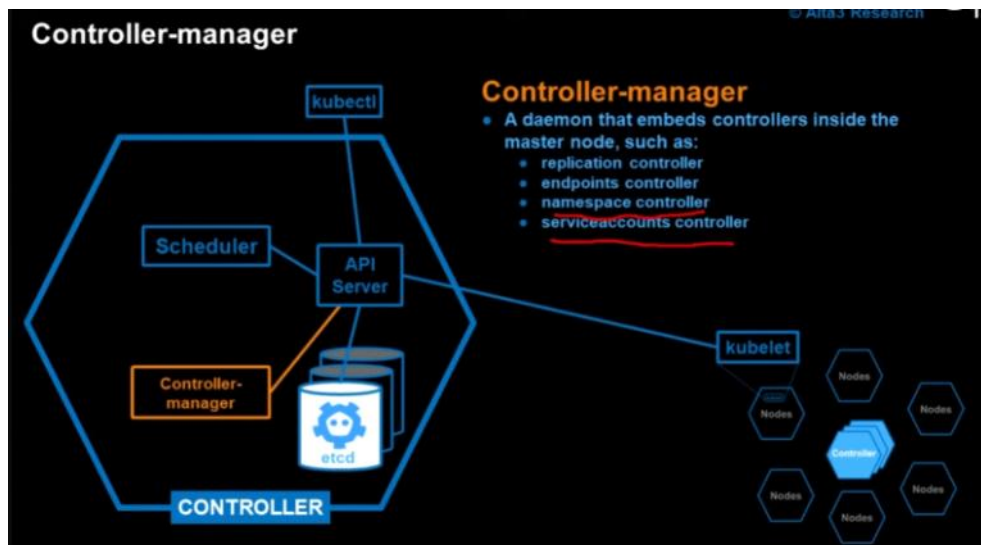
- **Kube config file** : This is a **document that should be on the same machine where kubectl is installed** and it includes things like the **location of the cluster**, where is the **certificate** and **key file** etc that will allow you to **authenticate who you are** talking to that cluster. **Without that setup, kube control is never going to work.**

```
apiVersion: v1
clusters:
- cluster:
  certificate-authority: fake-ca-file
  server: https://1.2.3.4
  name: development
contexts:
- context:
  cluster: development
  namespace: frontend
  user: developer
  name: dev-frontend
current-context: dev-frontend
kind: Config
preferences: {}
users:
- name: developer
  user:
    client-certificate: fake-cert-file
    client-key: fake-key-file
```

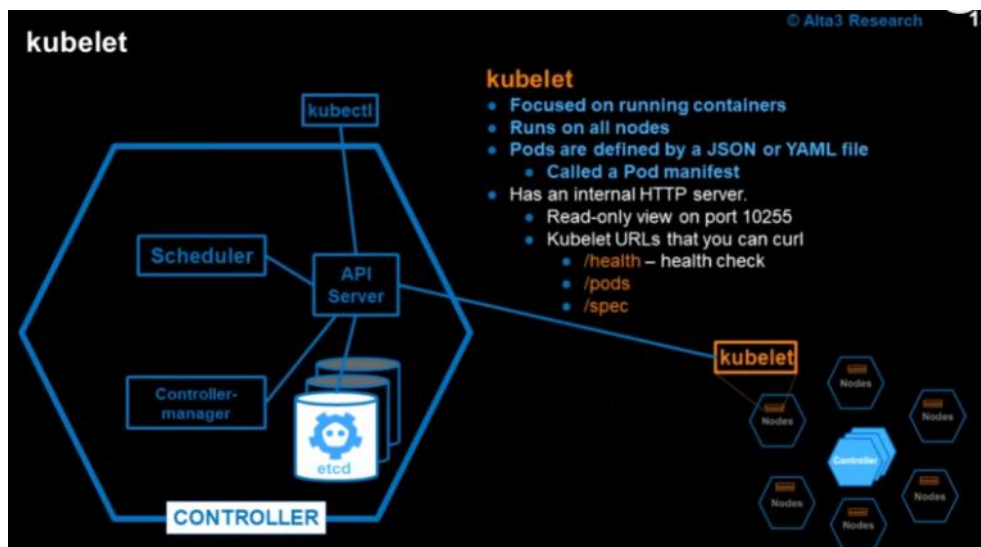
- **ETCD** : **etcd is a strongly consistent, distributed key-value store** that provides a reliable way to store data that needs to be accessed by a distributed system or cluster of machines. It is the **single source of truth in a Kubernetes cluster**
- **Scheduler** : The **scheduler assigns Pods to Nodes** based on:
 - ▶ Resource availability
 - ▶ Constraints
 - ▶ Affinity rules (either put by you or system itself)



- **Controller Manager** : The Controller Manager **runs controllers that maintain desired state** like **replica controller, endpoint controller** etc.

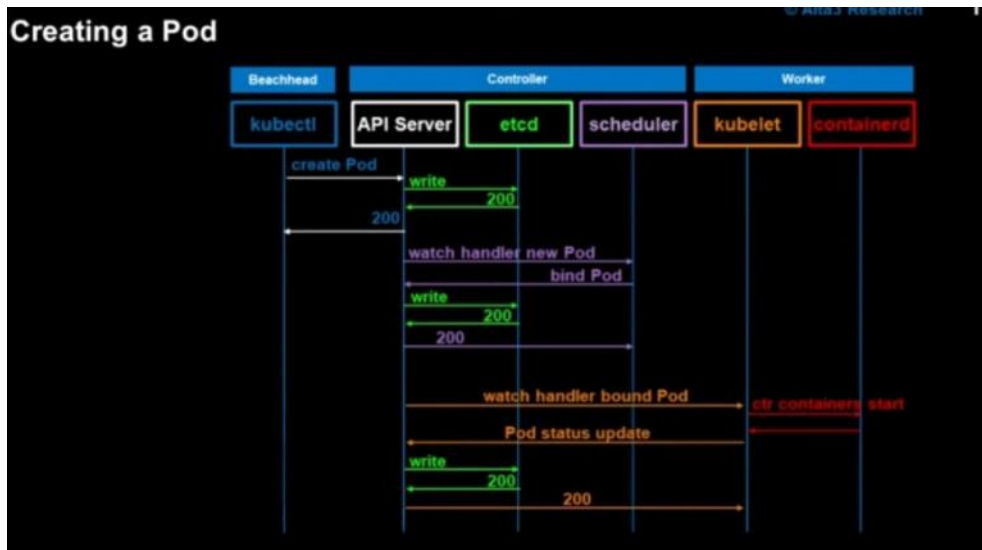


- **Kubelet**: The kubelet runs on each node and ensures containers in Pods are running. It's responsible for making sure the containers are started, stopped or restarted appropriately on each node in your cluster. It communicates with the API server. Kubelet is responsible for writing the commands inside of your container runtime environment, be it docker or podman or any other containerization technology.



Creation of a Pod : Workflow

1. You use Kubectl to write create command
2. Kubectl reaches out to API Server which then validates the commands and if everything is correct, it writes this instruction to etcd. Etcd then gives a 200 ok response to API Server which eventually is returned to you.
3. The API Server then reaches out to scheduler for it to know where to create the pod. The scheduler decides on a node (eg. worker node 2) where the pod needs to be created and sends back the information to API Server.
4. API server then writes this information to etcd and sends back a confirmation to scheduler.
5. API Server then sends information to kubelet of worker node 2 for pod creation.
6. Kubelet then creates a pod and then it spins up containers by running all the necessary commands in the container runtime environment. Through out the process, kubelet will keep on feeding status updates to API Server, informing it about the current status of pod creation.
7. API Server then keep on writing those information in the etcd



- **Manifest File** : Its file which states how your pod should be created.

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
  
```

Working with Kubernetes Objects and Manifests

- Objects are typically created declaratively via YAML manifests that describe the desired state.
- Key YAML concepts:
 - ▶ **Lists**: Denoted by dashes (-).
 - ▶ **Dictionaries** (Mappings): Key-value pairs separated by colons.
 - ▶ **Indentation**: Critical for nesting and ownership of keys.
- Required manifest fields:
 - ▶ **apiVersion**: Specifies the API version (must be correct and updated with Kubernetes upgrades).
 - ▶ **kind**: Object type (e.g., Pod, Deployment).
 - ▶ **metadata**: Includes name, namespace, and optional labels.
 - ▶ **spec**: Details the desired configuration.

Important Kubectl commands

Summary of Important kubectl Commands	
Command	Purpose
<code>kubectl get <resource></code>	List resources (pods, nodes, services, etc.)
<code>kubectl describe <resource></code>	Show detailed info and events for resource
<code>kubectl apply -f <file></code>	Create or update resources from manifest file
<code>kubectl delete <resource></code>	Delete resources
<code>kubectl label <resource></code>	Add, update, or remove labels
<code>kubectl logs <pod></code>	View logs from pod containers
<code>kubectl exec -it <pod> -- sh</code>	Open shell inside a container
<code>kubectl port-forward <pod></code>	Forward local port to pod port
<code>kubectl scale deployment</code>	Change number of pod replicas
<code>kubectl rollout</code>	Manage deployment rollouts and rollbacks

- **Namespace and Resource Management :** A **Namespace** is a logical isolation boundary within a cluster.
- Objects belong to namespaces; if unspecified, **they default to the default namespace.**
- **Resource Quotas:** Resource Quotas can be **applied per namespace** to limit CPU, memory, and other resource usage.
- Container resource requests and limits can be set in manifests to prevent starvation or excessive consumption.
 - ▶ **Resource Requests :** Minimum amount of resources your pod needs to run or else it will starve.
 - ▶ **Resource Limit :** This is the maximum amount your Pod's container is allowed to consume before being cut off.

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-resources-demo
  namespace: pod-resources-example
spec:
  resources:
    limits:
      cpu: "1"
      memory: "200Mi"
    requests:
      cpu: "1"
      memory: "100Mi"
  containers:
    - name: pod-resources-demo-ctr-1
      image: nginx
      resources:
        limits:
          cpu: "0.5"
          memory: "100Mi"
        requests:
          cpu: "0.5"
          memory: "50Mi"
    - name: pod-resources-demo-ctr-2
      image: fedora
      command:
        - sleep
        - inf

```

Probes for Pod Health : Probes are basically watchdogs that keeps track of pods/containers to enforce a certain behaviour.

- ▶ **Liveness Probes**: Detect if a container **is alive**; restart containers if unresponsive.
- ▶ **Readiness Probes**: Check if a container **is ready to serve traffic**; exclude it from service endpoints if not ready.
- Probes use HTTP requests or other mechanisms with configurable parameters like **delay, timeout, and failure thresholds**.

```

apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
    - name: liveness
      image: registry.k8s.io/e2e-test-images/agnhost:2.40
      args:
        - liveness
      livenessProbe:
        httpGet:
          path: /healthz
          port: 8080
          httpHeaders:
            - name: Custom-Header
              value: Awesome
        initialDelaySeconds: 3
        periodSeconds: 3

```

- Liveness probe attributes

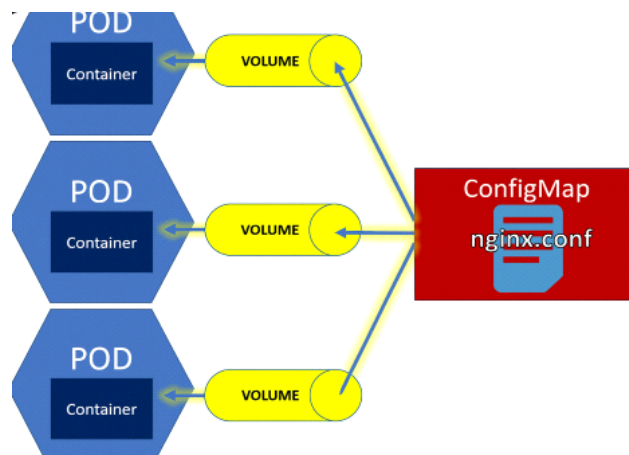
```

- containerPort: 9876
livenessProbe:
  initialDelaySeconds: 2 # HOW SOON AFTER CREATION ARE WE PROBING?
  periodSeconds: 5 # HOW OFTEN THEREAFTER ARE WE PROBING?
  timeoutSeconds: 1 # HOW LONG WE GIVING THE CONTAINER TO RESPOND?
  failureThreshold: 3 # HOW MANY CONSECUTIVE FAILURES BEFORE WE KILL THIS CONTAINER?
  httpGet:
    path: /health
    port: 9876

# POD IS CREATED > 2 (PROBE) > 5 (PROBE) > 5 ...

```

- Liveness and Readiness probe attributes are exactly the same.
- **kubectl port-forward** creates a temporary secure tunnel from a local machine to a Pod or Service inside a Kubernetes cluster, primarily used for debugging and development without exposing the service externally. Traffic goes through the API server.
 - ▶ **Forward to a Pod** : `kubectl port-forward pod/<pod-name> <local-port>:<pod-port>`
 - ▶ **Forward to a Service** : `kubectl port-forward service/<service-name> <local-port>:<service-port>`
- Limitations** : Not for production traffic. Not scalable. Single user tunnel.
- **ConfigMap** : A ConfigMap is a Kubernetes object used to externalize non-sensitive configuration data from container images, allowing applications to be configured dynamically across environments without rebuilding images.
 - ▶ Environment variables
 - ▶ Config files
 - ▶ Application properties
- Attaching ConfigMaps to Pod.
 - ▶ The configMap which contains the config file inside of it is **mounted as a volume** to the pod itself
 - ▶ **Once that volume has been mounted to the pod, we then mount the volume to the container.** The volume is mounted to the container **at a specific mount point**, which is going to be some arbitrary directory inside the container
- A single configMap can be mounted to as many pods as you want.



- Manifest attributes for mounting the volume where you created the configMap.

```

apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    configMap:
      name: myconfigmap

```

- **subPath:** By default Mount Path will overwrite existing directories. This is where we use subPath. SubPath says, instead of mapping the entire contents of the entire configMaps to the entire directory of the mountPath, lets write a subPath to just a file inside of the configMap

```

volumeMounts:
- mountPath: /var/lib/mysql
  name: site-data
  subPath: mysql

```

- A container using a ConfigMap as a [subPath](#) volume mount will not receive ConfigMap updates.
- **Secrets:** A Secret is a Kubernetes object used to store and manage sensitive data like passwords, keys, tokens etc separately from container images, allowing secure injection of credentials into Pods via environment variables or mounted volumes.
- Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store (etcd). Anyone with API access can retrieve or modify a Secret, and so can anyone with access to etcd. Encryption must be configured separately.

Types of Secrets

Built-in Type	Usage
Opaque	arbitrary user-defined data
kubernetes.io/service-account-token	ServiceAccount token
kubernetes.io/dockercfg	serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file
kubernetes.io/basic-auth	credentials for basic authentication
kubernetes.io/ssh-auth	credentials for SSH authentication
kubernetes.io/tls	data for a TLS client or server
bootstrap.kubernetes.io/token	bootstrap token data

- Logs: Kubernetes itself does not store logs permanently. Instead:
 - ▶ Applications write logs to **stdout** and **stderr**.
 - ▶ The container runtime (e.g., containerd) captures these logs.
 - ▶ The node stores them as files.
 - ▶ You retrieve them using **kubectl** or forward them to a logging system.
- By default, logs are stored on the node at: **/var/log/containers/**
- Viewing Logs with **kubectl logs <pod-name>**. By default this will give you the logs of all the containers in the order in which they are listed in the manifest file. If you want to get **log of a particular**

container then you need to mention the container name.

`logs <pod-name> -c <container-name>`

- **Pod Labels and Selectors:** Labels are key-value metadata attached to objects to group and select them. Used extensively by Deployments, Services, and Network Policies for resource identification and management.

Applying Labels

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod
  labels:
    runtime: webby
    ver: 2
    ssd: true
    env: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.18
    ports:
    - containerPort: 80
```

- We can use selector tag to select objects belonging to same label

```
Every 0.1s: kubectl get pods -L alta3                                bchd: Thu May 29 14:30:19 2025
```

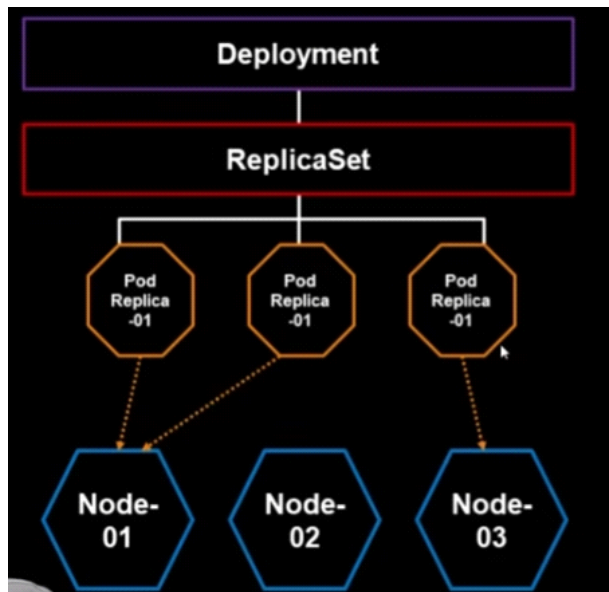
NAME	READY	STATUS	RESTARTS	AGE	ALTA3
another-pod	1/1	Running	0	82s	awesome
another-pod2	1/1	Running	0	45s	
demo-pod	1/1	Running	0	5m7s	awesome

```
student@bchd:~$
student@bchd:~$
student@bchd:~$ kubectl get pods --selector=alta3=awesome
```

NAME	READY	STATUS	RESTARTS	AGE
another-pod	1/1	Running	0	79s
demo-pod	1/1	Running	0	5m4s

```
student@bchd:~$
student@bchd:~$
```

- **Deployment :** A Deployment manages Pods and ensures a specified number of replicas are running. It provides:
 - ▶ Rolling updates
 - ▶ Rollbacks
 - ▶ Self-healing
 - ▶ Scaling
- Deployments manage ReplicaSets. It is actually **ReplicaSet that is responsible for pod creation/deletion/updation** in a deployment.



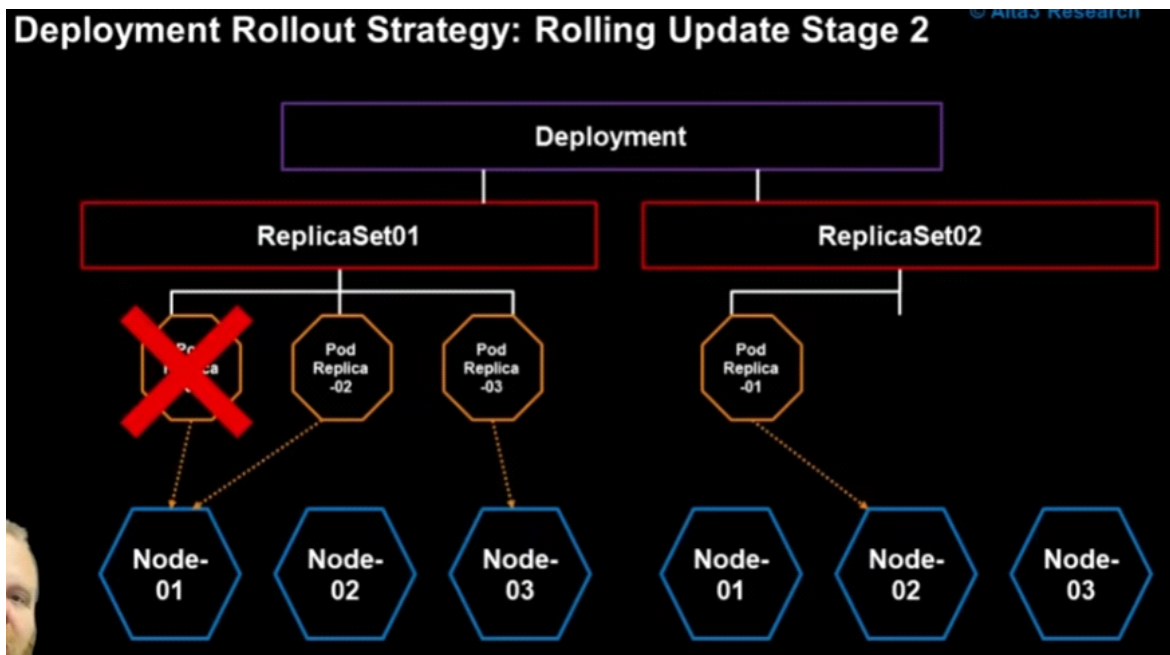
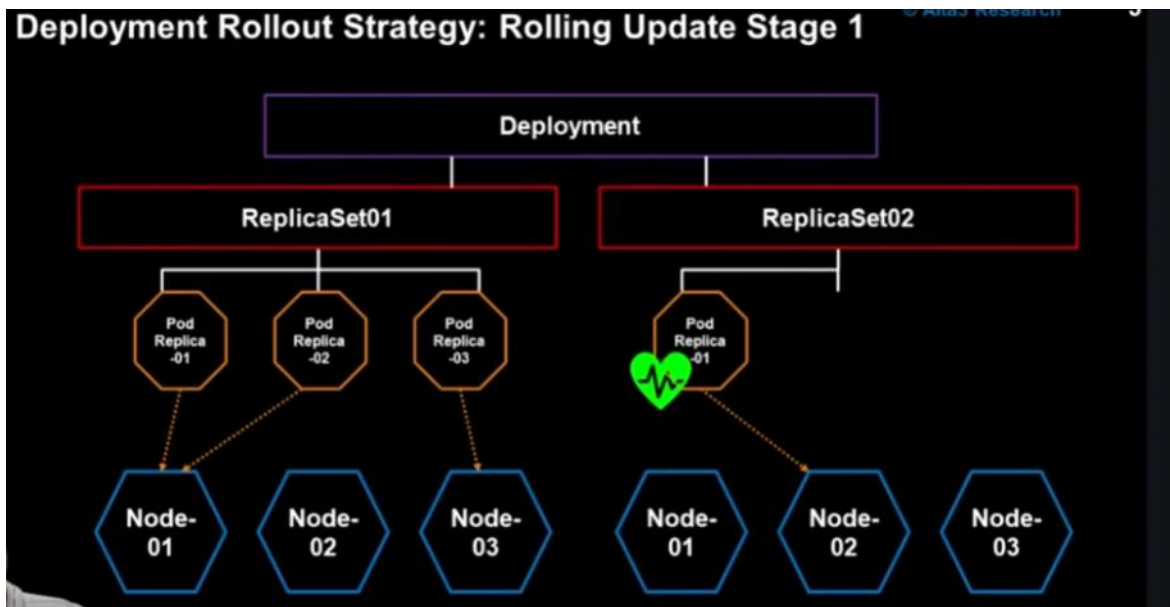
- **ReplicaSet** : A ReplicaSet ensures a specified number of identical Pods are running at all times. It is usually **managed by a Deployment**.
- Pods are rarely created standalone in production; instead, use Deployments:
 - ▶ **Manage replicas** (multiple pod copies).
 - ▶ Automatically **replace failed pods**.
 - ▶ Support **rolling updates with zero downtime** by creating new ReplicaSets and removing old ones.
 - ▶ Enable **rollbacks to previous versions**.
- Deployments use labels to select pods they manage; removing a pod's label causes the deployment to replace it.
- Deployment manifest file looks like this.

```

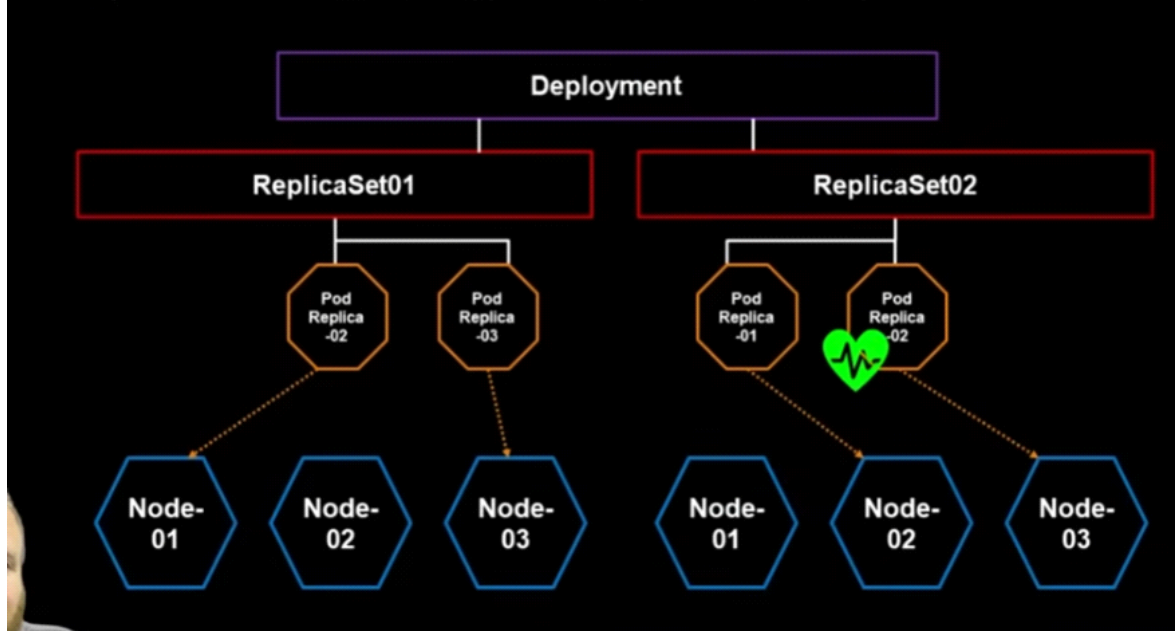
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment           Name of the deployment
  labels:                          Labels that this deployment targets
    app: nginx
spec:
  replicas: 3                      Create 3 replicas which matches label 'nginx'
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:                      While creating containers attach label as 'nginx'
        app: nginx
    spec:
      containers:
        - name: nginx              This is the basic details of the container
          image: nginx:1.14.2      that is going to be created
          ports:
            - containerPort: 80
  
```

- If you want to delete all the pods in a deployment, then you either want to **delete the deployment or scale the pods to zero**.
- **Advantages of using a Deployment over ReplicaSets:**
 - ▶ Does everything a ReplicaSet can do, PLUS:
 - ▶ **Version control** (roll out, roll back)

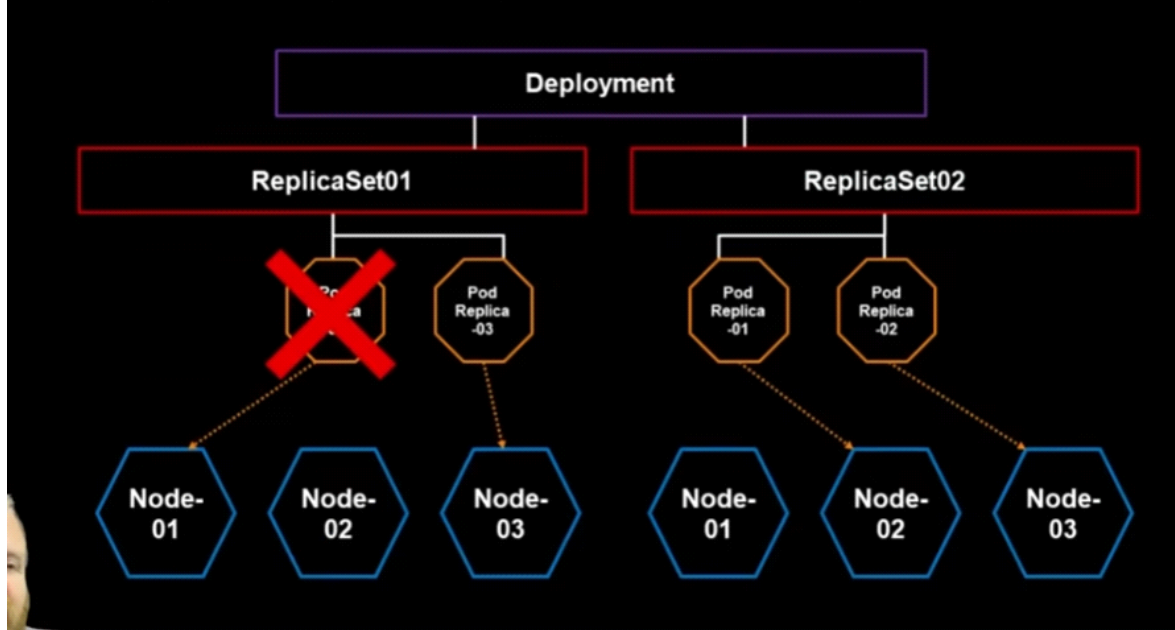
- ▶ **Zero down time**
- ▶ **Accepts any changes made to yaml**, EXPECTS change!
- In a **rolling deployment process**, the deployment creates a **new ReplicaSet with the same configuration and then terminates the existing replicaSet**. As it always tries to maintain the fixed number of replicas mentioned in the manifest file, as it creates one pod in the new ReplicaSet, it keeps on destroying pods in the existing replicaSet so as to always maintain the desired number of pods.
- If for some reason you want to roll back to the previous version of deployment, then the same process follows just in reverse order.



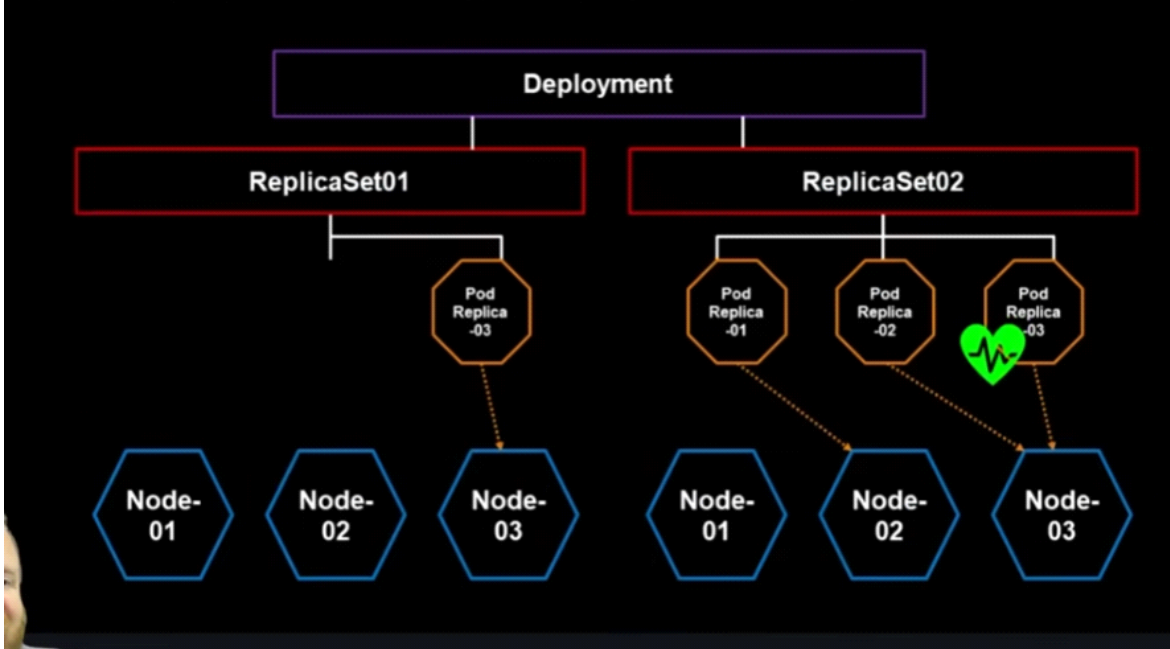
Deployment Rollout Strategy: Rolling Update Stage 3



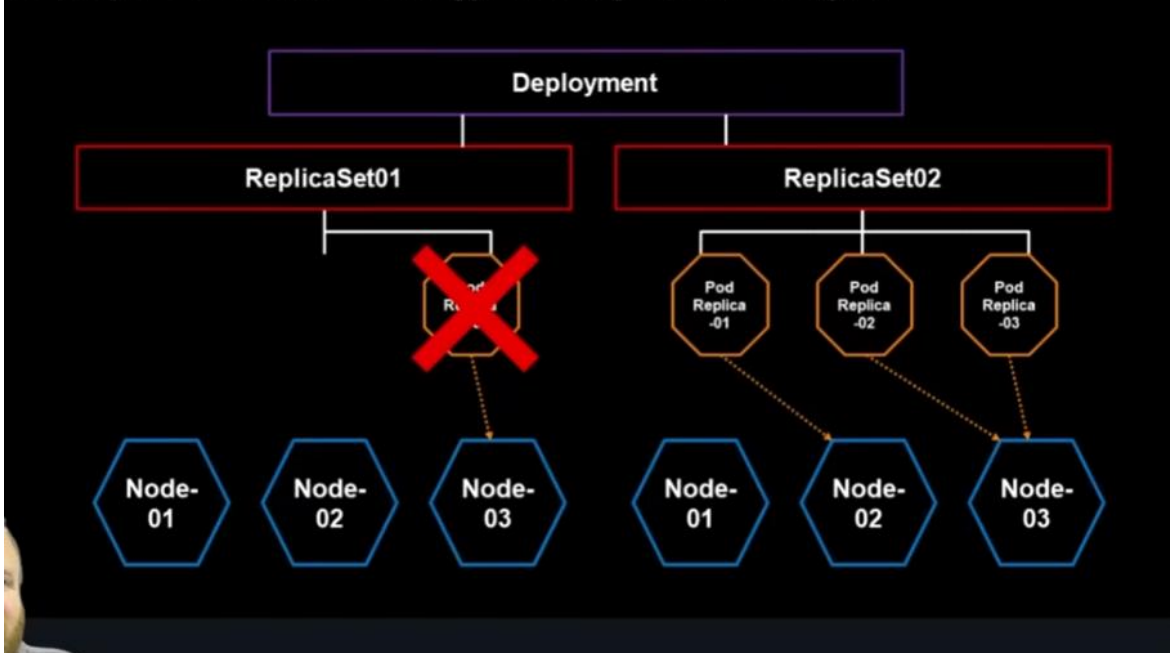
Deployment Rollout Strategy: Rolling Update Stage 4



Deployment Rollout Strategy: Rolling Update Stage 5



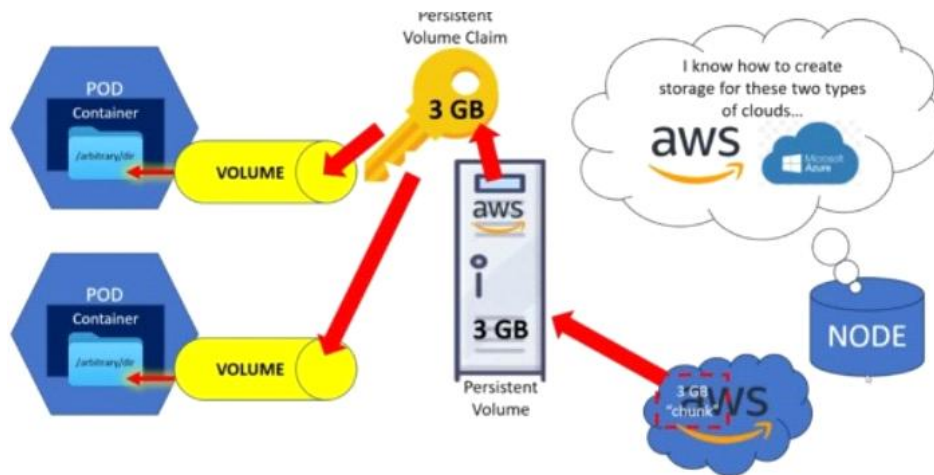
Deployment Rollout Strategy: Rolling Update Stage 6



- History commands keeps track of how many revisions does a particular deployment has gone through
`kubectl rollout history deploy <deployment-name>`
`kubectl rollout undo history deploy <deployment-name>` -> Reverse a deployment to previous version

Storage in Kubernetes

- Persistent Volumes (PV)** : PV represents storage resources from the underlying infrastructure (cloud, local disk). It is a cluster-level storage.
- Persistent Volumes represents real storage like:**
 - ▶ EBS
 - ▶ NFS
 - ▶ Cloud disks
- Persistent Volume Claims (PVC)** : PVC is a request for storage by pods; **PVC binds to a PV.**
- Volume Mounting:** Pods mount PVCs as volumes, which are then mounted inside containers at specific paths for persistent data storage outside container lifecycle.



- Each Persistent Volume can be mounted on multiple containers from different pods.
- **Storage Class** : Storage classes define types and parameters of storage, making Kubernetes storage-agnostic.
- A **StorageClass** tells Kubernetes:
 - ▶ Which **provisioner** to use (EBS, Azure Disk, Ceph, etc.)
 - ▶ What **type of storage** (SSD, HDD, premium, etc.)
 - ▶ Replication and performance parameters
 - ▶ Reclaim policy
 - ▶ Volume binding mode
- Instead of manually creating PVs, Kubernetes dynamically provisions them when a **PersistentVolumeClaim (PVC)** requests storage.

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: low-latency
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Retain # default value is Delete
allowVolumeExpansion: true
mountOptions:
  - discard # this might enable UNMAP / TRIM at the block storage layer
volumeBindingMode: WaitForFirstConsumer
parameters:
  guaranteedReadWriteLatency: "true" # provider-specific

```

- PersistentVolume Manifest file

```

---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: alta3-pv
spec:
  storageClassName: manual
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data"

```

- PersistentVolume Claim Manifest file

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nginx-pvc
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

```

- Pod Manifest File

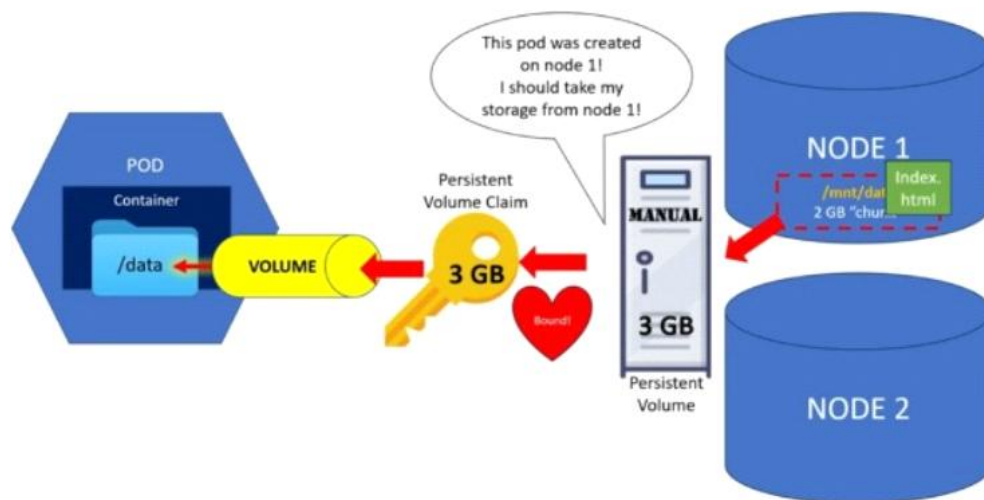
```

spec:
  containers:
    - name: nginx-with-pv
      image: nginx:1.18.0
      ports:
        - name: http
          containerPort: 80
          protocol: TCP
      nodeSelector:
        kubernetes.io/hostname: node-1
      volumeMounts:
        - name: nginx-pv-storage
          mountPath: "/data"
  volumes:
    - name: nginx-pv-storage
      persistentVolumeClaim:
        claimName: nginx-pvc

```

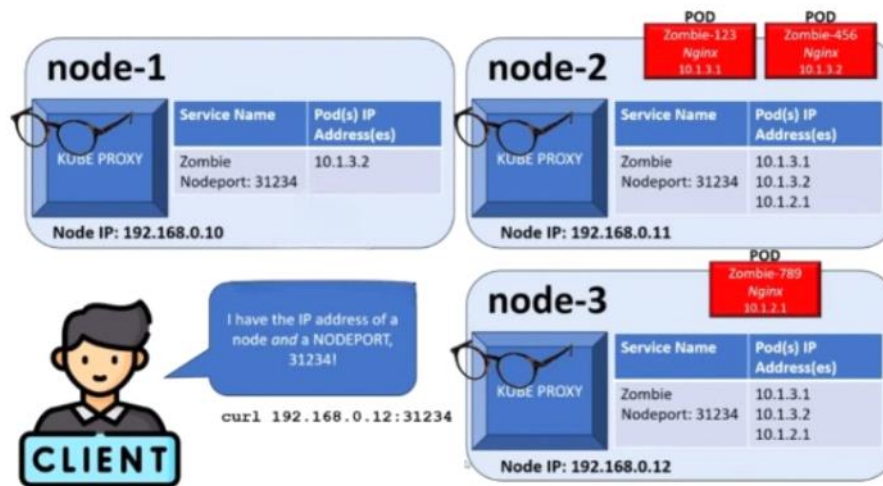
- **Node Selector (Optional):**
 - ▶ Used to schedule pods on specific nodes, important in scenarios where storage is node-specific.
- **Access Modes**
 - ▶ **ReadWriteOnce** (RWO) – Read Write by one node at a time.
 - ▶ **ReadOnlyMany** (ROX) – Many nodes (read-only)

- ▶ **ReadWriteMany** (RWX) – Many nodes (read-write)
- What happens when PVC is deleted?
 - ▶ **Delete** → Deletes underlying storage
 - ▶ **Retain** → Keeps storage (manual cleanup required)
- **volumeBindingMode**
 - ▶ **Immediate** → Volume created immediately
 - ▶ **WaitForFirstConsumer** → Volume created when Pod is scheduled (Best for multi-zone clusters)
- When a PVC does not specify a **storageClassName**, the default StorageClass is used.
- If you set the [storageclass.kubernetes.io/is-default-class](https://kubernetes.io/docs/concepts/storage/storage-classes/#is-default-class) annotation to true on more than one StorageClass in your cluster, and you then create a PersistentVolumeClaim with no storageClassName set, Kubernetes uses the most recently created default StorageClass.
- **You should try to have only default StorageClass in your cluster.**
- The storage is taken from the node where the pods are created

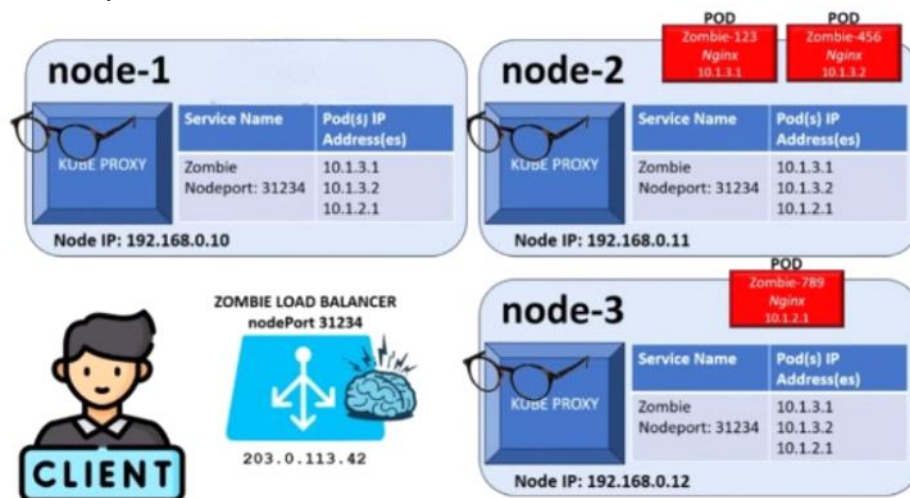


Networking and Services

- Each pod gets a unique, **ephemeral IP address**; these change on pod recreation.
- **Service** : A Service **provides a stable IP and DNS name to access pods regardless of their IP changes.**
- Types of Services:
 - ▶ **ClusterIP** : **Internal cluster access only.**
 - ▶ **NodePort** : **Exposes service on a static port on each node** for external access. **It allows external access into the cluster without using kubectl.**
 - If you have the nodeport , then you can be **routed to any node in the cluster**. Its not necessary that you will be directed to node1 only, if you use IP of that node and use port as NodePort. NodePort is global to the cluster.

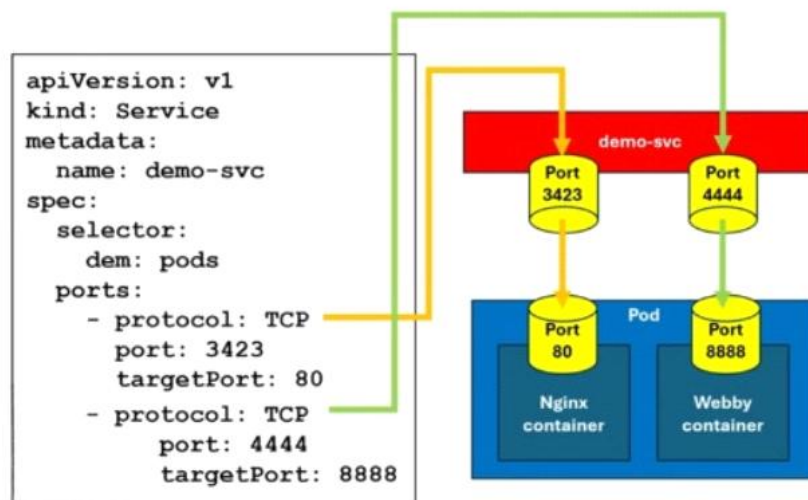


- **LoadBalancer** : Uses external load balancers (cloud providers) to route traffic to node ports. It uses nodeport internally.



- Description table

Service Type	Description	Use Case
ClusterIP	Default; exposes service on internal cluster IP only	Internal pod-to-pod communication
NodePort	Exposes service on each node's IP at a static port between 30000-32767	External access to cluster via node IP+port
LoadBalancer	Uses cloud provider's external load balancer to expose service externally	Managed external access with load balancing



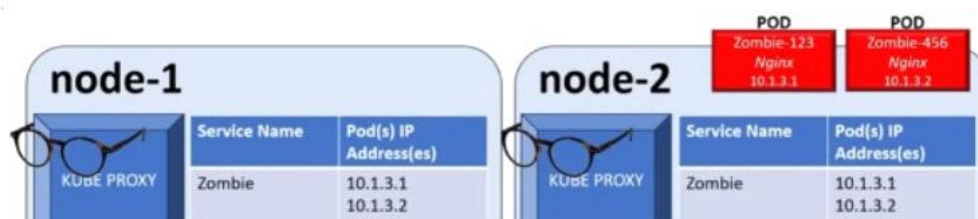
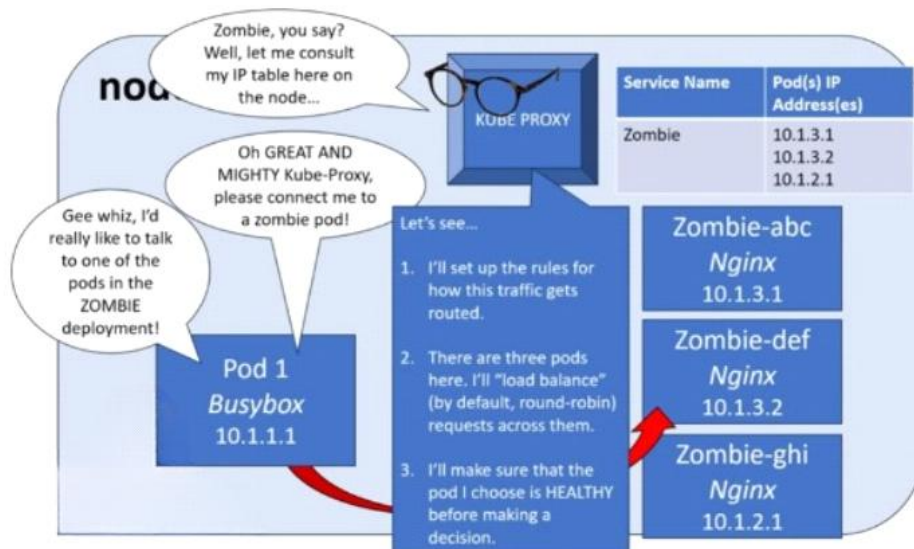
- Services **use selectors (usually labels) to identify the pods they route traffic to.**
- Pod-to-Pod Communication:
 - ▶ Pods can communicate across nodes and namespaces.
 - ▶ Services route traffic to healthy pods dynamically.
- **Network Policies:**
 - ▶ Network policies control ingress (incoming) and egress (outgoing) traffic based on pod selectors, namespaces, IP blocks, and ports.
 - ▶ **Network policies act like firewall rules** at the pod level to **restrict or allow pod communication.**
 - ▶ Applied per namespace, and policy rules can be very granular.
 - ▶ **Policies are namespace-scoped** and **use label selectors to target pods.**
 - ▶ Rules specify allowed sources/destinations by **IP ranges, namespaces, or pod labels.**
 - ▶ By default, pods allow all traffic unless restricted by policies.
 - ▶ Network policies provide fine-grained security controls to restrict pod communication.

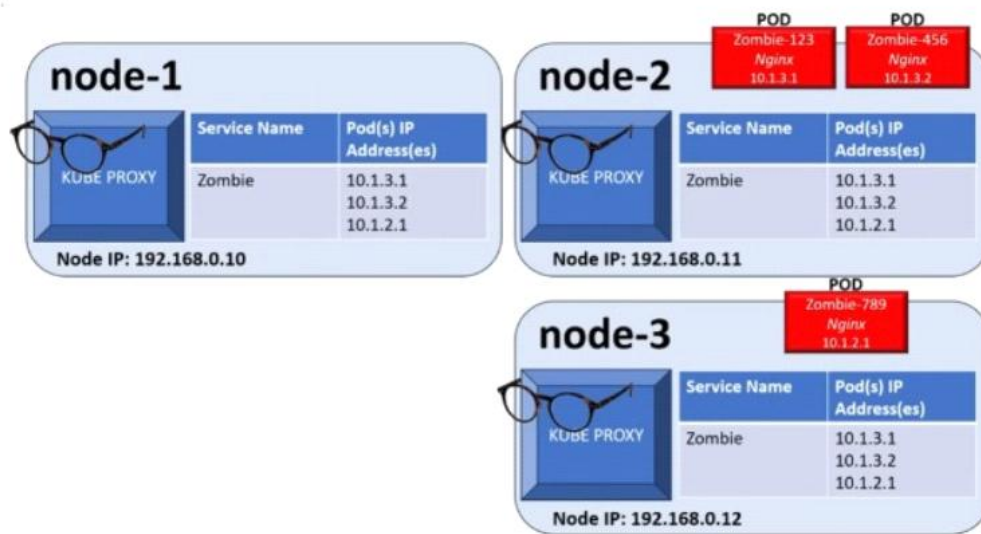
```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default # target the NS of the pods you wish to control
spec:
  podSelector: # what pods are you controlling?
    matchLabels:
      role: db
  policyTypes:
    - Ingress # traffic coming IN
    - Egress # traffic going OUT
  ingress:
    - from:
        - ipBlock: # must be from this IP range
            cidr: 172.17.0.0/16
            except:
              - 172.17.1.0/24
        - namespaceSelector: # must be from this NS
            matchLabels:
              project: myproject
        - podSelector: # must be from this pod with given label
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP # traffic coming only to given port
          port: 6379
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.0.0/24
      ports:
        - protocol: TCP
          port: 5978

```

- **kube-proxy** : It manages **network routing and load balancing** for Services.
- kube-proxy runs on each node, managing network rules and routing traffic to appropriate pods using round-robin load balancing.





What is Kubernetes?

[Kubernetes](#) often shortened to K8s (K, 8 letters, s) is an open-source Container Management tool that automates container deployment, container scaling, descaling, and container load balancing (also called a container orchestration tool).

- **Origin:** Developed by Google and written in GoLang
- **Launch:** Officially released in 2014.
- **CNCF Donation:** Donated to the Cloud Native Computing Foundation (CNCF) in 2015, which now maintains it.
- **Adoption:** widely used across major cloud providers today.

Core Components

◇ 1. Cluster

A **Kubernetes Cluster** is a set of machines (nodes) managed by Kubernetes to run containerized applications. It consists of:

- Control Plane
- Worker Nodes

◇ 2. Node

A **Node** is a machine (VM or physical) in the cluster that runs application workloads.

Each node runs:

- kubelet
- container runtime (containerd / Docker)
- kube-proxy

◇ 3. Control Plane

The **Control Plane** manages the cluster and maintains the desired state.

Core components:

- API Server
- Scheduler
- Controller Manager
- etcd

◇ 4. Pod

A **Pod** is the smallest deployable unit in Kubernetes.

It:

- Wraps one or more containers
- Shares network (same IP)
- Shares storage volumes

Pods are ephemeral and not meant to be created directly in production.

◇ 5. Deployment

A **Deployment** manages Pods and ensures a specified number of replicas are running.

It provides:

- Rolling updates
- Rollbacks
- Self-healing
- Scaling

Deployments manage ReplicaSets.

◇ 6. ReplicaSet

A **ReplicaSet** ensures a specified number of identical Pods are running at all times. It is usually managed by a Deployment.

◇ 7. Service

A **Service** provides stable networking access to a set of Pods.

Since Pods are ephemeral, Services:

- Provide a stable IP/DNS
- Load balance traffic

Types:

- ClusterIP (internal)
- NodePort (exposes on node)
- LoadBalancer (cloud external)

◇ 8. Namespace

A **Namespace** is a logical isolation boundary within a cluster.

Used to:

- Separate environments (dev, prod)
- Apply resource quotas
- Control access

◇ 9. ConfigMap

A **ConfigMap** stores non-sensitive configuration data for applications.

Used to inject:

- Environment variables
- Config files
- Application properties

◇ 10. Secret

A **Secret** stores sensitive data like:

- Passwords
- API keys
- Certificates

Secrets are base64-encoded and can be mounted as env vars or files.

◇ 11. Volume

A **Volume** is storage attached to a Pod.

Unlike containers, volumes:

- Persist data across container restarts (within the Pod)

◇ 12. PersistentVolume (PV)

A **PersistentVolume** is cluster-level storage provisioned for use by applications.

It represents real storage like:

- EBS
- NFS
- Cloud disks

◇ 13. PersistentVolumeClaim (PVC)

A **PersistentVolumeClaim** is a request for storage by a Pod.

Pods use PVCs to consume storage without knowing the underlying storage details.

◇ 14. StatefulSet

A **StatefulSet** manages stateful applications that require:

- Stable network identity
- Persistent storage
- Ordered deployment

Used for:

- Databases
- Kafka
- Distributed systems

◇ 15. DaemonSet

A **DaemonSet** ensures a Pod runs on every node (or selected nodes).

Used for:

- Logging agents
- Monitoring agents
- Security agents

◇ 16. Job

A **Job** runs a Pod to completion.

Used for:

- Batch processing
- One-time tasks
- Data migration

◇ 17. CronJob

A **CronJob** runs Jobs on a scheduled basis (like Linux cron).

◇ 18. Ingress

An **Ingress** manages external HTTP/HTTPS access to services.

It provides:

- Routing rules
- Host/path-based routing
- SSL termination

Requires an Ingress Controller.

◇ 19. Ingress Controller

An **Ingress Controller** is the component that implements Ingress rules.

Examples:

- NGINX Ingress Controller
- AWS ALB Controller

◇ 20. kube-apiserver

The **API Server** is the entry point for all Kubernetes operations.

All commands (kubectl) communicate through the API Server.

◇ 21. etcd

etcd is a distributed key-value store that stores the cluster state.

It is the single source of truth for Kubernetes.

◇ 22. Scheduler

The **Scheduler** assigns Pods to Nodes based on:

- Resource availability
- Constraints
- Affinity rules

◇ 23. Controller Manager

The **Controller Manager** runs controllers that maintain desired state.

Examples:

- Replica controller
- Node controller
- Endpoint controller

◇ 24. kubelet

The **kubelet** runs on each node and ensures containers in Pods are running. It communicates with the API server.

◇ 25. kube-proxy

kube-proxy manages network routing and load balancing for Services.

◇ 26. Resource Requests & Limits

- **Request** = Minimum guaranteed CPU/memory
- **Limit** = Maximum allowed CPU/memory

Used for:

- Scheduling
- Preventing resource abuse
- Avoiding OOMKilled

◇ 27. Horizontal Pod Autoscaler (HPA)

HPA automatically scales Pods based on:

- CPU usage
- Memory
- Custom metrics

◇ 28. Liveness Probe

Checks if a container is alive.

If it fails:

- Container is restarted.

◇ 29. Readiness Probe

Checks if a container is ready to receive traffic.

If it fails:

- Traffic is stopped but container is not restarted.

◇ 30. Desired State

The **Desired State** is the configuration you define in YAML.

Kubernetes continuously works to make actual state match desired state.

This reconciliation loop is Kubernetes' core principle.

Resources

17 February 2026 20:56

<https://www.youtube.com/watch?v=MTHGoGUFpvE>

<https://www.geeksforgeeks.org/devops/kubernetes-architecture/>

<https://www.geeksforgeeks.org/devops/introduction-to-kubernetes-k8s/>

<https://kubernetes.io/docs/home/>

System designs can involve a dizzying array of different [technologies](#), [concepts](#) and [patterns](#), but one technology (arguably) stands above the rest in terms of its *versatility*: Redis. This versatility is important in an interview setting because it allows you to go deep. Instead of learning about dozens of different technologies, you can learn a few useful ones and learn them deeply, which magnifies the chances that you're able to get to the level your interviewer is expecting.

Beyond versatility, Redis is great for its *simplicity*. Redis has a ton of features which resemble data structures you're probably used to from coding (hashes, sets, sorted sets, streams, etc) and which, given a few basics, are easy to reason about how they behave in a distributed system. While many databases involve a lot of magic (optimizers, query planners, etc), with only minor exceptions Redis has remained quite simple and good at what it does best: executing simple operations **fast**.

Ok, Redis is versatile, simple, and useful for system design interviews. Let's learn how it works.

Redis Basics

Redis is a self-described "[data structure store](#)" written in C. It's in-memory ☐ and single threaded ☐ making it very fast and easy to reason about.

One important reason you might not want to use Redis is because you need **durability**. While there are some reasonable strategies for (using Redis' Append-Only File [AOF](#)) to *minimize* data loss, you don't get the same guarantees you might get from e.g. a relational database about commits being written to disk. This is an intentional tradeoff made by the Redis team in favor of speed, but alternative implementations (e.g. [AWS' MemoryDB](#)) will compromise a bit on speed to give you disk-based durability. If you need it, it's there!

Some of the most fundamental data structures supported by Redis:

- Strings
- Hashes (objects/dictionaries)
- Lists
- Sets
- Sorted Sets (Priority Queues)
- Bloom Filters (probabilistic set membership; allows false positives)
- Geospatial Indexes
- Time Series

In addition to simple data structures, Redis also supports different communication patterns like Pub/Sub and Streams, partially standing in for more complex setups like Apache Kafka or AWS SNS (Simple Notification Service) / SQS (Simple Queue Service).

The core structure underneath Redis is a key-value store. Keys are strings while values which can be any of the data structures supported by Redis: binary data and strings, sets, lists, hashes, sorted sets, etc. All objects in Redis have a key.

Redis logical model

The choice of keys is important as these keys might be stored in separate nodes based on your [infrastructure configuration](#). Effectively, the way you organize the keys will be the way you organize your data and scale your Redis cluster.

Commands

Redis' wire protocol is a custom query language comprised of simple strings which are used for all functionality of Redis. The CLI is really simple, you can literally connect to a Redis instance and run these commands from the CLI.

```
SET foo 1
GET foo    # Returns 1
INCR foo    # Returns 2
XADD mystream * name Sara surname OConnor # Adds an item to a stream
```

The [full set of commands](#) is surprisingly readable, when grouped by data structure. As an example, Redis' Sets support simple operations like adding an element to the set (SADD), getting the number of elements or cardinality (SCARD, the count of set members), listing those elements (SMEMBERS) and checking existence (SISMEMBER) - close analogs to what you would have with a Set in any general purpose programming language.

Infrastructure Configurations

Redis can run as a single node, with a high availability (HA) replica, or as a cluster. When operating as a cluster, Redis clients cache a set of "hash slots" which map keys to a specific node. This way clients can directly connect to the node which contains the data they are requesting.

Redis infrastructure configurations

Think of hash slots like a phone book: the client keeps a local map from slots to nodes. If a slot moves during rebalancing or failover, the server replies with MOVED and the client refreshes its map (e.g. via CLUSTER SHARDS).

Each node maintains some awareness of other nodes via a gossip protocol so, in limited instances, if you request a key from the wrong node you can be redirected to the correct node. But Redis' emphasis is on performance so hitting the correct endpoint first is a priority.

Compared to most databases, Redis clusters are surprisingly basic (and thus, have some pretty severe limitations on what they can do). Rather than solving scalability

problems for you, Redis can be thought of as providing you some basic primitives on which you can solve them. As an example, with few exceptions, Redis expects all the data for a given request to be on a single node! **Choosing how to structure your keys is how you scale Redis.**

Performance

Redis is really, really fast. Redis can handle $O(100k)$ writes per second and read latency is often in the microsecond range. This scale makes some anti-patterns for other database systems actually feasible with Redis. As an example, firing off 100 SQL requests to generate a list of items with a SQL database is a terrible idea, you're better off writing a SQL query which returns all the data you need in one request. On the other hand, the overhead for doing the same with Redis is rather low - while it'd be great to avoid it if you can, it's doable.

This is completely a function of the in-memory nature of Redis. It's not a good fit for every use case, but it's a great fit for many.

Capabilities

Redis as a Cache

The most common deployment scenario of Redis is as a cache. In this case, the root keys and values of Redis map to the keys and values in our cache. Redis can distribute this hash map trivially across all the nodes of our cluster enabling us to scale without much fuss - if we need more capacity we simply add nodes to the cluster.

For example, you might cache a product under key `product:123` with the value stored as a JSON blob or a Redis Hash containing fields like `name`, `price`, and `inventoryCount`.

When using Redis as a cache, you'll often employ a time to live (TTL) on each key. Redis guarantees you'll never read the value of a key after the TTL has expired and the TTL is used to decide which items to evict from the server - keeping the cache size manageable even in the case where you're trying to cache more items than memory allows.

Using Redis in this fashion doesn't solve one of the more important problems caches face: the ["hot key" problem](#), though Redis is not unique in this respect vs alternatives like Memcached or other highly scaled databases like DynamoDB.

Redis as a cache

Redis as a Distributed Lock

Another common use of Redis in system design settings is as a distributed lock. Occasionally we have data in our system and we need to maintain consistency during updates (e.g. the very common [Design Ticketmaster](#) system design question), or we need to make sure multiple people aren't performing an action at the same time

(e.g. [Design Uber](#)).

Most databases (including Redis) will offer *some* consistency guarantees. If your core database can provide consistency, don't rely on a distributed lock which may introduce extra complexity and issues. Your interviewer will likely ask you to think through the edge cases in order to make sure you really understand the concept.

A very simple distributed lock with a timeout might use the atomic increment (INCR) with a TTL. This is basically a shared counter. When we want to try to acquire the lock, we run INCR. If the response is 1 (i.e. we were the first person to try to grab the lock, so we own it!), we proceed. If the response is > 1 (i.e. someone else beat us and has the lock), we wait and retry again later. When we're done with the lock, we can DEL the key so that other processes can make use of it.

More sophisticated locks in Redis can use the [Redlock algorithm](#) together with fencing tokens if you want an airtight solution.

Redis for Leaderboards

Redis' sorted sets maintain ordered data which can be queried in log time which make them appropriate for leaderboard applications. The high write throughput and low read latency make this especially useful for scaled applications where something like a SQL DB will start to struggle.

In [Post Search](#) we have a need to find the posts which contain a given keyword (e.g. "tiger") which have the most likes (e.g. "Tiger Woods made an appearance..." @ 500 likes).

We can use Redis' sorted sets to maintain a list of the top liked posts for a given keyword. Periodically, we can remove low-ranked posts to save space.

```
ZADD tiger_posts 500 "SomeId1" # Add the Tiger woods post
ZADD tiger_posts 1 "SomeId2" # Add some tweet about zoo tigers
ZREMRANGEBYRANK tiger_posts 0 -6 # Remove all but the top 5 posts
```

Redis for Rate Limiting

As a data structure server, implementing a wide variety of rate limiting algorithms is possible. A common algorithm is a fixed-window rate limiter where we guarantee that the number of requests does not exceed N over some fixed window of time W.

Implementation of this in Redis is simple. When a request comes in, we increment (INCR) the key for our rate limiter and check the response. If the response is greater than N, we wait. If it's less than N, we can proceed. We call EXPIRE on our key so that after time period W, the value is reset.

If you need a sliding window, you can store timestamps in a Sorted Set per key and remove old entries before counting; run the check in Lua to keep it atomic.

Redis for Proximity Search

Redis natively supports geospatial indexes with commands like GEOADD and GEOSEARCH. The basic commands are simple:

```
GEOADD key longitude latitude member # Adds "member" to the index at key "key"
GEOSEARCH key FROMLONLAT longitude latitude BYRADIUS radius unit # Searches the index at key "key" at specified position and radius
```

The search command, in this instance, runs in $O(N+\log(M))$ time where N is the number of elements in the radius and M is the number of items inside the shape.

Why do we have both N and M ? Redis' geospatial commands use geohashes under the hood to index the data. These geohashes allow us to grab candidates in grid-aligned bounding boxes. But these boxes are square and imprecise. A second pass takes the candidates and filters them to only include items that are within the exact radius.

Redis for Event Sourcing

Redis' streams are append-only logs similar to Kafka's topics. The basic idea behind Redis streams is that we want to durably add items to a log and then have a distributed mechanism for consuming items from these logs. Redis solves this problem with streams (managed with commands like XADD) and consumer groups (commands like XREADGROUP and XCLAIM).

Redis streams and consumer groups

A simple example is a work queue. We want to add items to the queue and have them processed. At any point in time one of our workers might fail, and in these instances we'd like to re-process them once the failure is detected. With Redis streams we add items onto the queue with commands like XADD and have a single consumer group attached to the stream for our workers. This consumer group is maintaining a reference to the items processed via the stream and, in the case a worker fails, provides a way for a new worker to claim (XCLAIM) and restart processing that message.

Redis for Pub/Sub

Redis natively supports a publish/subscribe (Pub/Sub) messaging pattern, allowing messages to be broadcast to multiple subscribers in real time. This is useful for building chat systems, real-time notifications, or any scenario where you want to decouple message producers from consumers (more discussion on this in our [Realtime Updates](#) pattern).

People frequently call out limitations of Redis Pub/Sub that are no longer true, e.g. Redis pub/sub is now sharded which enables scalability which was not possible in previous versions!

The basic commands are straightforward:

SPUBLISH channel message # Sends a message to all subscribers of 'channel' (the S prefix means "sharded")
SSUBSCRIBE channel # Listens for messages on 'channel'

When a client subscribes to a channel, it will receive any messages published to that channel as long as the connection remains open. This makes Pub/Sub great for ephemeral, real-time communication, but it's important to note that messages are not persisted—if a subscriber is offline when a message is published, it will miss that message entirely.

Pub/Sub clients **use a single connection to each node in the cluster** (rather than a connection per channel). Generally speaking, this means that in most cases you'll use a number of connections equal to the number of nodes in your cluster. It also means that you don't need millions of connections even if you have millions of channels!

Redis Pub/Sub

Redis Pub/Sub is simple and fast, but not durable. The delivery of messages is "at most once" which means that if a subscriber is offline when a message is published, it will miss that message entirely. If you need message persistence, delivery guarantees, or the ability to replay missed messages, consider using Redis Streams or a dedicated message broker like Kafka or RabbitMQ.

Pub/Sub is a great fit for interview scenarios where you need to demonstrate real-time communication patterns, but be ready to discuss its limitations and when you might need a more robust solution.

Need offline delivery or durable fan-out? Redis Streams are a good option or you can pair Pub/Sub with a queue (e.g., SNS→SQS, Kafka) or outbox pattern (i.e. write the messages to a database) so consumers can catch up later.

Can I roll my own Pub/Sub?

Some candidates recoil at the idea of using Redis' native Pub/Sub because they're concerned about scalability (often stemming from a misunderstanding, thinking that Pub/Sub uses a connection per channel). The typical proposal looks like this:

Instead of using Redis Pub/Sub, we'll create keys for each topic with the server address as a value. Then, when a user wants to publish a message to that topic, they can look up the key and send the message directly to that server.

While there may be some applications of this, it has some acute downsides.

First, the number of network hops is increased. With Pub/Sub, when we want to send a message, the pathway looks like this:

1. Client sends message to Pub/Sub node
2. Pub/Sub node dispatches message to all subscribers

Two network hops. With the homegrown Pub/Sub, the pathway looks like this:

1. Client requests subscribers for topic key from Redis
2. Redis responds with servers to contact
3. Client sends message to each server

Three network hops. The last hop is the most expensive because it's likely that we don't already have a TCP connection established to each subscriber. When we set up a Pub/Sub connection, we're establishing (and holding open) a TCP connection to each node. This makes the message quick to send. With our homegrown approach, we need to establish these new connections for each server before we send.

And next, we need to consider the resident memory cost. With Pub/Sub, we're only keeping track of channels that have active subscribers. When the last subscriber for a channel disconnects, the channel is removed from memory (publishes to that channel aren't received by anyone). With our homegrown approach, when a server goes down we need to somehow learn about it to remove the entry from our map. This may require some sort of heartbeat mechanism where each server reports "hey, I'm still alive listening to this topic", either explicitly or using a TTL. This adds a lot of complexity and chatter to the system, especially if the number of topics is high.

All said, if you have a use-case that seems like Pub/Sub, use Pub/Sub!

Shortcomings and Remediations

Hot Key Issues

If our load is not evenly distributed across the keys in our Redis cluster, we can run into a problem known as the "hot key" issue. To illustrate it, let's pretend we're using Redis to cache the details of items in our ecommerce store. We have lots of items so we scale our cluster to 100 nodes and our items are evenly spread across them. So far, so good. Now imagine one day we have a surge of interest for *one particular item*, so much that the volume for this item matches the volume for the rest of the items.

Hot key issue

Now the load on *one server* is dramatically higher than the rest of the servers. Unless we were severely overprovisioned (i.e. we were only using a small % of the existing CPU on each node), this server is now going to start failing.

There are lots of potential solutions for this, all with tradeoffs.

- We can add an in-memory cache in our clients so they aren't making so many requests to Redis for the same data.
- We can store the same data in multiple keys and randomize the requests so they are spread across the cluster.
- We can add read replica instances and dynamically scale these with load.

For an interview setting, the important thing is that you recognize potential hot key issues (+) and that you proactively design remediations (++).

Summary

Redis is a powerful, versatile, and simple tool you can use in system design interviews. Because Redis' capabilities are based on simple data structures, reasoning through the scaling implications of your decisions is straightforward: allowing you to go deep with your interviewer without needing to know a lot of details about Redis internals.

From <<https://www.hellointerview.com/learn/system-design/deep-dives/redis>>

ELK - Elasticsearch - Logstash - Kibana

22 January 2026 15:40

At a high level, the ELK stack (Elasticsearch, Logstash, Kibana) ingests data from multiple sources, processes it, stores it in a distributed search engine, and then visualizes it in dashboards for monitoring and analytics. It's widely used for log management, observability, and real-time search.

Core Components of ELK

Component	Role	Key Functions
Logstash	Data pipeline	Collects, parses, and transforms data from diverse sources (logs, metrics, events).
Elasticsearch	Search & analytics engine	Stores data in a distributed index, supports fast full-text search, aggregations, and analytics.
Kibana	Visualization layer	Provides dashboards, charts, and query interfaces to explore and monitor data.

How ELK Works (Step by Step)

1. Data Ingestion

- Logs, metrics, or events are collected from applications, servers, or cloud services.
- Tools like **Logstash** or **Beats** (lightweight shippers) forward this data.

2. Data Processing

- Logstash applies filters (e.g., parsing JSON, extracting fields, enriching with metadata).
- Data is normalized into a structured format.

3. Indexing & Storage

- Processed data is sent to **Elasticsearch**, which distributes it across nodes for scalability.
- Data is indexed for fast search and retrieval.

4. Search & Analytics

- Elasticsearch supports queries, aggregations, and full-text search.
- Enables anomaly detection, trend analysis, and correlation across logs.

5. Visualization

- **Kibana** connects to Elasticsearch, allowing users to build dashboards, charts, and alerts.
- Teams can monitor system health, track KPIs, and investigate incidents in real time.

Typical Use Cases

- **Log management:** Centralizing logs from microservices, servers, and containers.
- **Monitoring & observability:** Tracking application performance and infrastructure health.
- **Security analytics:** Detecting anomalies, intrusion attempts, or suspicious activity.
- **Business intelligence:** Analyzing customer behavior, transactions, or operational data.

Challenges & Trade-offs

- **Scaling:** Elasticsearch clusters need careful tuning for large data volumes.
- **Storage costs:** Logs can grow quickly; retention policies are critical.
- **Complexity:** Managing pipelines and indices requires operational expertise.
- **Performance:** Query speed depends on index design and hardware resources.

Best Practices

- Use **Beats** for lightweight data shipping.
- Apply **index lifecycle management** to control storage costs.
- Secure the stack with **role-based access control (RBAC)** and TLS.
- Monitor cluster health with Kibana and Elastic's monitoring tools.

 In short: **ELK = Collect (Logstash/Beats) → Store & Search (Elasticsearch) → Visualize (Kibana)**. It's a powerful, flexible stack for turning raw data into actionable insights.

Overview (AI generated)

26 March 2026 11:59

For interview prep as a Solutions Architect, you need to master Amazon S3's fundamentals, advanced features, and architectural use cases. Interviewers will test not only your knowledge of S3 concepts but also how you apply them in designing scalable, secure, and cost-optimized systems.

Core Concepts You Must Know

- **Object Storage Model:** S3 stores data as *objects* inside *buckets*. Each object has data, metadata, and a unique key.
- **Buckets:** Globally unique names, region-specific, used for organizing objects.
- **Durability & Availability:** S3 provides **11 nines (99.999999999%) durability** and high availability by replicating data across multiple AZs.
- **Storage Classes:**
 - **Standard** – frequent access.
 - **Intelligent-Tiering** – automatic cost optimization.
 - **Standard-IA / One Zone-IA** – infrequent access.
 - **Glacier / Glacier Deep Archive** – archival storage.
- **Data Consistency:** Strong read-after-write consistency for PUTs and DELETES.

Security & Access Control

- **IAM Policies:** Fine-grained access control.
- **Bucket Policies:** Resource-based permissions.
- **ACLs (Access Control Lists):** Legacy, less recommended.
- **Encryption:**
 - **SSE-S3** (managed keys).
 - **SSE-KMS** (customer-managed keys).
 - **Client-side encryption.**
- **Block Public Access:** Prevents accidental exposure.
- **VPC Endpoints:** Private connectivity to S3 without traversing the internet.

Performance & Optimization

- **Multipart Uploads:** Efficient for large files (>100 MB).
- **Transfer Acceleration:** Uses CloudFront edge locations for faster uploads/downloads.
- **Prefix Partitioning:** S3 automatically scales, but understanding key distribution helps avoid performance bottlenecks.
- **Requester Pays Buckets:** Useful for shared datasets.

Cost Management

- **Lifecycle Policies:** Transition objects between storage classes or expire them.
- **Intelligent-Tiering:** Automatically moves objects to cheaper tiers.

- **Monitoring:** Use CloudWatch metrics, S3 Storage Lens, and Cost Explorer.

Architectural Use Cases

- **Static Website Hosting:** S3 + CloudFront for global distribution.
- **Data Lake Foundation:** Store raw/processed data for analytics (Athena, Redshift Spectrum).
- **Backup & Disaster Recovery:** Cross-region replication (CRR) for resilience.
- **Hybrid Cloud Storage:** With AWS Storage Gateway.
- **Event-Driven Architectures:** S3 triggers Lambda, SNS, or SQS on object events.

Risks & Challenges

- **Data Exposure:** Misconfigured bucket policies can lead to breaches.
- **Cost Overruns:** Without lifecycle rules, storage costs can balloon.
- **Latency:** S3 is not block storage; unsuitable for low-latency transactional workloads.
- **Vendor Lock-in:** Heavy reliance on S3 APIs may limit portability.

Mnemonic: “SAFE DATA”

Each letter represents a key concept you should highlight in interviews.

- **S – Security**
IAM policies, bucket policies, encryption (SSE-S3, SSE-KMS), block public access, VPC endpoints.
- **A – Availability & Durability**
11 nines durability, multi-AZ replication, strong consistency.
- **F – Flexibility in Storage Classes**
Standard, Intelligent-Tiering, IA, One Zone-IA, Glacier, Glacier Deep Archive.
- **E – Event-Driven Integrations**
S3 triggers Lambda, SNS, SQS for serverless workflows and pipelines.
- **D – Data Lifecycle Management**
Lifecycle policies, versioning, replication (same-region, cross-region).
- **A – Access Optimization**
Transfer Acceleration, multipart uploads, requester pays, prefix partitioning.
- **T – Total Cost Control**
Intelligent-Tiering, lifecycle rules, monitoring with Storage Lens & CloudWatch.
- **A – Architecture Use Cases**
Static website hosting, data lakes, backups, disaster recovery, hybrid storage.

 **Interview Tip:** Go beyond definitions—show how you’d use S3 in real-world architectures (e.g., secure data lakes, disaster recovery, serverless pipelines). Employers want to see *design thinking*, not just memorization.

1. Durability vs Availability

- **Durability:** Probability your data won't be lost. S3 offers **99.999999999% (11 nines)** durability by replicating objects across multiple AZs.
- **Availability:** Probability you can access data when needed. S3 Standard offers **99.99% availability**.

☞ Interview phrasing: *"Durability is about data survival, availability is about data accessibility."*

2. Securing Sensitive Data in S3

- **IAM Policies:** Control who can access buckets/objects.
 - **Bucket Policies:** Resource-based rules for access.
 - **Encryption:** SSE-S3, SSE-KMS, or client-side.
 - **Block Public Access:** Prevent accidental exposure.
 - **VPC Endpoints:** Private access without internet.
- ☞ Emphasize *defense-in-depth*: multiple layers of security.

3. Storage Classes & When to Use

- **Standard:** Frequent access, mission-critical.
 - **Intelligent-Tiering:** Automatically optimizes cost for unpredictable access patterns.
 - **Standard-IA:** Infrequent access, but still high availability.
 - **One Zone-IA:** Cheaper, but stored in one AZ (not for critical data).
 - **Glacier / Deep Archive:** Archival, retrieval in minutes to hours.
- ☞ Interview tip: Tie each class to a *use case* (e.g., logs, backups, archives).

4. Cost-Effective Backup Strategy

- **Versioning:** Protect against accidental deletes/overwrites.
 - **Lifecycle Policies:** Transition old backups to IA or Glacier.
 - **Cross-Region Replication (CRR):** Disaster recovery.
 - **Intelligent-Tiering:** For unpredictable workloads.
- ☞ Example: *"Daily backups in Standard, older ones auto-transition to Glacier after 30 days."*

5. Bucket Policies vs IAM Policies vs ACLs

- **IAM Policies:** User/role-based, identity-centric.
 - **Bucket Policies:** Resource-based, applied at bucket level.
 - **ACLs:** Legacy, object-level permissions (rarely recommended).
- ☞ Interview phrasing: *"IAM = who, Bucket Policy = what resource, ACL = legacy fine-grained."*

6. S3 + Lambda Integration

- **Event Notifications:** S3 triggers Lambda on PUT/DELETE.

- **Use Cases:** Image resizing, log processing, ETL pipelines.
🔗 Example: *"Upload triggers Lambda → process → store results back in S3."*

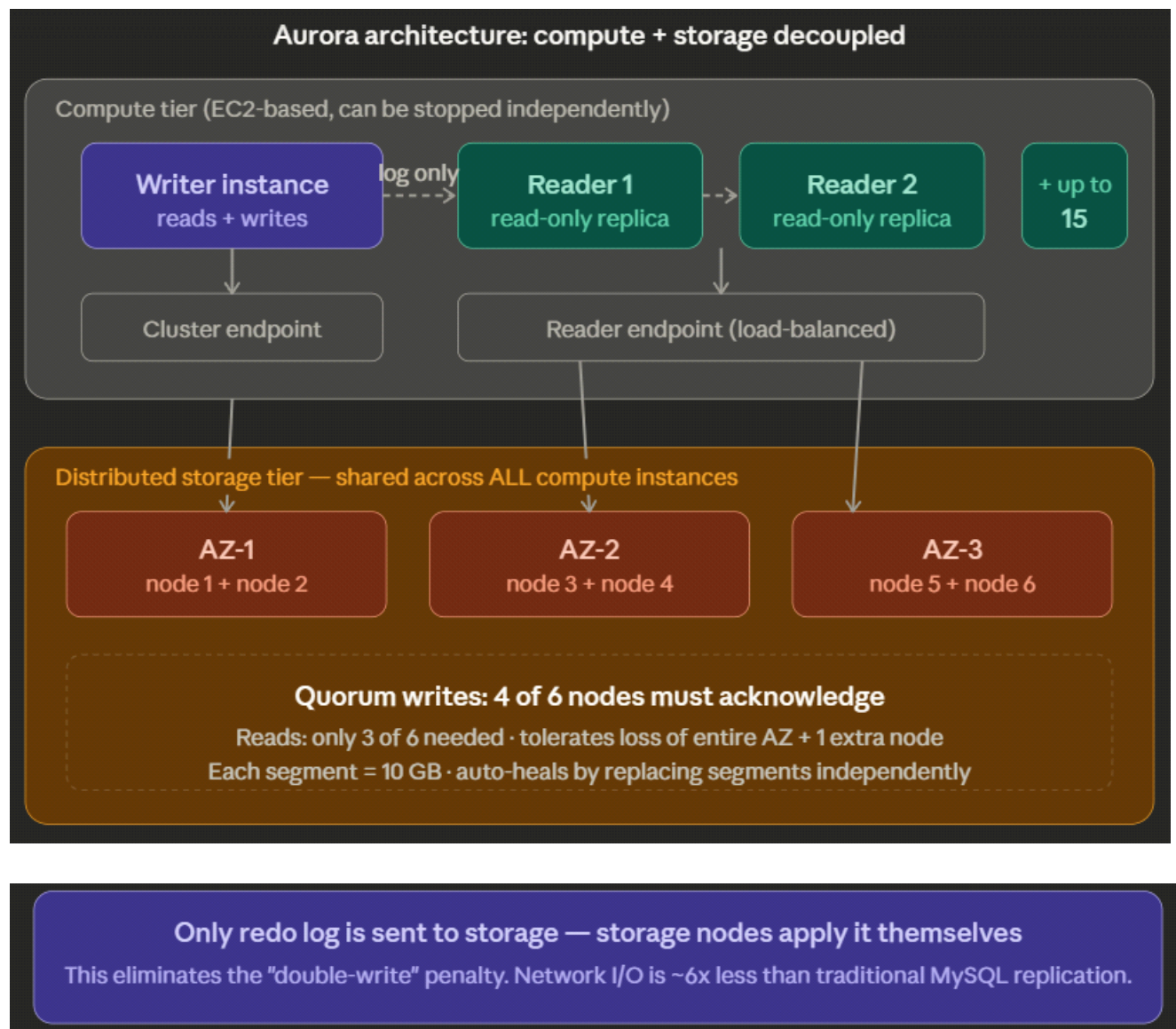
7. Global Content Delivery System

- **S3** for storage.
- **CloudFront CDN** for global edge caching.
- **Route 53** for DNS routing.
- **Replication** for regional resilience.
🔗 Interview phrasing: *"S3 + CloudFront = scalable, low-latency global delivery."*

8. Optimizing S3 Performance at Scale

- **Multipart Uploads:** For large files.
- **Parallelization:** Distribute workloads across prefixes.
- **S3 Byte-Range Fetches :** Request specific byte ranges for a file.
- **Transfer Acceleration:** Faster uploads via edge locations.
- **Requester Pays:** Cost control for shared datasets.
- **Monitoring:** CloudWatch + S3 Storage Lens.
🔗 Key insight: *"Performance scaling comes from parallelism and smart key distribution."*

Claude



Aurora separates the compute plane from the storage plane, and only ships redo log records to storage — not dirty pages. Storage nodes reconstruct data pages from the log themselves. This is what enables sub-10ms replica lag and near-instant failover.

Now the decision framework — when to choose which Aurora mode:

Aurora deployment modes — when to pick each			
Mode	Best for	Failover time	Scale model
Provisioned + read replicas	Steady high-traffic OLTP workloads	<30 sec (promotes existing replica)	Manual resize; up to 15 readers
Serverless v2 ACU-based	Variable / spiky traffic; dev/test	<30 sec (same as provisioned)	Auto scales in seconds per-ACU
Global DB cross-region	DR + geo-local reads globally	<1 min regional failover (RPO ~1s)	1 primary + up to 5 secondary regions
Multi-master deprecated	Was: write HA within region	AWS deprecated this — use Aurora Limitless for write scale now	
Limitless sharded writer	Millions TPS, petabyte-scale	Same as provisioned	Auto-shards across distributed routers
SA decision rule: Serverless v2 ≠ "free scaling" — it scales compute, not connections Use RDS Proxy in front of any Aurora to handle connection pooling at scale			

Now the most interview-critical comparison — Aurora vs RDS vs Aurora Serverless — the one that trips up most SA candidates:---

Aurora vs RDS — SA-level comparison			
Dimension	Aurora	RDS (MySQL/PG)	When RDS wins
Storage growth	Auto 10GB→128TB	Pre-provisioned	Predictable size
Replica lag	<10ms typical	Seconds to minutes	Lag is acceptable
Failover time	<30 sec	1-2 min typical	Non-critical apps
Cost	~20% more/vCPU	Lower base cost	Tight budget
Engine compat.	MySQL + PG APIs	Native engine	Edge-case compat.
Backtrack / PITR	Backtrack (in-place)	Restore to new DB	Budget-only

Aurora Backtrack: wind back the clock without restoring from backup — uniquely Aurora, frequently tested

Here's the complete conceptual breakdown an SA needs to articulate fluently:

The core architectural insight

Traditional MySQL replication ships **dirty pages** — every changed data block. Aurora ships only **redo log records**. Storage nodes apply the redo log themselves and materialize pages on demand. This reduces network I/O by roughly 6x, which is how Aurora achieves sub-10ms replica lag and sub-30 second failover while claiming up to 5x MySQL throughput and 3x PostgreSQL throughput.

Storage internals every SA must know

The storage volume is divided into **10 GB segments**, each replicated 6 times across 3 AZs (2 per AZ). Writes use a **4-of-6 quorum** — Aurora commits once 4 storage nodes acknowledge. Reads only need 3-of-6. This means the cluster tolerates loss of an entire AZ plus one additional node and **still serves both reads and writes**. When a segment degrades, Aurora heals it by replicating from a healthy peer — it never waits for the failed node to return. Storage grows automatically from 10 GB to 128 TB in 10 GB increments. You never provision storage upfront. You are billed only for what you use.

Endpoints — the one candidates always forget

Aurora has four distinct endpoint types that matter for architecture:

Cluster endpoint (writer endpoint): always points to the current writer. After

failover, DNS updates automatically. Your application should always use this for writes.

Reader endpoint: load-balances across all available read replicas. Round-robin by default. Use this for read-heavy query traffic.

Instance endpoints: connect to a specific instance. Use only for diagnostics or when you need to control exactly which replica handles a query.

Custom endpoints: you define a subset of instances. Use this when you have differently-sized replicas for different workloads — e.g. large instances for analytics queries, smaller ones for web-tier reads.

Serverless v2 — the nuance interviewers probe

Serverless v2 scales compute in **Aurora Capacity Units (ACUs)** — each ACU is approximately 2 GB RAM plus proportional CPU. You set a min and max ACU range; the instance scales within that range in under a second. This is genuinely different from Serverless v1 which had a cold start problem.

The trap: Serverless v2 scales compute, not connections. Lambda-based architectures calling Aurora Serverless v2 at scale will exhaust the connection limit long before they saturate CPU. The answer is always **RDS Proxy** — it pools connections and multiplexes them, so 10,000 Lambda functions share a pool of 100 actual database connections.

Global Database — the DR story

A Global Database has one primary region and up to 5 secondary regions. Replication lag to secondaries is typically **under 1 second** using dedicated replication infrastructure (not the public internet). The RPO in a regional failure is therefore ~1 second. RTO for a planned regional failover is under 1 minute. Secondary regions are read-only. You promote a secondary to primary in a DR event — this is a manual or automated operation but it does require DNS propagation. Write forwarding (sending writes from a secondary to the primary transparently) is now available but adds latency equal to the cross-region round-trip.

Aurora Backtrack — frequently tested, often missed

Backtrack rewinds the entire cluster to a point in time **without restoring from a snapshot**. It works because Aurora's storage is append-only — you can walk the redo log backward. Backtrack window is configurable (up to 72 hours). It is not a backup substitute; it's a fast undo for human error ("someone dropped a table"). Not available on Aurora Global Database clusters.

Aurora machine learning integrations

Aurora has native integration with SageMaker and Comprehend via SQL. You can call `aws_comprehend.detect_sentiment()` directly in a query. This is an SA differentiator — it shows you know Aurora is not just a faster RDS but a platform.

Connection handling — RDS Proxy

Any architecture with Lambda + Aurora must include RDS Proxy. Lambda's stateless, concurrent nature means each invocation may open a new connection. At scale this causes connection exhaustion. RDS Proxy sits between Lambda and Aurora, maintains a warm connection pool, and handles graceful failover — it retries connections transparently when Aurora fails over, reducing application-visible downtime.

Cost model to articulate

Aurora charges for: instance hours (per-second billing), storage (per GB-month), I/O (per million requests for provisioned; included in Aurora I/O-Optimized pricing), backup storage beyond the cluster size, and data transfer. Aurora I/O-Optimized pricing (newer tier) eliminates per-I/O charges and is typically cheaper for I/O-intensive workloads above ~25% I/O to storage ratio.

The SA answer template for "when would you choose Aurora?"

"I'd choose Aurora when the workload is relational, needs high availability with sub-30-second failover, has growing or unpredictable storage needs, or requires low-latency read replicas for read scaling. For truly spiky workloads like batch jobs or dev environments, Serverless v2 removes the need to right-size instances. For global applications needing disaster recovery with RPO under one second, Aurora Global Database is the right choice. I'd stay on standard RDS only if the budget is tight, if I need exact engine version compatibility for a specific MySQL or PostgreSQL minor version feature, or if the workload is stable enough that Aurora's overhead isn't justified."

Chatgpt

16 April 2026 12:14

From a **Solutions Architect interview** perspective, you don't need just features—you need to understand **how, when, and why to use Aurora**, plus trade-offs and architecture decisions.

What is Aurora?

 Amazon Aurora is a **cloud-native relational database** compatible with:

- MySQL
- PostgreSQL

But built for:

- High performance
- High availability
- Scalability

Core Architecture (Very Important)

Key Idea

Aurora separates:

- **Compute (DB instances)**
- **Storage (distributed, shared layer)**

Storage Layer

- Auto-scales from **10 GB → 128 TB**
- Data replicated **6 copies across 3 AZs**
- Self-healing (no manual replication setup)

Why This Matters

- Failover is fast (no data copy needed)
- Readers share same storage → no lag like traditional replicas

High Availability & Failover

- Writer + multiple read replicas
- Failover time: ~30 seconds

Types of Endpoints

- **Writer endpoint** → writes
- **Reader endpoint** → load balances reads
- **Custom endpoints** → control routing

Performance Advantages

- 5x faster than MySQL
- 3x faster than PostgreSQL

Why?

- No double buffering
- Log-based storage writes
- Parallel query execution

Scaling Options

1. Vertical Scaling

- Change instance size

2. Read Scaling

- Up to 15 read replicas
- Near real-time replication

3. Auto Scaling

- Add/remove replicas automatically

4. Serverless Mode

 Aurora Serverless

- Auto scales compute based on load
- Pay per usage
- Good for unpredictable workloads

Security

- Encryption at rest (KMS)
- Encryption in transit (TLS)
- IAM authentication
- VPC isolation

Backup & Recovery

- Continuous backup to S3
- Point-in-time recovery (PITR)
- Retention up to 35 days

Advanced Features (Interview Gold)

Aurora Global Database

- Cross-region replication (<1 sec lag)
- DR strategy

Backtrack

- Rewind DB without restore

Parallel Query

- Offloads query processing to storage layer

⚡ Zero-Downtime Patching

- Rolling updates

🤖 Aurora vs RDS (Key Question)

Feature	Aurora	RDS
Storage	Distributed	EBS
Performance	Higher	Standard
Replication	Built-in	Manual
Failover	Faster	Slower
Cost	Higher	Lower

🔑 When to Use Aurora

☑ Use Aurora if:

- High throughput OLTP system
- Need high availability
- Read-heavy workloads
- Global apps

✗ Avoid Aurora if:

- Small/simple app (cost not justified)
- No need for high scalability
- Tight budget

🔧 Real Architecture Example

E-commerce System

- Writer → handles orders
- Read replicas → product catalog
- Cache → Redis
- Aurora → primary DB

⚠ Trade-offs (Very Important for Interview)

✗ Cons

- Expensive
- Vendor lock-in
- Some MySQL/Postgres features missing

🤖 Common Interview Questions

1. How does Aurora achieve high availability?

👉 6 copies across 3 AZs + shared storage

2. How is Aurora different from MySQL?

☞ Storage + replication architecture

3. How do you scale reads?

☞ Reader endpoint + replicas

4. DR strategy?

☞ Global DB + cross-region failover

🧠 Pro-Level Insight (What Interviewers Love)

☞ Aurora is **not just a database—it's a distributed system**

- Storage is log-structured
- Compute is stateless
- Failover is metadata switch

🔑 Final Takeaways

- Separation of compute & storage = core concept
- Built-in replication = HA + performance
- Serverless = elasticity
- Global DB = DR

If you want, I can:

- Draw a **complete system design using Aurora**
- Compare **Aurora vs DynamoDB vs RDS (when to use what)**
- Or give **mock interview questions + answers for SA role**

From <<https://chatgpt.com/c/69dde409-d04c-83e8-9adf-1b7dbe1333b4>>

Amazon Aurora is a cloud-native, fully managed relational database service from AWS that's MySQL and PostgreSQL compatible, designed for high performance, scalability, and availability. As a solutions architect, you need to understand its architecture (shared storage, replicas, multi-AZ durability), operational features (auto-scaling, backups, failover), and cost/performance trade-offs.

Core Concepts

- **Architecture:**
 - Aurora clusters consist of **one primary (writer) instance** and up to **15 read replicas**.
 - All instances share a **cluster volume** that automatically replicates data across **3 Availability Zones with 6 copies** for durability.
 - **Compute is decoupled from storage**, reducing I/O bottlenecks.
- **Performance:**
 - Up to **15x faster than standard MySQL** and **3x faster than PostgreSQL**.
 - Storage auto-scales from **10 GB to 128 TB**.
- **Engines:** Supports **MySQL-compatible** and **PostgreSQL-compatible** modes.

Key Features for Architects

- **High Availability:** Automatic failover to replicas if the primary fails.
- **Scalability:** Read replicas for horizontal scaling; serverless option for auto-scaling compute.
- **Durability:** 11 9s (99.999999999%) durability with multi-AZ replication.
- **Managed Service:** Automated backups, patching, monitoring, and security updates.
- **Global Databases:** Cross-region replication for disaster recovery and low-latency reads.
- **Security:** Encryption at rest and in transit, IAM integration, VPC isolation.

Use Cases

- **High-performance apps:** SaaS platforms, e-commerce, analytics.
- **Mission-critical workloads:** Financial services, healthcare, government.
- **Global deployments:** Applications needing multi-region read/write capabilities.

Best Practices for Solutions Architects

- **Choose engine compatibility** based on existing workloads (MySQL vs

PostgreSQL).

- **Use replicas** for read scaling and failover.
- **Implement Aurora Serverless v2** for unpredictable workloads.
- **Apply IAM + KMS encryption** for compliance.
- **Plan cost optimization**: Monitor instance sizes, storage growth, and replica usage.
- **Design for resilience**: Use global databases for DR, and test failover scenarios.

⚠ Trade-offs & Challenges

- **Cost**: Aurora is more expensive than standard RDS, but offers better performance and resilience.
- **Complexity**: Requires careful planning for multi-region/global deployments.
- **Vendor lock-in**: Aurora is AWS-specific, though compatible with open-source engines.

🎯 Interview-Ready Talking Points

- **Explain architecture**: “Aurora separates compute from storage, replicates data across 3 AZs, and supports up to 15 replicas for scaling.”
- **Highlight resilience**: “Automatic failover and 11 9s durability make it ideal for mission-critical workloads.”
- **Discuss cost/performance trade-offs**: “Aurora costs more than RDS but delivers enterprise-grade scalability and availability.”
- **Mention global reach**: “Aurora Global Database enables cross-region replication for DR and low-latency reads.”

🗨 In short: For a solutions architect interview, focus on **Aurora’s architecture, scaling, durability, managed features, and trade-offs**. Be ready to explain how you’d design for **high availability, global reach, and cost optimization**.

Would you like me to create a **cheat sheet diagram** showing Aurora’s architecture (primary, replicas, shared storage, multi-AZ replication) so you can use it as a quick visual in interviews?

As a solutions architect preparing for interviews, you need to frame AWS Lambda not just as a coding tool but as a *strategic enabler* of serverless architectures—highlighting scalability, cost optimization, event-driven design, and integration with broader AWS services. Interviewers expect you to connect Lambda’s technical features to business outcomes.

Core Concepts of AWS Lambda (Architect’s View)

- **Serverless Compute:** Runs code without provisioning servers; scales automatically based on demand.
- **Event-Driven Model:** Triggered by AWS services (S3, DynamoDB, API Gateway, CloudWatch, etc.).
- **Pay-per-Use:** Billed for execution time and requests, making it cost-efficient for sporadic workloads.
- **Stateless Execution:** Each invocation is independent; state management requires external storage (e.g., DynamoDB, S3).
- **Concurrency & Scaling:** Lambda scales horizontally; architects must design for concurrency limits and throttling.

Architect-Level Responsibilities

- **Integration Patterns:**
 - API Gateway + Lambda → RESTful APIs
 - S3 + Lambda → Data processing pipelines
 - DynamoDB Streams + Lambda → Real-time event handling
- **Security Best Practices:**
 - Use IAM roles with least privilege
 - Encrypt environment variables with KMS
 - VPC integration for private resources
- **Monitoring & Observability:**
 - CloudWatch Logs & Metrics
 - X-Ray for tracing distributed requests
- **Optimization:**
 - Right-sizing memory for performance vs. cost
 - Avoid cold start latency with provisioned concurrency

Interview-Ready Comparison Table

Dimension	Lambda Strengths	Architect’s Considerations
Scalability	Auto-scales instantly	Manage concurrency limits

Cost	Pay only for execution time	Watch out for high-frequency triggers
Flexibility	Supports multiple languages (Python, Java, Node.js, Go, .NET)	Choose language based on ecosystem fit
Security	IAM integration, KMS encryption	Ensure least privilege & compliance
State Management	Stateless by design	External persistence required
Deployment	CI/CD via SAM, CDK, or Serverless Framework	Standardize deployment pipelines

🔗 Common Interview Questions (Solutions Architect Focus)

- 1. How does Lambda fit into a microservices architecture?**
→ Emphasize decoupling, event-driven workflows, and cost efficiency.
- 2. What are Lambda's limitations?**
→ Execution time (15 min max), package size limits, statelessness.
- 3. How do you handle cold starts?**
→ Provisioned concurrency, language choice (Node.js/Python faster).
- 4. When would you *not* use Lambda?**
→ Long-running tasks, heavy compute jobs, or applications needing persistent connections.
- 5. How do you secure Lambda functions?**
→ IAM roles, VPC integration, environment variable encryption.

⚠️ Risks & Trade-offs

- **Cold Start Latency:** Impacts user-facing APIs; mitigate with provisioned concurrency.
- **Vendor Lock-In:** Deep integration with AWS services may reduce portability.
- **Debugging Complexity:** Distributed event-driven systems require strong observability practices.
- **Cost Spikes:** High-frequency triggers can unexpectedly increase bills.

☑️ **Interview Tip for You, Subhasish:** Frame Lambda answers around *business impact*—how it reduces operational overhead, scales seamlessly, and integrates into resilient architectures. Use consulting-style language: “Lambda enables cost-optimized, event-driven solutions that align with modern cloud-native strategies.”

Would you like me to create a **mnemonic cheat sheet** (like “L-A-M-B-D-A” acronym) so you can recall Lambda's pillars instantly during interviews?

Chatgpt

⚡ What is AWS Lambda?

👉 AWS Lambda is a **serverless compute service** where you run code **without managing servers**, triggered by events.

🧠 Core Concept

- You upload code
- AWS runs it **on-demand**
- You pay only for execution time

👉 Fits into **event-driven architecture**

📺 How Lambda Works

Event → Lambda Function → Execution → Response

Common Triggers

- API calls → Amazon API Gateway
- File uploads → Amazon S3
- Streams → Amazon Kinesis
- Queues → Amazon SQS

⚙️ Key Characteristics

🔗 Stateless

- No memory between executions
- Use external storage (DB, cache)

⌚ Short-lived

- Max execution: **15 minutes**

📦 Event-driven

- Runs only when triggered

🚀 Scaling (Very Important)

💧 Automatic Scaling

- Lambda scales **horizontally automatically**
- Each request = separate execution environment

⚠️ Concurrency

- Default limit: ~1000 concurrent executions (can increase)

Cold Start

- Delay when function starts fresh

☞ Mitigation:

- Provisioned concurrency
- Keep functions warm

💰 Pricing Model

- Pay per:
 - Number of requests
 - Execution time (ms)
 - Memory allocated

☞ No cost when idle

🔄 Execution Model

Each invocation runs in:

- Isolated container
- With configured memory (128 MB → 10 GB)

🔒 Security

- IAM roles control access
- Runs inside VPC (optional)
- Supports encryption

🔗 Integration Patterns (Interview Gold)

1. Synchronous (Request/Response)

- API Gateway → Lambda

2. Asynchronous

- S3 → Lambda

3. Event Stream Processing

- Kinesis → Lambda

4. Queue-based (Decoupled)

- SQS → Lambda

🏗️ Common Architectures

🔧 Serverless Web App

- API Gateway → Lambda → Amazon DynamoDB

📁 File Processing

- S3 upload → Lambda → process image

📡 Event-driven System

- Event → Lambda → downstream services

⚠ **Limitations (Very Important)**

✗ **Not Suitable For:**

- Long-running jobs (>15 min)
- Stateful applications
- Heavy compute workloads

✗ **Cold Starts**

- Latency issue for low-traffic apps

✗ **Debugging Complexity**

- Distributed system challenges

🗯 **Lambda vs EC2 (Classic Question)**

Feature	Lambda	EC2
Server Management	None	Required
Scaling	Automatic	Manual
Billing	Per execution	Per instance
Control	Limited	Full

🗯 **Lambda vs Containers**

- Lambda → event-driven, short-lived
- Containers → long-running, more control

📖 **Best Practices**

☑ **Keep Functions Small**

- Single responsibility

☑ **Use Environment Variables**

- Config management

☑ **Handle Retries Properly**

- Avoid duplicate processing

☑ **Use Dead Letter Queues (DLQ)**

- Capture failed events

☑ **Monitor with CloudWatch**

- Logs, metrics, alarms

🗯 **Advanced Concepts (High-Value)**

⚡ **Provisioned Concurrency**

- Eliminates cold starts

⚡ **Lambda Layers**

- Share common code

⚡ **Step Functions**

👉 AWS Step Functions

- Orchestrate multiple Lambdas

⚡ **EventBridge**

👉 Amazon EventBridge

- Event routing between services

🔧 **Real Scenario (Interview)**

Image Processing System

- Upload → S3
- Trigger → Lambda
- Process → store in S3
- Notify → SNS

⚠️ **Trade-offs (Critical for SA Role)**

☑️ **Pros**

- No infra management
- Auto scaling
- Cost-efficient

✗ **Cons**

- Cold starts
- Execution limits
- Vendor lock-in

🗯️ **When to Use Lambda**

☑️ **Use If:**

- Event-driven workloads
- Microservices
- APIs
- Automation tasks

✗ **Avoid If:**

- Long-running processes
- High-performance computing
- Stateful systems

Key Takeaways

- Lambda = **event-driven, serverless compute**
- Scaling is automatic but **concurrency matters**
- Design for **stateless, short-lived execution**
- Integrates deeply with AWS ecosystem

Pro Interview Insight

 Lambda is not just compute—it's a **glue service** connecting AWS components.

If you want, I can:

- Do a **full system design using Lambda (end-to-end)**
- Compare **Lambda vs ECS vs EC2 vs Fargate**
- Or give **real SA interview questions with answers**

From <<https://chatgpt.com/c/69dde409-d04c-83e8-9adf-1b7dbe1333b4>>

Amazon EC2 (Elastic Compute Cloud) is AWS's core compute service, offering resizable virtual servers with flexible pricing, performance, and placement options. To master EC2, you need to understand its instance families, purchasing models, networking/placement strategies, lifecycle features (like hibernate), and monitoring metrics.



EC2 Instance Types

EC2 instances are grouped into **families**, each optimized for specific workloads:

- **General Purpose (T, M series):** Balanced compute, memory, networking.
- **Compute Optimized (C series):** High-performance processors for batch processing, gaming, scientific modeling.
- **Memory Optimized (R, X, z series):** Large in-memory databases, analytics.
- **Storage Optimized (I, D, H series):** High IOPS, big data, NoSQL.
- **Accelerated Computing (P, G, Inf series):** GPUs, ML training/inference, HPC.
- **Specialized (Mac instances, bare metal):** Niche workloads like iOS builds or hardware-level access.



Purchasing Options

AWS offers multiple cost models:

- **On-Demand:** Pay per second/hour, max flexibility.
- **Reserved Instances:** 1–3 year commitment, up to 75% savings.
- **Savings Plans:** Flexible commitment to usage, up to 72% savings.
- **Spot Instances:** Up to 90% cheaper, but can be interrupted.
- **Dedicated Hosts/Instances:** Single-tenant hardware for compliance/licensing.
- **Capacity Reservations/Blocks:** Guarantee availability in specific AZs.



Placement Groups

Placement groups control how instances are physically arranged:

- **Cluster:** Low-latency, high-bandwidth networking (HPC, ML).
- **Spread:** Instances across hardware racks for fault tolerance.
- **Partition:** Logical groups for large distributed systems (HDFS, Cassandra).



Lifecycle Features

- **EC2 Hibernate:** Saves RAM state to EBS, allowing fast resume (ideal for long-running apps).
- **Auto Scaling:** Dynamically adjusts instance count based on demand.
- **Elastic Load Balancing (ELB):** Distributes traffic across instances.

- **Elastic IPs:** Static IPs for consistent addressing.
- **AMI (Amazon Machine Images):** Templates for launching pre-configured instances.

Key Metrics to Monitor (via CloudWatch)

Monitoring EC2 ensures performance and cost efficiency:

- **CPU Utilization:** Detects under/over-provisioning.
- **Network In/Out:** Bandwidth usage, bottlenecks.
- **Disk I/O (Read/Write Ops):** Storage performance.
- **Status Checks (System/Instance):** Hardware or OS health.
- **Memory Utilization (via custom metrics):** Critical for memory-heavy workloads.
- **EBS Volume Metrics:** Latency, throughput, burst balance.
- **Spot Interruption Notices:** For fault-tolerant architectures.

Risks & Trade-offs

- **Spot Instances:** Cost-effective but risky for critical workloads.
- **Reserved/Savings Plans:** Require accurate forecasting; overcommitment wastes money.
- **Placement Groups:** Cluster improves performance but reduces fault tolerance.
- **Hibernate:** Not supported on all instance types; requires EBS-backed storage.

EC2 Instance Storage is a critical part of Amazon EC2 because storage directly impacts performance, durability, and cost. AWS provides multiple storage options, each suited for different workloads. Let's break them down:

EC2 Storage Options

1. Amazon EBS (Elastic Block Store)

- **Type:** Block-level storage volumes attached to EC2.
- **Features:**
 - Persistent (data survives instance stop/terminate if volume is retained).
 - Snapshots for backup and disaster recovery.
 - Encryption at rest and in transit.
- **Volume Types:**
 - **gp3/gp2 (General Purpose SSD):** Balanced price/performance.
 - **io2/io1 (Provisioned IOPS SSD):** High-performance, consistent latency.
 - **st1 (Throughput Optimized HDD):** Big data, streaming workloads.
 - **sc1 (Cold HDD):** Lowest cost, infrequent access.

2. Instance Store (Ephemeral Storage)

- **Type:** Local disks physically attached to the host.
- **Features:**
 - Extremely fast (low latency).
 - Temporary—data is lost when instance stops/terminates.
- **Use Cases:** Caching, temporary buffers, scratch space.

3. Amazon EFS (Elastic File System)

- **Type:** Managed NFS file system.
- **Features:**
 - Shared, scalable file storage.
 - Accessible by multiple EC2 instances simultaneously.
 - Pay-per-use, auto-scaling capacity.
- **Use Cases:** Content management, web serving, shared development environments.

4. Amazon S3 (Simple Storage Service)

- **Type:** Object storage (not block-level).
- **Features:**
 - Highly durable (11 nines).
 - Infinite scalability.
 - Lifecycle policies, versioning, encryption.
- **Use Cases:** Backups, static content, big data lakes.

5. Amazon FSx

- **Type:** Managed file systems (Windows File Server, Lustre).
- **Features:**
 - FSx for Windows: SMB protocol, Active Directory integration.
 - FSx for Lustre: High-performance for HPC workloads.

Lifecycle Features Related to Storage

- **EC2 Hibernate:** Requires EBS-backed instances to save RAM state.
- **Snapshots:** Point-in-time backups of EBS volumes.
- **Elastic Volumes:** Resize or change volume type without downtime.

Key Storage Metrics (CloudWatch)

- **VolumeReadOps / VolumeWriteOps:** Number of I/O operations.
- **VolumeReadBytes / VolumeWriteBytes:** Data throughput.
- **VolumeQueueLength:** Pending I/O requests.
- **VolumeIdleTime:** Periods with no activity.
- **BurstBalance:** For gp2 volumes, tracks burst credits.
- **DiskReadOps / DiskWriteOps (Instance Store):** Local disk activity.

Risks & Trade-offs

- **EBS vs. Instance Store:** EBS is persistent but slower; Instance Store is fast but ephemeral.
- **Cost vs. Performance:** io2 volumes are expensive but necessary for mission-critical workloads.
- **Shared Storage:** EFS and FSx simplify management but add latency compared to local disks.

An Auto Scaling Group (ASG) in AWS is a service that automatically manages a fleet of EC2 instances to maintain application availability and optimize cost. It ensures that the right number of instances are running to handle load, scaling up when demand increases and scaling down when demand decreases.

Core Concepts of Auto Scaling Groups

- **Launch Template/Configuration:** Defines the instance type, AMI, key pair, security groups, and EBS volumes for new instances.
- **Desired Capacity:** The target number of instances you want running.
- **Minimum & Maximum Size:** Bounds for scaling (never fewer than min, never more than max).
- **Scaling Policies:** Rules that trigger scaling actions (e.g., CPU > 70% → add 2 instances).
- **Health Checks:** Monitors instance health (EC2 or ELB checks) and replaces unhealthy ones.
- **Lifecycle Hooks:** Allow custom actions during instance launch/termination (e.g., warm-up scripts).
- **Termination Policies:** Decide which instance to terminate first when scaling down (oldest, newest, closest to AZ balance).

Scaling Types

- **Dynamic Scaling:** Responds to metrics (CPU, memory, request count).
- **Scheduled Scaling:** Predefined scaling at specific times (e.g., business hours).
- **Predictive Scaling:** Uses ML to forecast demand and scale proactively.

Key Metrics to Monitor (CloudWatch)

- **GroupInServiceInstances:** Number of healthy instances.
- **GroupDesiredCapacity:** Target instance count.
- **GroupPendingInstances:** Instances being launched but not yet active.
- **GroupTerminatingInstances:** Instances being shut down.
- **CPUUtilization / RequestCountPerTarget:** Common triggers for scaling.
- **NetworkIn/Out:** Detects traffic spikes.

Use Cases

- **Web Applications:** Scale out during peak traffic, scale in during off-hours.
- **Batch Processing:** Add compute nodes when jobs queue up.
- **Disaster Recovery:** Maintain high availability across multiple AZs.

Risks & Trade-offs

- **Over-scaling:** Can lead to unnecessary costs if policies are too aggressive.
- **Under-scaling:** Poor user experience if thresholds are too conservative.
- **Warm-up Time:** New instances may take time to initialize; lifecycle hooks help mitigate this.
- **Stateful Apps:** Scaling is easier for stateless apps; stateful workloads need session persistence (e.g., via Redis or RDS).

Untitled

27 March 2026 01:25

[NGINX Explained - What is Nginx](#)



The N+1 Database Query Problem: A Simple Explanation and Solutions



N+1 Problem

If you work with databases and applications, you may have heard of the **N+1 problem**. It's a common performance issue that happens when an application makes too many database queries instead of fetching data efficiently. This can slow down your app and make it harder to scale.

What Is the N+1 Problem?

Imagine you have a list of students, and each student has multiple courses they are enrolled in. This is a one-to-many relationship because one student can have many courses.

Now, let's say you need to get all students along with their courses. If your application is not optimized, it might do the following:

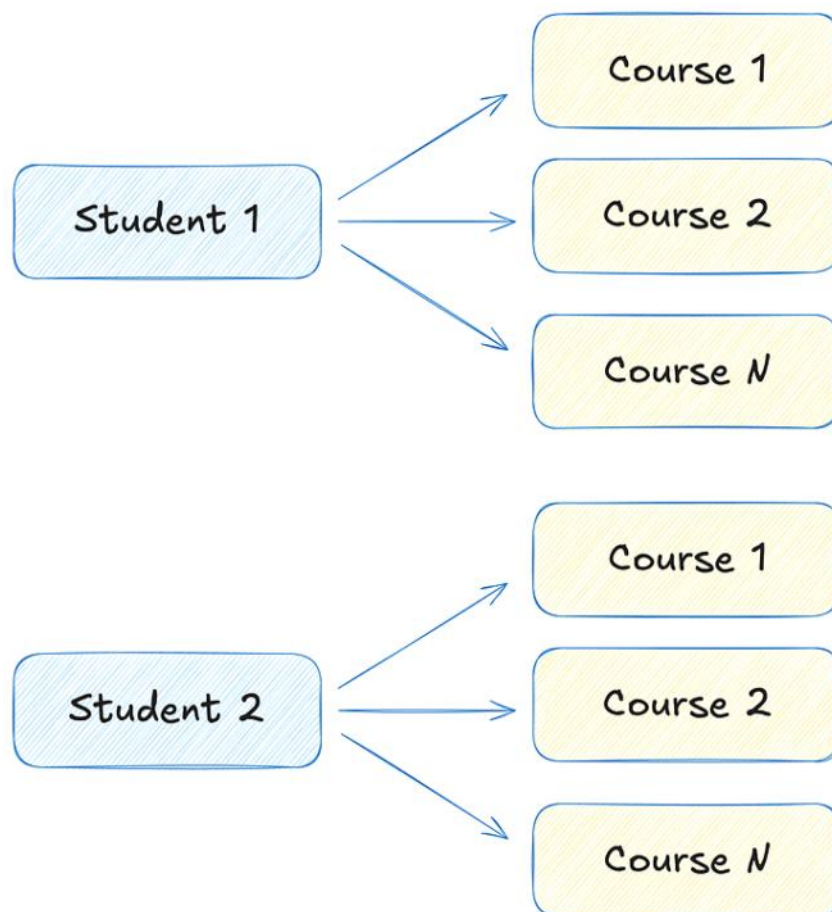
1. First, fetch all students:

```
SELECT * FROM students;
```

2. Then, for each student, fetch their courses one by one:

```
SELECT * FROM courses WHERE student_id = ?;
```

If you have 100 students, this results in 101 queries (1 query for students + 100 queries for courses). That's why it's called the **N+1 problem** — you run N (number of students) + 1 queries.



Why Is This a Problem?

The N+1 problem can seriously slow down your application for several reasons:

- **Too many database queries** overload the database server.
- **More network traffic** between your application and the database.
- **Slower response times** as the application waits for multiple

queries to complete.

- **Performance gets worse** as the dataset grows (e.g., 1,000 students would result in 1,001 queries!).

How to Fix the N+1 Problem

1. Use Joins to Fetch Data in One Query

Instead of making separate queries for each student's courses, use a JOIN to fetch all data in a single query:

```
SELECT students.*, courses.*  
FROM students  
LEFT JOIN courses ON students.id = courses.student_id;
```

This way, you only run one query instead of 101, making your application much faster.

2. Batch Queries Instead of Fetching One by One

If you don't want to use a join, another solution is **batch querying**:

```
SELECT * FROM courses WHERE student_id IN (1, 2, 3, 4, 5);
```

Instead of fetching courses one at a time, this query gets them all in one request. In this case we have just 2 requests, 1 for cars and 1 for students.

ORM Features to Avoid N+1

Many Object-Relational Mappers (ORMs) have built-in solutions to prevent the N+1 problem.

Example: **TypeORM in Node.js**

Using eager loading:

```
const students = await getRepository(Student)  
    .createQueryBuilder("student")  
    .leftJoinAndSelect("student.courses", "course")
```

```
.getMany();
```

This ensures all students and their courses are fetched in one query.

Using batch loading:

```
const students = await studentRepository.find({ relations: ["courses"] });
```

This fetches students and their courses in a single database request.

Example: **Hibernate in Java**

Using JOIN FETCH (JPQL / HQL)

This is often the most direct way to solve the N+1 problem. You explicitly tell Hibernate to fetch the associated entities in the same query using a JOIN FETCH clause.

```
// In your Spring Data JPA Repository
@Query("SELECT DISTINCT a FROM Author a JOIN FETCH a.books")
List<Author> findAllWithBooks();
```

Pros: Very efficient (single query). Standard JPA feature.

Cons: Can lead to Cartesian products if you JOIN FETCH multiple one-to-many or many-to-many associations in the same query (use separate queries or other strategies in such cases).

In Summary

The N+1 problem happens when an application makes too many small queries instead of fetching data efficiently. It slows down applications and makes them harder to scale. You can fix it using JOINS or batch queries.

Untitled

25 March 2026 11:08

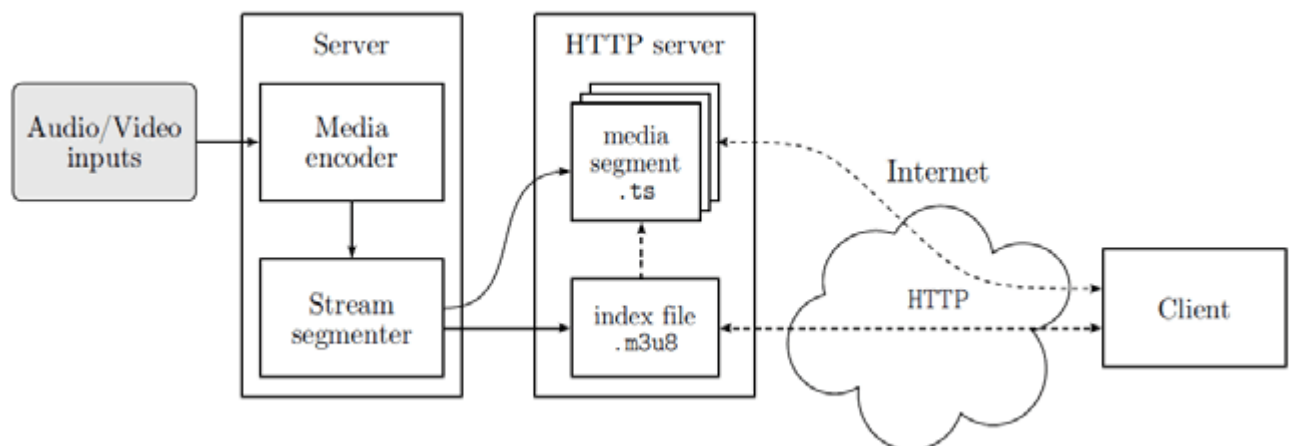
<https://medium.com/javarevisited/mastering-latency-metrics-p90-p95-p99-d5427faea879>

Streaming Protocols

09 March 2026 11:15

HTTP Live Streaming (HLS)

- **How It Works:** HLS works by breaking the video stream into small chunks called segments, which are then delivered to the viewer via HTTP. These segments are listed in a manifest file (playlist) that the client uses to request the video segments in sequence.
- **Technical Details:**
 - ▶ **Segments:** Video is encoded in multiple bitrates and divided into segments (usually 6-10 seconds each).
 - ▶ **Manifest File:** An **M3U8** file that lists the available streams and segments.
 - ▶ **Adaptive Bitrate Streaming:** Automatically switches between different bitrate streams based on the viewer's network conditions.
- **Key Features:**
 - ▶ **Scalability:** Efficiently handles a large number of concurrent viewers.
 - ▶ **Wide Compatibility:** Supported by most devices and browsers.
 - ▶ **Use Cases:** Ideal for VOD services like Netflix and live streaming platforms that require broad device compatibility.



Dynamic Adaptive Streaming over HTTP (DASH)

- **How It Works:** Similar to HLS, DASH also breaks the video stream into segments and delivers them via HTTP. However, it uses an **XML-based manifest** file called **MPD** (Media Presentation Description) to manage the stream.
- **Technical Details:**
 - ▶ **Segments:** Encoded in multiple bitrates and divided into segments.
 - ▶ **MPD File:** Lists the streams, segments, and their attributes.
 - ▶ **Codec Agnostic:** Can work with various codecs like H.264, H.265, and VP9.
- **Key Features:**

- ▶ **Flexible:** Works with various codecs, offering flexibility.
- ▶ **Interoperability:** Designed to work across different devices and platforms.
- ▶ **Use Cases:** Suitable for both live streaming and VOD, particularly in environments with diverse devices.

Real-Time Messaging Protocol (RTMP)

- **How It Works:** RTMP maintains a persistent connection and allows low-latency communication by streaming video data in small packets over TCP. This is done by breaking down video and audio data into smaller packets using an RTMP encoder. These packets are sent from the encoder to a streaming server, where they are stored and prepared for distribution.
- **Technical Details:**
 - ▶ **Persistent Connection:** Keeps a constant connection between the client and server.
 - ▶ **Chunks:** Video data is broken into smaller packets or chunks.
 - ▶ **Low Latency:** Delivers content with minimal delay.
- **Key Features:**
 - ▶ **Client to Server streaming:** Popularly used to stream video from the creator's device to streaming servers.
 - ▶ **Low Latency:** Delivers content with minimal delay.
 - ▶ **Interactive Streaming:** Supports features like live chat and audience interaction.
 - ▶ **Use Cases:** Commonly used for live streaming platforms such as Twitch and YouTube Live.

Real-Time Streaming Protocol (RTSP)

- **How It Works:** RTSP is used for establishing and controlling media sessions between endpoints. It is primarily used in IP cameras and surveillance systems.
- **Technical Details:**
 - ▶ **Session Control:** Allows control over playback functions like play, pause, and stop.
 - ▶ **TCP/UDP:** Can use both protocols for data transport.
 - ▶ **On-Demand Streaming:** Suitable for environments where real-time control over media is needed.
- **Key Features:**
 - ▶ **Network Adaptability:** Handles network variations efficiently.
 - ▶ **Control:** Provides fine-grained control over media sessions.
 - ▶ **Use Cases:** Ideal for CCTV, IP cameras, and other surveillance applications.

Secure Reliable Transport (SRT)

- **How It Works**: SRT uses UDP for data transmission, adding mechanisms for error correction, encryption, and packet loss recovery to ensure reliable and secure streaming.
- **Technical Details**:
 - ▶ **UDP-Based**: Faster transmission with lower latency than TCP.
 - ▶ **Error Correction**: ARQ (Automatic Repeat reQuest) and FEC (Forward Error Correction) for reliable delivery.
 - ▶ **Encryption**: AES encryption for secure transmission.
- **Key Features**:
 - ▶ **Secure Transmission**: Uses AES encryption for secure data transfer.
 - ▶ **Error Correction**: Mitigates packet loss and jitter for stable streaming.
 - ▶ **Use Cases**: Best for remote production and live streaming over public internet connections.

WebRTC

- **How It Works**: WebRTC enables peer-to-peer communication directly between browsers and devices, facilitating real-time video, audio, and data sharing.
- **Technical Details**:
 - ▶ **Peer-to-Peer**: Direct connection between clients, reducing server load.
 - ▶ **STUN/TURN Servers**: Used for NAT traversal and relay when direct connection is not possible.
 - ▶ **Signaling Server**: But browsers don't magically find each other. A separate signaling server acts as a matchmaker.
 - ▶ **DTLS/SRTP**: Encryption protocols for secure transmission.
- **Key Features**:
 - ▶ **Low Latency**: Provides near-instantaneous communication.
 - ▶ **Peer-to-Peer**: Facilitates direct connections, reducing server load.
 - ▶ **Use Cases**: Ideal for video conferencing and interactive applications like Google Meet and Zoom.

WebRTC HTTP Ingestion Protocol/HTTP Egress Protocol (WHIP/WHEP)

- **How It Works**: WHIP and WHEP are extensions of WebRTC designed to simplify the process of ingesting and delivering WebRTC streams using HTTP.
- **Technical Details**:

- ▶ **WHIP**: Simplifies ingestion of WebRTC streams by using HTTP for signaling.
- ▶ **WHEP**: Simplifies the delivery of WebRTC streams by using HTTP for signaling and media transport.
- ▶ **Seamless Integration**: Designed to integrate easily with existing WebRTC infrastructure.
- ▶ **Use Cases**: Suitable for large-scale live streaming and real-time communication platforms.

Resource

11 March 2026 11:05

<https://dev.to/varungujarathi9/understanding-streaming-protocols-2pg3>

1. The "Rate Limiter" for Communications APIs

The Prompt: Design a global rate-limiting service that handles millions of API requests per second across multiple regions.

- **Key Challenge:** How do you ensure consistency across regions without adding significant latency to the API call?
- **Discussion Points:** * Sliding window algorithms vs. Token bucket.
 - Redis for centralized state vs. local in-memory caches with eventual consistency.
 - Handling "thundering herds" when a specific API key is suddenly used by thousands of distributed clients.

DESIGN

Follow the below link

<https://www.tldraw.com/f/k5vAy9JIYQZ7yvdtBPebb?d=v-383.-1193.2226.1213.page>

Design High-Volume Notification Dispatcher

06 April 2026 14:46

High-Volume Notification Dispatcher

The Prompt: Design a system that receives a single "Broadcast" request and must deliver it to 10 million users via SMS or Push notification within 5 minutes.

- **Key Challenge:** Managing downstream provider limits (telecom carriers) and ensuring "exactly-once" or "at-least-once" delivery.
- **Discussion Points:**
 - Using **Kafka** or **RabbitMQ** for massive buffering.
 - Partitioning strategies to ensure horizontal scalability of "worker" nodes.
 - Dead Letter Queues (DLQ) for handling transient carrier failures.

DESIGN

Here's a comprehensive breakdown of the system design, followed by the architecture diagram.

The core challenge: 10M notifications in 5 minutes = ~33,000 messages/second. A single service can't do this — the answer is aggressive parallelism at every layer.

The key design decisions

Fan-out via partitioned queues. The broadcast API does almost nothing — it writes a single job record and publishes one event. The heavy lifting is done by workers pulling from a distributed queue (Kafka). You partition by user segment so 100s of workers process in parallel.

Pre-segmentation, not real-time lookup. At broadcast time, you don't query your user DB for 10M rows — that's too slow. Instead, user segments are precomputed and stored in a fast store (Redis or a dedicated segment service). Workers fetch pre-built batches of user IDs.

Channel routing. Each user has a preferred channel (push or SMS) stored in their profile. Workers fan out into two sub-queues — one for push (via APNs/FCM), one for SMS (via Vonage/Twilio). Push is cheaper and faster; SMS is the fallback for users without an active device token.

Backpressure and rate limiting. Telco SMS gateways throttle you. You need a rate-limiter layer in front of SMS workers that caps outbound RPS per gateway, with overflow spilling to a secondary gateway or a delay queue.

Delivery tracking via async writes. Workers don't write to your DB on every

send — that's 10M writes hammering your primary. Instead, they publish delivery events to a separate Kafka topic, and a lightweight consumer batches-upserts into Cassandra (optimized for high write throughput).> 💡 Each box is clickable — tap any component to deep-dive into it.



Capacity math (back-of-envelope)

Layer	Throughput needed	Design choice
Fan-out workers	~33K user IDs/sec	50 pods × 700 IDs/sec each
Push senders	~25K push/sec	30 pods, APNs supports 1000 req/conn
SMS senders	~8K SMS/sec	Capped by gateway; use 3+ gateways
Kafka partitions	100+	One consumer per partition = parallelism

Cassandra
writes

~33K/sec

Trivial for Cassandra; batch to reduce
ops

Follow-up angles the interviewer may probe

Idempotency — If a worker crashes mid-batch, some users get notified twice. Use a deduplication key (broadcast_id + user_id) checked before send.

Dead Letter Queue (DLQ) — After N retries, failed messages land in a DLQ. A separate process retries DLQ messages after a cooldown or pages an on-call engineer.

Priority lanes — If you need to guarantee SMS for high-value users before the rest, use a separate high-priority Kafka topic consumed first.

Thundering herd — Don't schedule all 10M at exactly T+0. Introduce jitter across fan-out batches so you don't spike APNs/Twilio simultaneously.

Vonage-specific angle — Since this is Vonage's question, mentioning their own **Messages API** (which abstracts SMS/WhatsApp/push) as the delivery layer shows domain awareness.

Followup questions

How do SMS sender workers respect gateway rate limits?

Great deep-dive question — rate limiting at the SMS layer is where most candidates go shallow. Here's the full picture.

The problem

You have potentially 8,000+ SMS/sec to send, but Vonage/Twilio impose per-account or per-number rate limits — typically in the range of **100–3,000**

SMS/sec depending on your tier and country. Blow past it and you get 429 Too Many Requests, messages fail, and your 5-minute SLA is gone.

You also can't just "slow down globally" — that stalls the whole pipeline. You need **per-gateway, per-number precision**.

The token bucket algorithm (the right answer)

Each SMS gateway gets its own token bucket:

Bucket capacity = burst allowance (e.g. 500 tokens)

Refill rate = gateway RPS limit (e.g. 300 tokens/sec)

Cost per message = 1 token

Before each send, the worker atomically acquires a token. If the bucket is empty, the worker waits (blocks briefly) or backs off. This is better than a fixed-window counter because it handles bursts gracefully — you can use accumulated tokens for a short spike without exceeding the sustained rate.

In practice, the bucket lives in Redis — shared across all SMS sender pods so limits are enforced cluster-wide, not per-pod.

EVAL lua_script KEYS[1]=gateway:vonage:tokens ARGV[1]=now ARGV[2]=rate ARGV[3]=capacity

A Lua script does the acquire atomically (no race conditions across pods).

Multiple gateways = parallelism + fallback

Don't rely on a single gateway. Stripe-route across 3+ gateways:

Gateway A → 3,000 SMS/sec (primary)

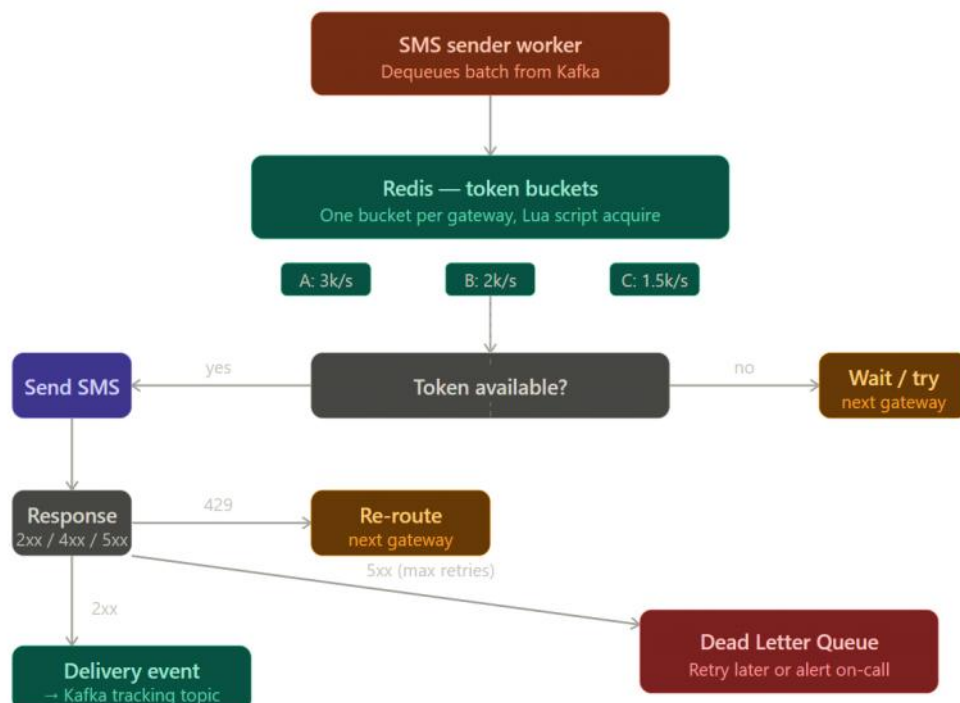
Gateway B → 2,000 SMS/sec (secondary)

Gateway C → 1,500 SMS/sec (overflow / fallback)

Workers pick a gateway based on **weighted round-robin** or least-loaded selection. If Gateway A returns 429 or a network error, the worker immediately re-routes to B — no retry delay wasted on a saturated endpoint.

The full worker flow

1. Dequeue batch of user IDs from SMS Kafka topic
2. For each user → look up phone number + locale
3. Acquire token from Redis bucket for target gateway
 - └ if bucket empty → wait up to 200ms, then try next gateway
4. Send HTTP request to gateway API
5. On 2xx → publish delivery event to tracking topic
 - On 429 → re-route to next gateway, re-enqueue
 - On 5xx → exponential backoff, max 3 retries
 - On fail → push to Dead Letter Queue (DLQ)



Key numbers to state in the interview

Token bucket parameters you'd tune:

- Burst capacity: $\sim 2\times$ the sustained rate (absorbs short spikes without 429s)
- Refill interval: every 10–50ms (not once per second — finer granularity = smoother traffic)
- Redis TTL on bucket keys: refresh on every acquire to avoid stale state

Backoff on 5xx: exponential with jitter — wait = $\min(2^{\text{attempt}} * 100\text{ms} + \text{random}(50\text{ms}), 5\text{s})$. Jitter prevents all workers retrying simultaneously (thundering herd on the gateway).

The interviewer's follow-up you should anticipate

"What if Redis goes down? Your token buckets are gone."

Answer: **local fallback buckets in each worker process** (in-memory, approximate). They won't be perfectly coordinated across pods, but they prevent a complete outage. You accept slight over-sending during Redis recovery — which is a better tradeoff than stalling 10M messages. Log the anomaly and reconcile afterward.

Design Distributed "Numbers" Inventory Management

06 April 2026 14:47

Distributed "Numbers" Inventory Management

The Prompt: Vonage manages millions of virtual phone numbers. Design a system where customers can search for, reserve, and buy numbers.

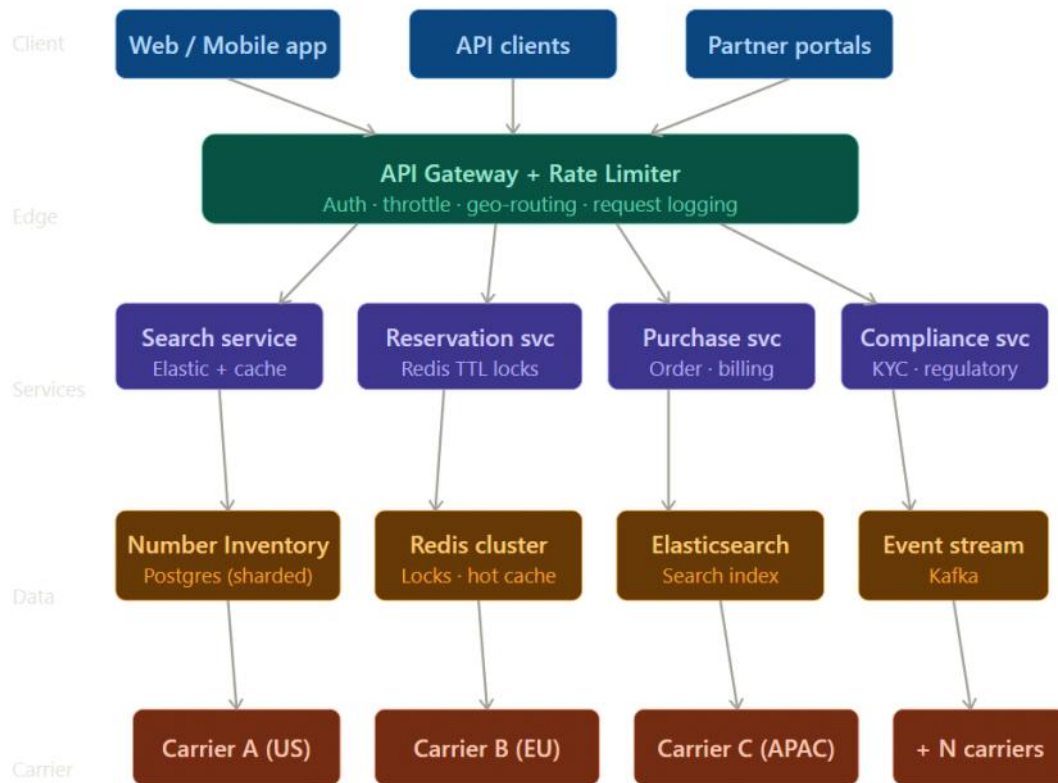
- **Key Challenge:** Preventing "double-booking" of the same number when two customers click "Buy" at the exact same millisecond in different parts of the world.
- **Discussion Points:**
 - **Database Isolation Levels:** Moving from Read Committed to Serializable or using Optimistic Locking.
 - State machine design for number lifecycles (Available -> Reserved -> Provisioned -> Released).
 - Handling distributed transactions across microservices (Saga pattern vs. 2PC).

DESIGN

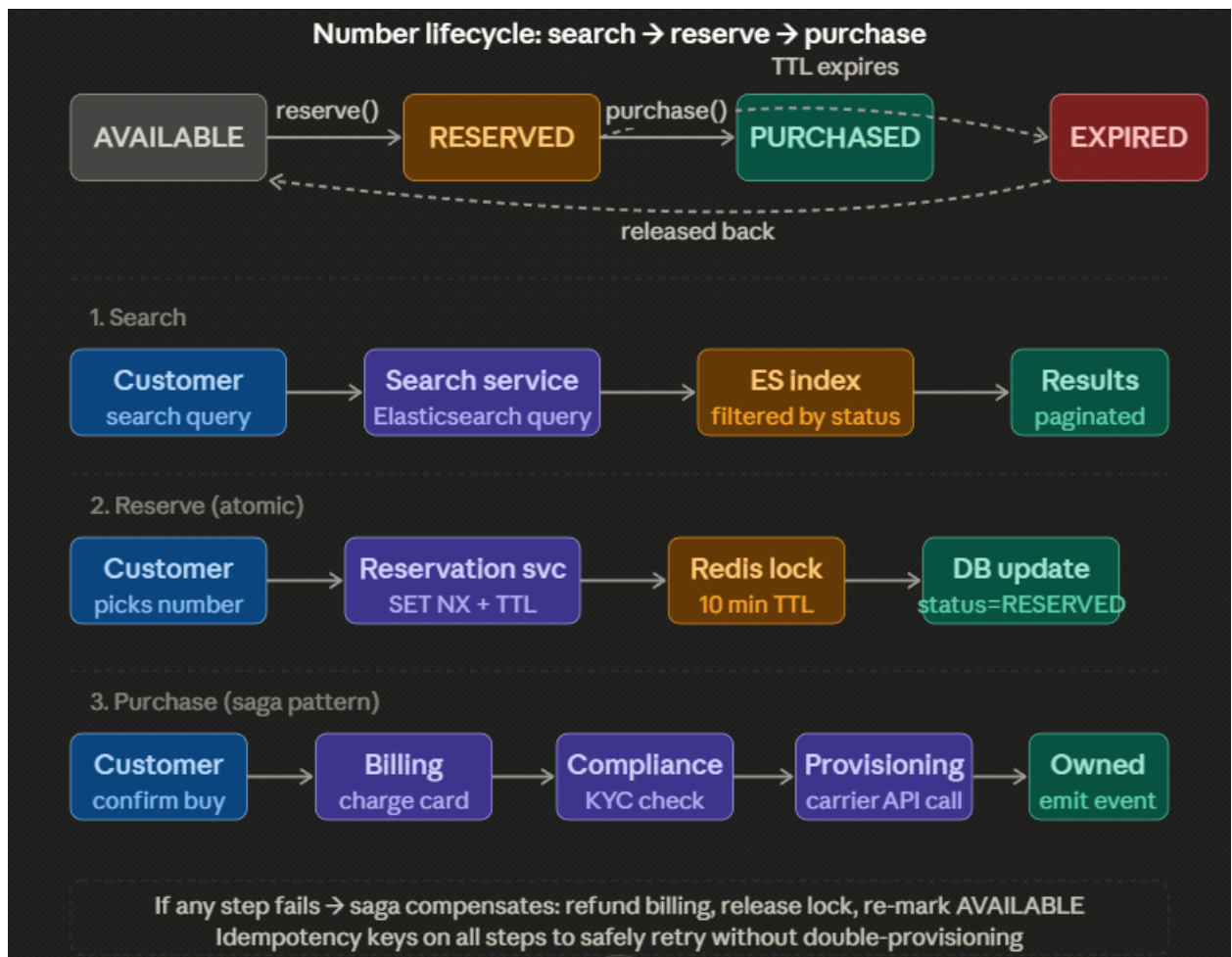
Here's a comprehensive system design covering all three layers — architecture, data model, and the end-to-end flows for search, reserve, and purchase.

The core challenge is that phone numbers are globally distributed finite resources with strict regulatory constraints, real-time availability requirements, and race conditions during reservation/purchase. The system must handle millions of concurrent searches while preventing double-selling and respecting country-specific compliance rules.

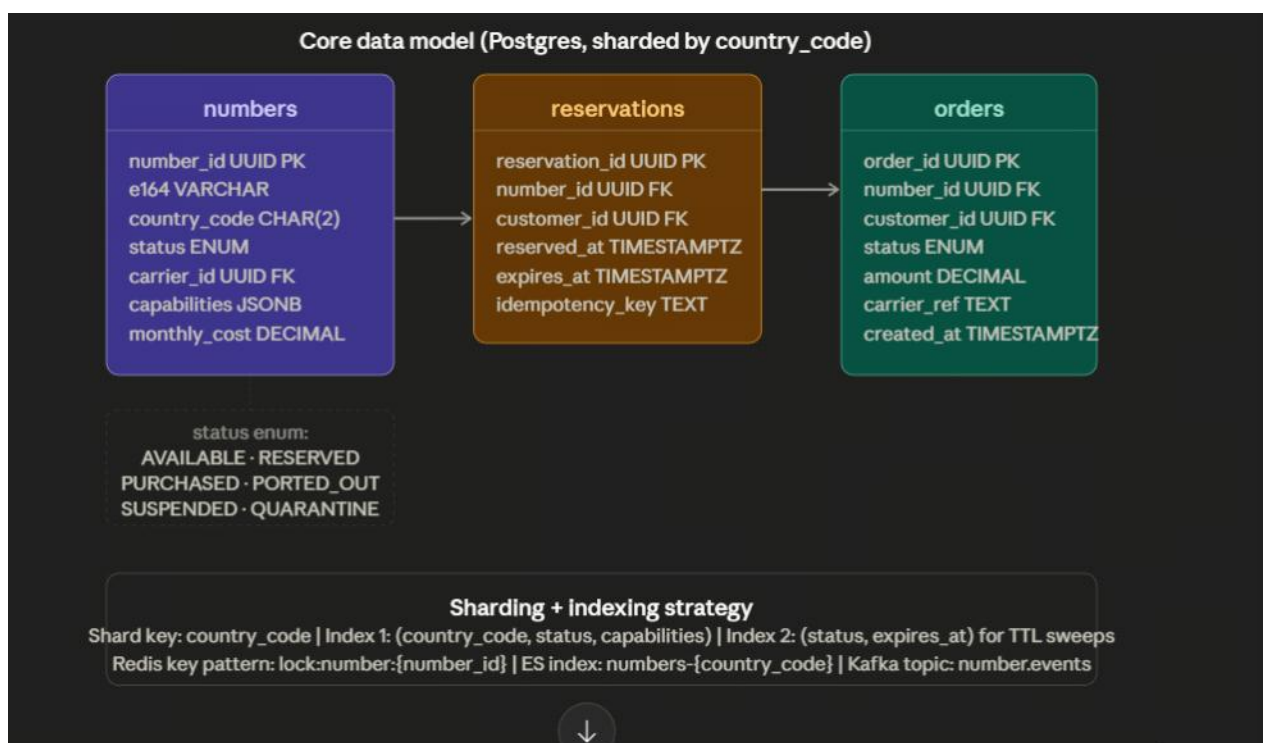
Let's start with the high-level architecture:



Now the critical flow — how search, reservation, and purchase interact, with the race condition prevention at the center:



Now the data model — the most interview-critical piece:---



Here's how to walk the interviewer through each design decision:

Search service — Elasticsearch is the right tool here because customers search

by country, area code, number pattern (e.g. vanity numbers like +1-800-555-HELP), capabilities (SMS/voice/fax), and price range. These are multi-dimensional full-text + filter queries. The ES index only contains AVAILABLE numbers. The key insight: ES is an eventually consistent read replica — it never owns the source of truth. The Postgres numbers table does.

Reservation and the race condition — This is the crux of the problem. Two customers searching simultaneously can both see the same number as available. You prevent double-reservation using Redis SET NX PX 600000 (set if not exists, with 10-minute TTL). Only one customer wins the atomic operation. The loser gets a 409 Conflict and is shown alternatives. After the Redis lock is set, you write status=RESERVED to Postgres synchronously. This is the "write-behind" pattern: Redis is the fast gate, Postgres is the durable record.

Purchase as a distributed saga — A purchase touches billing (charge the card), compliance (KYC for regulated countries like Germany or Brazil), and carrier provisioning (call the carrier API to actually activate the number). Any step can fail. You model this as a saga with compensating transactions: if carrier provisioning fails after billing succeeds, you issue an automatic refund. Idempotency keys on every step prevent double-charging if retries happen.

Compliance and regulatory constraints — This is a Vonage-specific complexity. Many countries (Germany, India, Brazil) require proof of address or business registration before a number can be assigned. The compliance service gates the purchase flow and can place a number in QUARANTINE status until documents are verified. This is a non-obvious extension most candidates miss.

Shard key choice — You shard Postgres by country_code because inventory is naturally partitioned by geography (US numbers and UK numbers never compete), carrier APIs are regional, and regulatory rules are country-scoped. This avoids cross-shard joins in the hot paths.

The TTL sweep job — A background scheduler queries reservations WHERE expires_at < NOW(), releases the Redis locks, and resets status=AVAILABLE in Postgres, then re-indexes to Elasticsearch. This keeps inventory fresh without manual intervention.

1. Multi-Tenant API Usage Dashboard

The Prompt: Design a real-time dashboard for Vonage customers to track their API usage, costs, and success rates across multiple products (SMS, Voice, Video).

- **The Challenge:** Data is generated by hundreds of microservices. The dashboard must support "drill-downs" (e.g., *Show me all failed SMS sent to India in the last 10 minutes*) with sub-second latency.
- **Managerial/Architectural Pivot:**
 - * **Data Pipeline:** Would you use a Pull or Push model for logs? (e.g., Kafka Streams vs. ELK stack).
 - **Storage:** OLAP vs. OLTP. How do you handle cost-effective storage for historical data (cold storage) vs. real-time data?
 - **Hands-on Check:** How do you ensure the dashboard doesn't impact the performance of the core message-sending path?

DESIGN

This is one of the richest system design problems for an EM interview — it spans data engineering, storage strategy, and production safety all at once. Let me give you both the architectural answer and the interactive dashboard to visualize it.

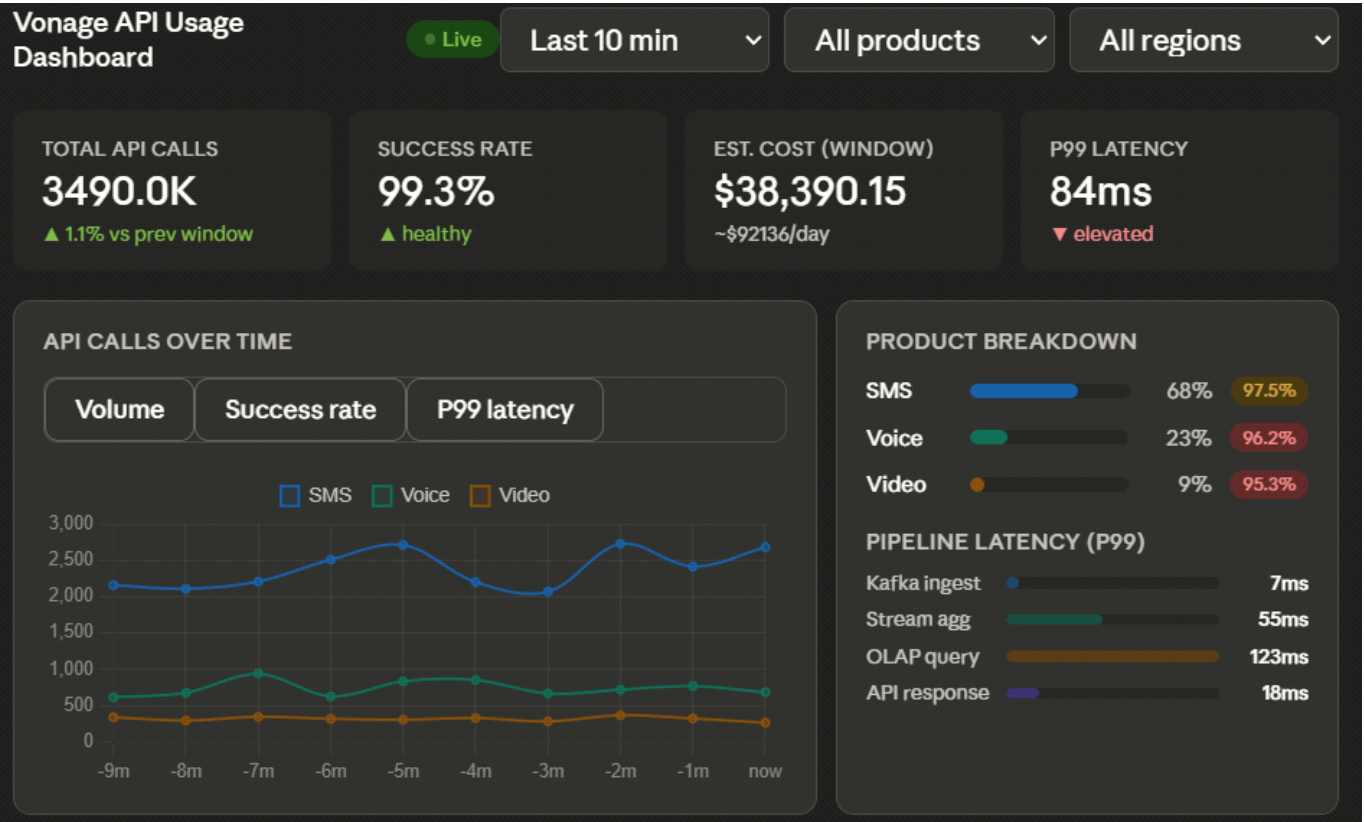
Here's the core architecture I'd pitch for this:

Data pipeline — Push (event-driven) over Pull. Pull models (ELK polling, scheduled scrapes) introduce latency floors you can't shrink. With hundreds of microservices, you need each service to emit structured events on every API call to Kafka. Kafka Streams then does windowed aggregation (5s tumbling windows) before landing in your query layer. ELK's Logstash is better suited for unstructured logs and debugging, not sub-second drill-downs.

Storage — tiered. Hot data (last 24h) lives in Apache Druid or ClickHouse — OLAP engines that can answer "failed SMS to India, last 10 min" in <200ms at scale. Warm data (last 30 days) stays in a columnar store like BigQuery or Redshift. Cold data (90d+) lands in S3/GCS Parquet with lifecycle rules. The key insight is that these aren't separate silos — a query router dispatches to the right tier based on the time range, transparently to the dashboard user.

Isolation from the core message path — this is the most critical constraint.

The write path should *never* be synchronous with the dashboard. The pattern I'd use: microservices write a tiny fire-and-forget event to a local Kafka producer (async, buffered), which decouples dashboard ingestion from the hot path entirely. The dashboard reads from replicas, never from the primary OLTP store. Circuit breakers on the Kafka producer side mean that if the analytics pipeline backs up, it sheds load gracefully without touching SMS/Voice latency.



DRILL-DOWN — RECENT FAILURES Deep dive ↗

All

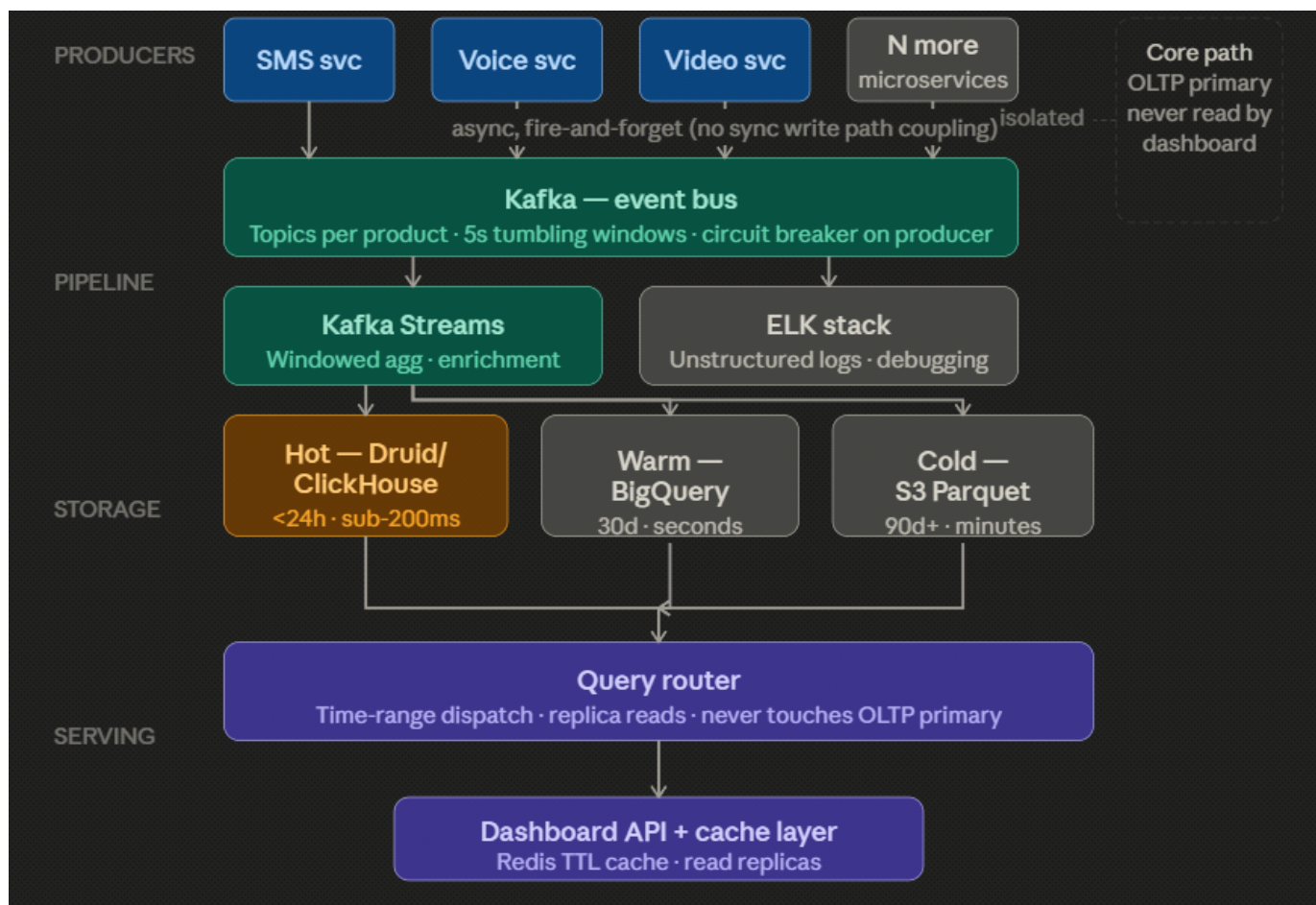
All regions

All statuses

TIME	PRODUCT	REGION	DESTINATION	STATUS	ERROR CODE	LATENCY	COUNT
00:02 ago	SMS	IN	+91-9820...	failed	CARRIER_REJECT	1420ms	847
00:04 ago	SMS	IN	+91-9876...	failed	TIMEOUT	5001ms	213
00:06 ago	Voice	US	+1-415...	degraded	RATE_LIMIT	890ms	52
00:07 ago	SMS	GB	+44-7900...	failed	ERR_5XX	320ms	19
00:08 ago	Video	DE	room:eur-west	degraded	NETWORK_DROP	2100ms	7
00:09 ago	SMS	BR	+55-11...	failed	AUTH_FAIL	290ms	34
00:10 ago	Voice	IN	+91-8012...	failed	CARRIER_REJECT	3200ms	128

Architecture note: These rows are served from hot OLAP storage (Druid/ClickHouse, <24h window). Queries targeting older data are automatically routed to warm columnar store. No query touches the core message-sending OLTP path.

Now let me give you the architecture diagram that ties the pipeline together:



Every box in both diagrams is clickable to spawn a follow-up question — useful for drilling into any layer during an interview discussion.

Here's how I'd frame the three pivot questions if asked directly:

Pull vs Push: Push every time for real-time. Pull models poll on a schedule — you're inherently bounded by polling frequency. With Kafka, each microservice emits a structured event (product, status_code, destination_country, latency_ms, timestamp) on every API call. Kafka Streams does 5-second tumbling window aggregations into your OLAP store. The key discipline is making the Kafka producer strictly async with a bounded queue — if Kafka is slow or down, the producer sheds events, not messages.

OLAP vs OLTP for storage: You never want the dashboard touching the OLTP primary. Instead, a three-tier approach: Druid or ClickHouse for hot data (last 24h, sub-200ms queries), BigQuery/Redshift for warm (30 days, seconds), and S3 Parquet with lifecycle rules for cold (90d+, minutes). A query router dispatches based on the requested time range, transparent to the UI. Druid's pre-aggregated rollups are particularly well-suited for the "failed SMS to India, last 10 min" drill-down pattern — you pre-roll up on (product, country, status_code, 1-minute bucket) and queries hit materialized results, not raw rows.

Isolation from the message path: Three layers of protection. First, the Kafka producer in each microservice is fire-and-forget with a bounded in-memory buffer — if the analytics pipeline falls behind, it drops events, never blocks the send path. Second, the dashboard reads from Druid replicas, not the OLTP

primary. Third, the dashboard API layer has a Redis TTL cache (30-second TTL for aggregate metrics) so that even a traffic spike on the dashboard doesn't translate to query load downstream. The core message-sending path has zero synchronous dependency on anything in the analytics stack.

Followup questions

You have used kafka here but kafka is pull based model. Doesnt it contradict your analogy here?

Answer:

Great catch — this is a really important distinction worth unpacking precisely. You're right that **Kafka consumers use a pull model** internally — consumers poll Kafka brokers for new messages. So let me clarify what I meant and where the terminology was imprecise.

The Push vs Pull debate in this context is about the *producer side*, not the consumer side.

The question "Push or Pull for logs?" is really asking: *do microservices actively push their events out, or does a central collector pull/scrape them?*

In a **Pull model** (e.g., ELK with Logstash polling, Prometheus scraping, or a collector cron-querying service logs), a central agent periodically reaches out to services and asks "what happened?" — you're bounded by the polling interval, so sub-second latency is impossible by design.

In a **Push model**, microservices proactively emit events the moment something happens — they push to a broker or endpoint. Kafka fits here because **your microservices push events into Kafka topics**. The fire-and-forget Kafka producer in each service is the push act. Yes, downstream consumers then pull from Kafka — but that pull happens continuously in near-real-time (millisecond poll intervals), not on a slow schedule.

So the precise framing should be:

- **Producer side → Push:** microservices push structured events into Kafka immediately on each API call.
- **Consumer side → Pull:** Kafka Streams consumers poll Kafka brokers, but at millisecond intervals, making it effectively real-time.

The latency advantage over ELK/Logstash polling comes from the producer-side push — events arrive in Kafka within milliseconds of occurring, rather than waiting for a collector to scrape them on a 30-60 second schedule.

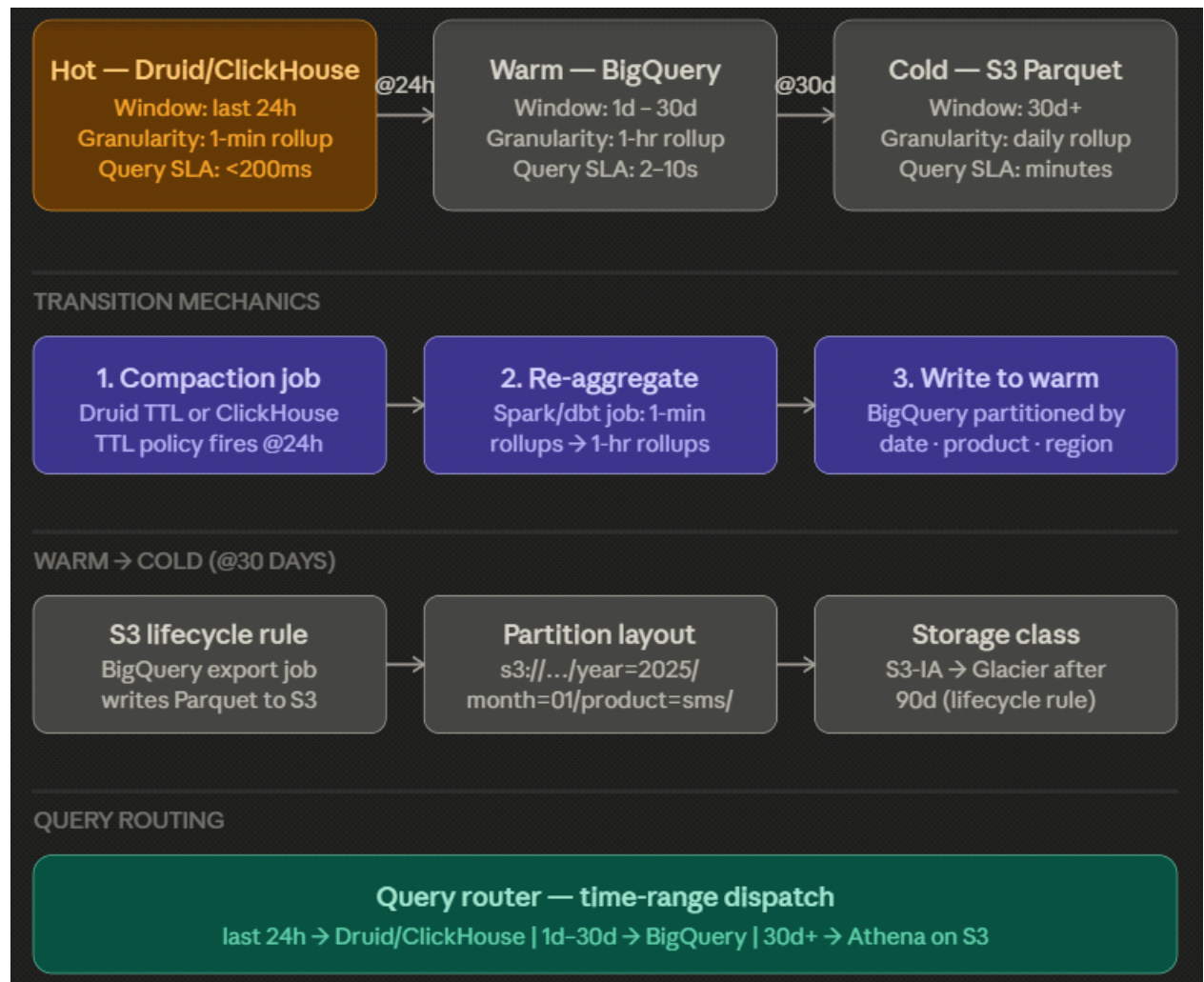
So Kafka doesn't contradict the push model — it *implements* push on the write path, while using an efficient pull mechanism internally for consumer coordination. A better way I should have phrased it: "Push-based event emission via Kafka" rather than just "Push model."

How the data transition happen between hot , warm and cold data storage?

Answer:

Great question — this is where a lot of candidates hand-wave and say "just use lifecycle rules" without explaining the mechanics. Let me break it down properly.

There are two distinct problems here: **how data moves** between tiers, and **how queries know where to look**. Here's the full picture:



Hot → Warm transition (@24h)

Druid and ClickHouse both support native TTL policies. When a data segment ages past 24 hours, a compaction job kicks off — this isn't just a file move. It re-aggregates: the 1-minute rollups stored in hot storage get collapsed into 1-hour rollups before landing in BigQuery. You intentionally lose fine granularity because nobody querying "last 28 days" needs per-minute precision — and 1-hour rollups are ~60x smaller, which matters enormously at Vonage's event volume.

Warm → Cold transition (@30d)

A scheduled export job (BigQuery → S3 as Parquet) runs nightly. The partition layout is critical: year=/month=/day=/product=/region= — this lets Athena prune partitions aggressively so a query like "all failed SMS in India in January"

only scans the relevant folders, not the entire dataset. After 90 days, an S3 lifecycle rule moves objects from Standard-IA to Glacier Instant Retrieval, cutting storage cost by ~70%.

The part most candidates miss — data overlap during transition

There's a window where the same data exists in both hot and warm simultaneously, intentionally. You don't delete from Druid the moment BigQuery receives the data — you keep a 2-4 hour overlap to handle late-arriving events (Kafka consumers that were lagging, retries, etc.). The query router knows about this overlap and deduplicates by preferring the hot store for any timestamp within the last 26 hours, falling back to warm only beyond that.

Query router logic (the glue)

The router inspects the requested time range from the dashboard and dispatches accordingly — Druid for last 24h, BigQuery for 1-30 days, Athena on S3 beyond that. Cross-tier queries (e.g. "last 45 days") fan out to both BigQuery and Athena, merge results in memory, and return a unified response. The dashboard never knows which tier served the data.

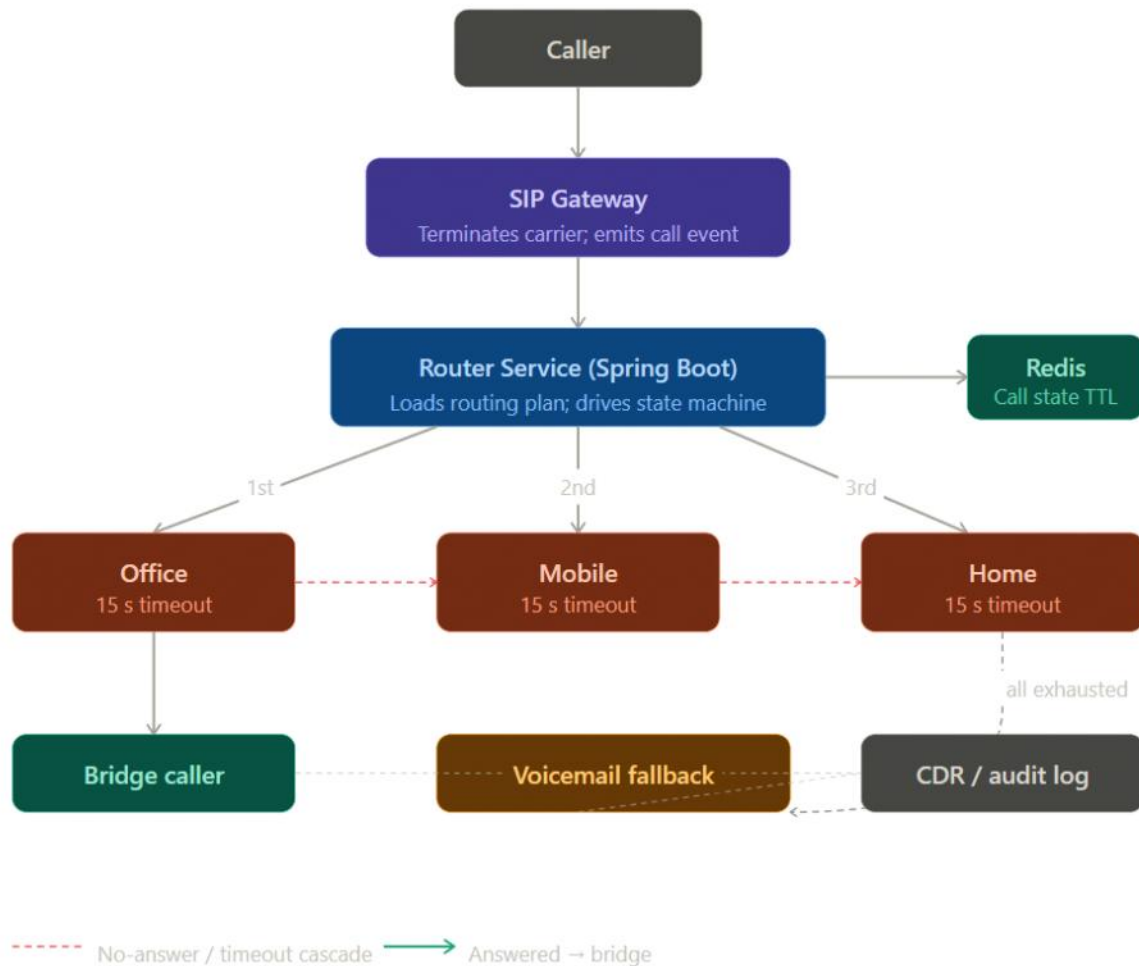
2. Global "Follow-Me" Voice Routing

The Prompt: Design a system that routes an incoming voice call to a sequence of numbers (e.g., Office -> Mobile -> Home) until someone picks up.

- **The Challenge:** Latency is the enemy. Every millisecond of delay in routing creates a "silent gap" for the caller.
- **Managerial/Architectural Pivot: * State Management:** How do you track the "call state" if the primary routing server fails mid-call?
 - **Concurrency:** In **Java/Spring Boot**, would you use Virtual Threads (Project Loom) or Reactive Programming (WebFlux) to handle thousands of simultaneous active call threads efficiently?
 - **Integration:** How do you handle a "no-answer" from a downstream carrier within a strict timeout window?

DESIGN

This is a genuinely rich design problem — let me build an interactive architectural diagram you can explore, then break down each challenge precisely. Now let's dig into each architectural challenge precisely.



State management: surviving a mid-call server crash

The core insight is that call state must be **externalized immediately** — before you touch the first carrier. The moment the SIP gateway emits the call event, your Router Service writes an atomic record to Redis:

```

json
{
  "callId": "uuid",
  "ani": "+918012345678",
  "plan": ["Office", "Mobile", "Home"],
  "currentLeg": 0,
  "legStartedAt": 1712345678000,
  "status": "ringing_office"
}

```

Set a TTL slightly longer than your maximum total routing window (e.g., 3 legs × 20s + buffer = 90s). If the primary router crashes mid-call, a standby picks up by reading this key, calculating elapsed time on the current leg, and either bridging or advancing. You get exactly-once leg advancement by using a Redis WATCH/MULTI/EXEC optimistic lock — only one standby wins the race to

advance state.

For Vonage specifically, their Answer URL and Event URL webhooks already externalize call state on their side. Your Redis record is the **mirror** that lets your application layer recover independently of Vonage's session.

Concurrency: Virtual Threads vs Reactive (WebFlux)

This is the most interesting trade-off for a Vonage EM interview.

Virtual Threads (Project Loom) are the right answer for call routing, and here's why: each active call is fundamentally a *sequential state machine* — dial leg 1, wait up to 15s, advance, wait, advance. The code reads as blocking imperative logic (`Thread.sleep`, `carrier.dial(...).get()`), which is easy to reason about, debug, and test. With virtual threads, you can have 50,000 concurrent waiting call threads at almost zero cost — they're parked in heap, not pinned to OS threads.

WebFlux/Reactive would force you to model that same sequential state machine as a Mono chain with `timeout()` operators and `flatMap` cascades. It's genuinely harder to reason about for stateful telephony flows, and the debugging experience with reactive stack traces is painful when a carrier SIP response arrives 12 seconds into a `timeout(15s)` window.

The rule of thumb: use reactive when you have **fan-out without state** (broadcasting events to N subscribers). Use virtual threads when you have **sequential state with waiting** (which is exactly call leg progression).

// Virtual thread: readable, debuggable, correct

```
java
// Virtual thread: readable, debuggable, correct
executorService.submit(() -> {
    for (String leg : routingPlan) {
        var result = carrier.dial(leg, Duration.ofSeconds(15)); // parks, not blocks OS
        redisState.advance(callId, leg, result);
        if (result == ANSWERED) { bridge(callId); return; }
    }
    voicemail.route(callId);
});
```

Handling carrier no-answer within a strict timeout window

The latency enemy here is **carrier silence** — some carriers return 180 Ringing but never follow up with 200 OK or 486 Busy. You cannot trust them to respect your timeout. Your options:

Dual-clock enforcement: Start your own `ScheduledExecutorService` timer the moment you send the INVITE. Don't wait for the carrier's 408. When your timer fires, send a CANCEL yourself and immediately issue the next leg's INVITE. This eliminates the carrier's contribution to gap latency.

Silent pre-dial (parallel seizing): For zero perceived gap, dial the next leg 1–2 seconds *before* the current timeout expires, hold it in a provisional state, and connect it immediately on timeout. This consumes two carrier legs briefly but the caller hears no silence. Vonage's API supports this via early media handling.

SIP 183 Session Progress detection: If the current leg sends 183 (early media / ringback), you know a human might still answer. If it doesn't within 3–4 seconds of 180 Ringing, treat it as a dead leg and begin parallel pre-dial.

The **silent gap budget** in practice is: your timer fires (0ms) → CANCEL sent → carrier ACKs (50–150ms network) → next INVITE sent → carrier processes (50–200ms) → ringback to caller (50ms). You're looking at 150–400ms of silence. Pre-dialing gets that to effectively 0ms perceived.

This is a system where the interviewer will probe whether you've *felt the pain* of carrier variance — the candidate who mentions dual-clock enforcement and pre-dialing rather than just "set a timeout" immediately signals production experience.

3. Webhook Delivery Service with Exponential Backoff

The Prompt: When an SMS is delivered, Vonage sends a "Delivery Receipt" webhook to the customer's URL. Design a system that ensures these webhooks are delivered even if the customer's server is down.

- **The Challenge:** Customers have different server capacities. You cannot overwhelm a small customer with 10,000 concurrent webhooks, but you must deliver at high speed for enterprise clients.
- **Managerial/Architectural Pivot:** * **Retries:** Implementation of **Exponential Backoff with Jitter** to prevent a "thundering herd" on customer servers.
 - **Isolation:** How do you prevent one "slow" customer's webhooks from clogging the queue for all other customers? (Hint: Sharded queues or priority lanes).
 - **Security:** How do you sign the payload to ensure the customer knows the webhook truly came from [Vonage](#)?

DESIGN

This is a classic distributed systems design problem, and a great one for an EM interview. Let me break it into its four interlocking concerns — reliability, isolation, throughput control, and security — then visualise the full architecture.

The core idea is that you're not building a simple queue; you're building a *per-customer fair-use delivery engine*. Each concern compounds the previous one.



Now let me walk through each design decision:

1. Reliability — exponential backoff with full jitter

The retry formula is $\text{delay} = \text{random}(0, \min(\text{cap}, \text{base} \times 2^n))$. The $\text{random}(0, \dots)$ part is the full jitter — it desynchronises all retrying jobs so they don't all fire at the exact same second after a customer's server recovers. Without jitter you get the "thundering herd": every webhook that failed at $T+0$ retries at $T+30\text{s}$ in lock-step, hitting a just-recovering server all at once and knocking it down again.

Critically, retries are **re-enqueued** rather than sleeping. A worker that sleeps blocks a thread; a worker that pushes a delayed message back to the queue is free to process other customers immediately.

2. Isolation — sharded queues per customer tier

One slow customer (their server is taking 30 seconds per response) must not block the queue for everyone else. The solution is per-customer queue shards: the router hashes on `customer_id` and places messages into that customer's dedicated sub-queue. Workers are bounded per customer — a standard customer might have a concurrency cap of 50, so even if they have 100,000 pending webhooks, only 50 are in-flight at once. This is sometimes called a **bulkhead pattern**: one flooded compartment doesn't sink the ship.

Enterprise customers get a negotiated higher concurrency limit (e.g. 500). Trial customers are capped very low (e.g. 5) and put in an isolated queue so their slow servers can't affect standard customers either.

3. Security — HMAC payload signing

Every webhook carry a Vonage-Signature header of the form:

Vonage-Signature: t=1712345678,v1=<hmac>

Where `hmac` = HMAC-SHA256(customer_secret, timestamp + "." + raw_body).

Including the timestamp prevents **replay attacks** — the customer's server should reject any webhook where $|\text{now} - t| > 300\text{s}$. The customer validates by computing the same HMAC with their shared secret and comparing it to `v1`.

This proves the payload genuinely came from Vonage and hasn't been tampered with in transit.

Signing happens in the **ingest service**, not per-retry, so the signature is stable across all delivery attempts for the same event.

4. Capacity control — the DLQ and observability

After N attempts (5 is typical) a message moves to a **dead-letter queue**. This triggers an alert and a customer-facing dashboard event. Operators can replay DLQ messages manually once the customer's server is confirmed healthy. The DLQ also surfaces systemic patterns — a customer whose DLQ is constantly filling up is a signal to proactively reach out or throttle their inbound traffic.

Key architectural tradeoffs to mention in the interview:

- *Push-only vs pull*: this design is push-based. An alternative is to let customers poll for missed webhooks, which is simpler but breaks the real-time contract most customers expect.
- *Queue technology*: SQS with per-customer visibility timeouts works; Kafka with per-customer partitions works better at very high throughput but is operationally heavier. For Vonage's scale, Kafka is the likely answer.
- *Idempotency*: the customer's endpoint must handle duplicate delivery (retry N might succeed even though retry N-1 actually delivered). Vonage should send a stable `webhook_id` so customers can deduplicate.

Dealing With Contention

05 May 2026 22:10

Managing Long Running Tasks

05 May 2026 22:12